Scalable Vector Machines.
RVV Design and Architecture
from an 8-bit bus to a 512-bit register.

BTRON BX
Cryptographic Telesystems, LLC.

November 2026

Contents

Contents **i**

Preface **vii**

A Note to the RTL Reader **ix**

Notation and Conventions **xi**

1 Introduction **1**
1.1 How to read this book . . . 1
1.2 Three shapes of data . . . 1
1.3 The blocks . . . 2
1.4 Four streams . . . 2
1.5 Fixed width and scalable length . . . 2
1.6 Register windows . . . 3
1.7 Arbitrary length . . . 3
1.8 Arrays and vectors . . . 3
1.9 Chaining . . . 4
1.10 Static and dynamic instructions . . . 4
1.11 Ports, stalls, and issue . . . 5
1.12 Grouping vector registers . . . 5
1.13 Instruction classes . . . 5
1.14 The host adaptor . . . 6

2 Machines That Remain MSX and Amiga **9**
2.1 The place of the original machines . . . 9
2.2 What compatible means in RTL . . . 9
2.3 Why the peripherals survive the change of processor . . . 10
2.4 The machine contract . . . 10

3 The RISC-V Ladder, from an 8-bit Bus to a 64-bit Element **11**
3.1 Datapath width is not XLEN . . . 11
3.2 One decode for scalar and vector . . . 11
3.3 Latency you should publish . . . 12
3.4 What moves upward unchanged . . . 12

4 SVM Profiles and the Ratified Vector Extensions **13**
4.1 The names . . . 13
4.2 Why SVM-8 is not Zve32x . . . 13
4.3 The `misa` rule . . . 14
4.4 Xsvm . . . 14
4.5 Fixed point without a private lane type . . . 14

5 Vector State **15**
5.1 The register file . . . 15
5.2 Configuration registers . . . 15
5.3 Legality at configuration time . . . 16
5.4 Tail and mask policy . . . 16
5.5 Precise traps . . . 16

6 The Instruction Classes **17**
6.1 A minimum that is still RVV . . . 17
6.2 Configuration . . . 17
6.3 Loads and stores . . . 17
6.4 Integer arithmetic . . . 17
6.5 Reductions, masks, and permutes . . . 18
6.6 Floating point . . . 18
6.7 Custom geometry . . . 18
6.8 Instructions you will be tempted to skip . . . 18

7 Lanes, Register Files, and Memory on an FPGA **19**
7.1 Three widths . . . 19
7.2 The Intel register ladder . . . 20
7.3 How many lanes . . . 21
7.4 The register file . . . 21
7.5 Chaining . . . 22
7.6 Memory width . . . 22
7.7 What to read in other people's cores . . . 22

8 RV32: The 32-bit Hart **23**
8.1 The integer file . . . 23
8.2 The address . . . 23
8.3 What crosses into the vector unit . . . 23
8.4 Fetch . . . 24

9 BX-8: An MSX-Compatible Machine with SVM-8 **25**

9.1 The board-level picture . . . 25
9.2 The hart . . . 25
9.3 SVM-8 . . . 26
9.4 Opcode map for SVM-8 . . . 26
9.4.1 Configuration — `vsetvli` . . . 26
9.4.2 Unit-stride load — `vle8.v` . . . 26
9.4.3 Integer arithmetic (OPIVV / OPIVX) . . . 27
9.4.4 Compare (mask result) . . . 27
9.4.5 Move and splat . . . 28
9.4.6 Mask logicals . . . 28
9.5 R800 fetch . . . 28
9.6 Assembler kernels that must run . . . 29
9.6.1 Where these kernels run . . . 29
9.6.2 fill8 — fill a region with a byte . . . 30
9.6.3 copy8 — unit-stride byte copy . . . 30
9.6.4 mask8 — selective fill under a generated mask . . . 30
9.6.5 add8 — element-wise byte add with saturation . . . 30
9.7 What you elaborate . . . 30

10 BX-16: An Amiga-Compatible Machine with SVM-16 35
10.1 The board-level picture . . . 35
10.2 SVM-16 . . . 35
10.3 Widening, not a private fixed-point format . . . 35
10.4 Xsvm . . . 36
10.5 Opcode map for SVM-16 . . . 36
10.5.1 Configuration . . . 36
10.5.2 Unit-stride load/store (EEW = 8 and 16) . . . 36
10.5.3 Constant-stride load — `vlse16.v` . . . 36
10.5.4 Integer arithmetic (SEW = 8, 16) . . . 37
10.5.5 Widening multiply and narrowing . . . 37
10.5.6 Reduction and slide . . . 37
10.5.7 Optional Xsvm (custom-0) . . . 37
10.6 68000, SH-2, and SH-4 fetch . . . 38
10.7 Assembler kernels that must run . . . 38
10.7.1 Where these kernels run . . . 39
10.7.2 sew16 — halfword fill . . . 39
10.7.3 vwmul16 — widening multiply then narrow with saturation . . . 39
10.7.4 stride16 — constant-stride gather of halfwords . . . 39
10.7.5 redsum16 — integer reduction . . . 41
10.7.6 sad16 — sum of absolute differences . . . 41
10.7.7 sw8 — one Smith–Waterman stripe . . . 41
10.8 Harts . . . 41
10.9 Elaboration . . . 42

11 BX-32: Embedded Floating Point with Zve32f 45
11.1 The architecture . . . 45
11.2 Floating point in the lane . . . 45
11.3 Where SVM-16 software feels the change . . . 46
11.4 Opcode map for Zve32f (BX-32) . . . 46
11.4.1 Configuration . . . 46
11.4.2 Unit-stride, stride and indexed load/store . . . 46

11.4.3 Integer arithmetic (SEW = 8, 16, 32) . . . 46
11.4.4 Floating-point arithmetic (SEW = 32) . . . 47
11.4.5 Mask and move . . . 47
11.5 MIPS fetch . . . 47
11.6 Assembler kernels that must run . . . 48
11.6.1 sew32 — word fill . . . 48
11.6.2 fma32 — fused multiply-add strip . . . 48
11.6.3 gemm32 — outer product of a 4-wide panel . . . 48
11.6.4 index32 — indexed load (gather) . . . 50
11.6.5 wmul32 — element-wise window . . . 51
11.6.6 bfly32 — radix-2 butterfly . . . 51
11.6.7 cmul32 — complex multiply by one scalar . . . 52
11.7 Matrices . . . 52
11.8 BLAS names . . . 54
11.9 Harts . . . 59

12 RV64: The 64-bit Hart **63**
12.1 The integer file . . . 63
12.2 The address . . . 63
12.3 What crosses into the vector unit . . . 63
12.4 Fetch . . . 64

13 BX-64: Application Vectors with V and Zve64d **65**
13.1 The architecture . . . 65
13.2 What arrives with sixty-four-bit elements . . . 65
13.3 Opcode map for Zve64d / V (BX-64) . . . 66
13.3.1 Configuration . . . 66
13.3.2 Unit-stride load at SEW=64 — `vle64.v` . . . 66
13.3.3 Integer / FP arithmetic (OP-V) at any SEW including 64 . . . 66
13.3.4 Mask and move . . . 67
13.4 Assembler kernels that must run . . . 67
13.4.1 sew64 — double-word fill . . . 67
13.4.2 fma64 — double-precision fused multiply-add . . . 67
13.4.3 gemm64 — outer product of a 2-wide panel . . . 68
13.4.4 index64 — indexed load at SEW=64 . . . 68
13.5 The same bodies at SEW 64 . . . 68
13.6 Harts . . . 70
13.7 Xsvm on this machine . . . 74

14 BX-128: A Pinned 128-bit Register **79**
14.1 The board-level picture . . . 79
14.2 The architecture . . . 79
14.3 Widening . . . 79
14.4 Opcode map for BX-128 . . . 80
14.4.1 Configuration . . . 80
14.4.2 Unit-stride load at SEW=64 — `vle64.v` . . . 80
14.5 RV64 and COP3 fetch . . . 80
14.6 Assembler kernels that must run . . . 80
14.7 Harts . . . 83

15 BX-512: A Pinned 512-bit Register **85**

15.1 The board-level picture . . . 85
15.2 The architecture . . . 85
15.3 Widening . . . 85
15.4 Opcode map for BX-512 . . . 86
15.4.1 Configuration . . . 86
15.4.2 Unit-stride load at SEW=64 — `vle64.v` . . . 86
15.5 RV64 and COP3 fetch . . . 86
15.6 Assembler kernels that must run . . . 86
15.7 Harts . . . 89

16 RV128I: A 128-bit Address **91**
16.1 The board-level picture . . . 91
16.2 The architecture . . . 91
16.3 Widening . . . 92
16.4 Opcode map for RV128I . . . 92
16.4.1 Load double unsigned — `ldu` . . . 92
16.4.2 Load quadword — `lq` . . . 93
16.4.3 Store quadword — `sq` . . . 93
16.4.4 Shift immediate . . . 93
16.4.5 The `*D` operations . . . 94
16.4.6 Compressed parcels . . . 94
16.5 Native fetch . . . 94
16.6 Assembler kernels that must run . . . 95
16.7 Harts . . . 95

17 One Bluespec Description **99**
17.1 Complete opcode contract . . . 99
17.2 Aggregated opcode tables (ISA) . . . 100
17.2.1 Canonical layouts (32-bit, two equal-width half-words) . . . 100
17.2.2 Mnemonic coverage by profile . . . 101
17.2.3 Explicit summary — SVM-8 (BX-8) . . . 101
17.2.4 Explicit summary — SVM-16 (BX-16) . . . 101
17.2.5 Explicit summary — Zve32f (BX-32) . . . 101
17.2.6 Explicit summary — Zve64d / V (BX-64) . . . 102
17.2.7 Key constants (funct6 / width / mop) . . . 102
17.3 Parameters . . . 102
17.4 Interfaces . . . 103
17.5 The configuration rule . . . 103
17.6 Arithmetic rules . . . 104
17.7 What SVM-8 still contains . . . 104

18 Bring-Up, from `vsetvl` to a Running Kernel **105**
18.1 Order . . . 105
18.2 The first vector add . . . 105
18.3 Bus contention on the compatible machines . . . 106
18.4 When to leave the simulator . . . 106

19 Tests That Travel Up the Ladder **107**
19.1 One suite . . . 107
19.2 The fill . . . 107
19.3 Illegal SEW . . . 108

19.4 RV128 images . . . 108
19.5 Xsvm tests are separate . . . 108
19.6 Random parcels . . . 108

20 What Fits on the First FPGA **109**
20.1 The file, in bytes . . . 109
20.2 DSP blocks . . . 109
20.3 The first images . . . 109
20.4 Clocks . . . 110
20.5 A finished first board . . . 110

Profile Card **111**

Checklist against RVV 1.0 **113**

MSX and Amiga Compatibility Checklist **115**
BX-8, in place of an MSX . . . 115
BX-16, in place of an Amiga . . . 115
BX-32 and BX-64 . . . 115

Exercises **117**

Glossary **119**

References **121**

Preface

This book is a construction manual for one family of FPGA machines. The family is called BX. Each member is a RISC-V hart whose data path grows from eight bits to sixty-four, and whose vector unit grows with it under the RISC-V Vector Extension.

The eight-bit member is built so that it can take the place of an MSX: the slots, the video display processor, the sound generator, and the memory map stay recognisable. The sixteen-bit member is built so that it can take the place of an Amiga: a sixteen-bit bus, chipset-style shared memory, and the traffic patterns of a blitter and a copper. In both cases the processor that issues the instructions is RISC-V. The original machines remain the reference for peripherals and for the work the software has to do. The hart is the part that stands in their place.

Above those two machines the book stops speaking of profiles and starts speaking of ratified extensions. BX-32 implements Zve32f. BX-64 implements the application extension V, or at least Zve64d. A program written against the narrow profiles is still an RVV program. It uses the same registers, the same configuration instruction, and the same opcode map. What changes is the widest element the machine accepts.

The text is for a reader who will write the register transfer logic, preferably in Bluespec SystemVerilog, and who will synthesise it. It is not a programmer's companion to the vector intrinsics, and it is not a history of the home computers whose peripheral sets the machines keep.

The architectural choices that the rest of the book treats as fixed are these. The integer register width, XLEN, is 32 on BX-8, BX-16, and BX-32, and 64 on BX-64, BX-128, and BX-512. Chapter 8 opens the 32-bit file. Chapter 12 opens the 64-bit file. Chapter 16 is RV128I, an unfrozen base whose integer register is 128 bits. There is no separate eight-bit or sixteen-bit base instruction set; the narrowness is in the ALU, the bus, and the vector element length. The vector file always has thirty-two registers. Geometry operations that the vector specification does not define live in an optional custom extension and are silent unless a board asks for them.

Element width stops at 64 bits, which is where RVV 1.0 stops. BX-128 and BX-512 fix the register length at 128 and 512 bits. Those numbers are VLEN. XLEN on those two machines stays 64. The default elaboration fetches RV32 or RV64. A host elaboration, named on the profile card, sits the same vector unit on an R800, 68000, SuperH, or MIPS softcore. That elaboration is an instruction-port adaptor. It is not a second integer pipeline hidden in the RV decoder, and it is not a claim that the board is the original console.

A Note to the RTL Reader

Read the book in order the first time. The profiles in Part I are short on purpose. They exist so that later chapters can say “SVM-8” instead of repeating a list of legal element widths.

Then build in the order of Part IV. A scalar RV32I that fetches, decodes, and writes back is the whole of the first milestone. The vector unit begins as a configuration instruction and a single vector add. Memory, widening, masks, and floating point come after that, and only on the machines whose profile includes them.

Keep three files open while you read: the RVV specification, the profile card at the back of this book, and your elaboration parameters. If a paragraph and the profile card disagree, the card is the contract for the implementation and the paragraph is wrong.

Two habits will save you more time than any micro-architectural trick. First, make an illegal configuration raise an illegal-instruction exception in the same cycle you decode `vsetvl`, not deep in a lane. Second, do not advertise a ratified extension string until every instruction that string requires is present and tested. SVM-8 and SVM-16 are profiles of the encoding. They are not Zve32x.

Simulation comes before the board. A strip-mined fill that runs, with different vector lengths, on all four elaborated designs is the acceptance test of the ladder. Place and route is the acceptance test of the budget chapter, not of the architecture.

Notation and Conventions

Table 1: Names used throughout the book.

Name	Meaning
XLEN	Width of an integer register, in bits. 32 or 64 on the BX machines. 128 on RV128I.
Datapath	Width of the integer ALU and of the data bus, in bits.
ELEN	Widest vector element the implementation supports.
VLEN	Bits in one vector register. A power of two, at least ELEN.
SEW	Selected element width for the current configuration.
EEW	Effective element width of one instruction.
VL	Number of elements the current instruction processes.
VLMAX	Maximum VL for the current SEW and LMUL.
LMUL	Register grouping factor.
SVM-8	BX-8 profile: ELEN 8, VLEN 32, EEW 8.
SVM-16	BX-16 profile: ELEN 16, VLEN 128, EEW 8 and 16.
Host escape	The byte, halfword, or word a foreign fetch recognises before an OP-V parcel.
CpID	Coprocessor identity in a 68000 F-line word. BX uses 2.
Parcel endian	Which half of the 32-bit OP-V word travels first in the host stream.

Instructions are written in the usual assembly form, `vadd.vv v1, v2, v3`. Configuration is `vsetvl` or `vsetvli`. The thirty-two vector registers are `v0` through `v31`. The mask register is `v0`.

A *profile* is a named subset of RVV behaviour that this book specifies and that the specification does not ratify under that name. A *ratified extension* is a string the RISC-V manuals define: Zve32f, Zve64d, or V.

Bluespec fragments are illustrative. They show interfaces and rules, not a vendor core. Module names use the prefix `Bx`.

Intel examples use Intel syntax, destination first. ARM examples use unified syntax, destination first. A host harness file inserts an escape before every RV parcel, including scalar

words. A ROM does not. The folder README is the distinction.

The words MSX and Amiga, in this book, name a peripheral and bus contract. They do not name the original microprocessor. BX-8 is MSX-compatible in that contract and can replace an MSX in a rack or on a desk. BX-16 is Amiga-compatible in that contract and can replace an Amiga in the same sense.

Chapter 1

Introduction

This chapter is the machine, before any board. The later chapters fix the opcodes, the profiles, and the Bluespec parameters. What belongs here is the shape of the data, the units that move it, and the single adaptor that lets one vector unit sit in front of more than one instruction set.

1.1 How to read this book

Three paths through the same text.

Retro board.
Read this chapter through the adaptor, then the host section of BX-8 or BX-16 or BX-32, then the kernel binaries. The softcore is yours. The book specifies the port.

Register transfer.
Read the state chapter, the instruction classes, the FPGA chapter, and the Bluespec chapter. The adaptor appears there as an interface with the same fields as the table below.

Comparison.
Read the kernel tables. The columns are RVV, Intel, and ARM. A label shares its row with the instruction at that address. Skip the opcode matrices.

The scalable object is the adaptor, not a new story for each console. The hosts in this edition are the ASCII R800, the Motorola 68000 of the Amiga contract, the Hitachi SH-2 and SH-4, and three MIPS cores: the R3000A, the NEC VR4300, and the Toshiba Emotion Engine. The 68000 is in the list because BX-16's peripheral contract is the Amiga. The others are Japanese designs or MIPS implementations those designs use.

NES, Master System, TurboGrafx-16, Super Famicom, 3DO, Xbox, GameCube, Wii, and the later x86, PowerPC, and ARM consoles are outside this edition. The Hudson HuC6280 is a 6502 derivative and is outside with them. The NEC V810, used by the Virtual Boy and the PC-FX, mixes 16-bit and 32-bit instructions; drawing its adaptor record is an exercise, and this edition does not freeze an escape for it.

1.2 Three shapes of data

A BX machine holds scalars, vectors, and, when you ask for it, a matrix step.

A scalar is one XLEN-wide register. XLEN is 32 on BX-8, BX-16, and BX-32, and 64 on BX-64, BX-128, and BX-512. Those files are Chapters 8 and 12. Chapter 16 widens that register to 128 bits, and that base sits outside these profiles. The scalar file is the address file and the loop file. A vector is one of thirty-two registers, each VLEN bits wide, interpreted as elements of the selected width. A matrix is column-major memory, as Section 11.7 stores it. Xsvm can offer a four-by-four and a three-element cross product as custom opcodes. Xsvm is off unless the elaboration asks for it. The profiles themselves contain no matrix instruction.

SVM is that split. The S is the hart, or the host softcore. The V is the RVV profile. The M is optional.

1.3 The blocks

Four blocks do the work.

The scalar ALU operates on one XLEN register at a time. On BX-8 and BX-16 it is narrower than XLEN and takes several bus beats to finish a word. The architectural result is the RV32 result.

The vector functional units operate on elements. An integer unit covers add, subtract, logic, shift, compare, and the saturating forms the profile requires. A floating-point unit appears at BX-32. A mask unit writes and consumes `v0`. A permute unit, where the profile has slides or gathers, moves elements inside the register file.

The vector load/store unit moves elements between the vector file and memory. Unit stride is required on every machine. Constant stride begins at BX-16. Indexed access begins at BX-32.

The address of a vector load is a scalar register. There is no separate vector address file. The host chapters use that fact: the R800 pair, the 68000 address register, the SuperH register, or the MIPS register is wired to the same base-address port the RV `rs1` field already names.

1.4 Four streams

Flynn's four labels classify a machine by how many instruction streams and data streams it has in flight.

Label	Streams	Where BX puts it
SISD	one instruction, one datum	the scalar ALU
MISD	several instructions, one datum	not built
SIMD	one instruction, several data	the vector unit
MIMD	several instructions, several data	a second hart, where the profile allows one

MISD is in the table so the classification is complete. BX does not build a pipeline in which several different instructions share one datum. SIMD is the label that contains both a fixed-width SIMD unit and a vector unit. The difference that matters for this book sits inside that one label: a fixed-width unit names its width in the instruction, and a vector unit names its width in a register that software sets. MIMD, when you build it, is the second hart of BX-32 or BX-64. It is a second scalar core with the same vector profile, not a second encoding.

1.5 Fixed width and scalable length

MMX, SSE, AVX, and AVX-512 are successive Intel register widths: 64, 128, 256, and 512 bits. Each width added mnemonics. NEON is a 128-bit ARM file. SVE is an ARM file

whose length the implementation chooses and software discovers. SME adds a matrix tile and an outer-product instruction; it is the matrix analogue, and the kernels in this book are one-dimensional, so SME appears in a gap cell when a matrix instruction is the thing a reader might confuse with the kernel.

RVV keeps one encoding. `vtype` selects the element width and the register grouping. `vsetvl` returns how many elements fit. The same `vle8.v` runs on SVM-8 and on a BX-64 image. What changes is the largest element the machine accepts.

RVV 1.0 element widths stop at 64 bits. A 128-bit element is not an opcode you can hunt for in this book. VLEN on BX-16 is already 128 bits, and BX-32 and BX-64 may elaborate 256 or 512. That number is the register length, not the element length.

The per-kernel tables in the machine chapters are the comparison. This section only fixes the rule: on those ISAs the width is in the mnemonic, and here the width is in `vtype`.

1.6 Register windows

The scalar file is a flat RV file, thirty-two names, all visible to every call. Register windows, in the sense of a sliding set of names that a call instruction rotates, are not part of BX and not part of RVV.

The only overlap of architectural names is the grouping of vector registers described later in this chapter. Grouping changes how many names a single instruction reads. It does not save and restore a window on a call. The host softcores follow their own calling conventions. The adaptor does not add a window file in front of them.

1.7 Arbitrary length

`VL` is the number of elements the current instruction touches. `VLMAX` is the most elements one register group can hold at the current element width and grouping. Software that has more elements than `VLMAX` asks again. That loop is strip mining.

On BX-8, VLEN is 32 and SEW is 8, so `VLMAX` is 4. A copy of 10 bytes runs as three strips: 4, 4, and 2.

`kernels/bx8/copy8.sew8.lmul1.tu.mu.vlen32.xlen32.le.bin`

`vsetvli` writes the granted length into `t0`. The loads and stores use that length. The pointer advances by bytes, which at SEW 8 is the same number. The same loop at SEW 16, 32, or 64 changes the configuration immediate, the load and store mnemonic, and the shift that turns an element count into a byte count. The control flow stays. An Intel transcription of the same ten bytes changes register class and mnemonic when the register grows from 64 bits to 128. The kernel tables show that change on each routine.

The strips in this book walk forward, from low address to high. A reverse walk, which handles the short strip first and then steps down, is legal RVV and is not what these kernels do.

The table above is the acceptance kernel `copy8` of Chapter 9. The hex and the binary path sit under that table.

1.8 Arrays and vectors

An array processor replicates the work in space. Four processing elements, one instruction broadcast, four elements retired in the same cycle because four elements of hardware exist.

A vector processor can do the same work by streaming elements through a pipeline. The instruction says “these `VL` elements.” The hardware retires them over time, a lane group per cycle. Chaining, in the next section, is what lets the next instruction start on the first group before the last group of the previous instruction has retired.

Table 1.1: Kernel comparison: copy8

a0 = n, a1 = src, a2 = dst

ISA	Address	Hex	Assembly
RVV	0000	02050063	copy8: beqz a0, end
RVV	0004	000572D7	loop: vsetvli t0, a0, e8, m1, tu, mu
RVV	0008	02058087	vle8.v v1, (a1)
RVV	000C	020600A7	vse8.v v1, (a2)
RVV	0010	005585B3	add a1, a1, t0
RVV	0014	00560633	add a2, a2, t0
RVV	0018	40550533	sub a0, a0, t0
RVV	001C	FE0514E3	bnez a0, loop
RVV	0020	00008067	end: ret
Intel	0000	F30F6F06	copy8: movdqu xmm0, [rsi]
Intel	0004	F30F7F07	movdqu [rdi], xmm0
Intel	0008	4883C610	add rsi, 16
Intel	000C	4883C710	add rdi, 16
Intel	0010	4883EA10	sub rdx, 16
Intel	0014	77EA	ja copy8
ARM	0000	4C407020	copy8: ld1 {v0.16b}, [x1]
ARM	0004	4C007040	st1 {v0.16b}, [x2]
ARM	0008	91004021	add x1, x1, #16
ARM	000C	91004042	add x2, x2, #16
ARM	0010	F1004000	subs x0, x0, #16
ARM	0014	54FFFF68	b.hi copy8

BX builds both cuts, and they are parameters. The number of lanes is the spatial cut. The beats a lane takes are the temporal cut. One lane on BX-8 retires a four-element register in four cycles. Four lanes retire it in one. Both pass the same architectural tests.

Locality follows the cut. Each lane owns a slice of every vector register, so an element and its neighbours in that slice stay next to the ALU that consumes them. The FPGA chapter gives the slice width and the port count. The rule here is only that the slice is the unit of wiring.

1.9 Chaining

Chaining means a consumer instruction starts on the first element group as soon as that group is valid, while the producer is still writing later groups. The apparent latency of a load followed by an add is the latency of one group, not the latency of the whole vector.

Build chaining from BX-32 upward, where a floating-point pipeline is several cycles deep and the kernels are long enough to fill it. On BX-8 a four-element register finishes before a chain controller pays for itself. BX-16 may overlap a load and an arithmetic instruction only when the load's destination is not a source of the arithmetic instruction. That overlap is a scoreboard on three register numbers. It is the in-order rule, not a chain file.

1.10 Static and dynamic instructions

`copy8` has seven instructions in the body: configure, load, store, two address adds, the length subtract, and the branch. Ten bytes at `VLMAX` 4 take three trips through that body, twenty-one dynamic instructions, plus the opening branch and the return.

A scalar byte copy on the same hart is a load, a store, two pointer adds, a count add, and a branch, six instructions per byte. Ten bytes are sixty dynamic instructions in the body. The vector form wins because one static load describes every element of the strip. The fetch path of a small FPGA stays narrow. The work is inside the vector instruction, in the lane, not in the decoder.

The same arithmetic is why a host with an 8-bit or 16-bit instruction word can still justify a 32-bit OP-V parcel. The parcel is wide. It is rare. The bytes it moves are the scanline.

1.11 Ports, stalls, and issue

A vector-vector arithmetic instruction reads two register-file slices and writes one. A masked instruction also reads the mask bit for that element. That is the port picture the first FPGA image has to close timing on: two reads, one write, and a narrow mask read. The FPGA chapter turns the picture into a lane.

A stall, in this book, is a cycle the vector unit does not receive a bus grant because video or audio owns the bus. The element state does not change on that cycle. The instruction remains the current instruction. BX-8 and BX-16 are built so that this wait is the compatible behaviour: the picture does not tear because a copy held the bus.

Issue is in order. BX-8 and BX-16 hold one vector arithmetic instruction in the unit at a time. A later instruction waits until the register numbers it needs are free. Out-of-order issue, with a rename file and a reorder buffer, is a different machine. These cores do not build it. Precise exceptions stay the ones the profiles already name: an illegal SEW sets `vill` and the following vector instruction traps, and a faulting load reports the element in `vstart` on the machines that implement that register.

1.12 Grouping vector registers

LMUL is the number of architectural registers one instruction treats as a single wider register. The thirty-two names stay. A grouped instruction occupies a contiguous, aligned run of them.

SVM-8 fixes LMUL at 1. Four 8-bit elements fill the 32-bit register, and no instruction in that profile needs a second register to hold a result.

SVM-16 allows a group only for a widening destination. A 16-bit multiply that keeps the full product writes 32-bit elements, and those elements take two registers. The narrowing shift that brings a pixel back reads that group. The source registers of the multiply stay LMUL 1.

From BX-32 the ratified rules apply, including the groupings Zve32f permits. The profile card is the list of what an elaboration actually enables. Software that needs a group asks for it in `vsetvl`. An illegal group sets `vill`.

1.13 Instruction classes

The encoding is one 32-bit parcel, major opcode OP-V for arithmetic and configuration, and the vector load and store opcodes for memory. The classes the rest of the book implements are these.

Class	First machine
Configuration (`vsetvl`, `vsetvli`)	BX-8
Unit-stride load and store	BX-8
Integer arithmetic, compare, splat	BX-8
Mask logicals	BX-8
Constant-stride load and store	BX-16
Widening multiply, narrowing clip	BX-16
Reduction, slide by one	BX-16
Indexed load and store	BX-32
Floating-point arithmetic and fused multiply-add	BX-32
Double-precision forms	BX-64
Custom geometry (Xsvm)	optional, never part of a claim

The bit fields are in the instruction chapter and in the opcode maps of the machines. This list is the taxonomy those maps are sorted by.

1.14 The host adaptor

The vector unit consumes one 32-bit OP-V parcel on every host. The adaptor recognises an escape in the host instruction stream, assembles that parcel, and presents it on the same instruction port the RV32 hart uses. There is no second vector encoding. There is no R800, 68000, SuperH, or MIPS pipeline in this book. You bring a softcore that already fetches that instruction set. Old ROMs keep working when they never execute the escape, and when the scalar core really is that CPU.

Host	Escape	Endian	`rs1` selects	What stays
R800	`ED 00`	little	BC DE HL IX IY SP	MULUB, MULUW
68000	`F400`	big	D0–D7, A0–A7	A-line, CpID 1
SH-2	`0000`	big	R0–R15	the rest of the map
SH-4	`0000`	little	R0–R15	`Fxxx`, including FTRV
R3000A	`4C000000`	little	R0–R31	COP2, the GTE
VR4300	`4C000000`	big	R0–R31	COP1, the FPU
EE	`4C000000`	little	low 32 bits of the GPR	COP2, which is VU0

`F400` is an F-line word with coprocessor id 2. `4C000000` is a COP3 word with every other field zero. The MIPS adaptor claims that word on the instruction bus. With coprocessor 3 disabled, the host would raise Coprocessor Unusable; the adaptor takes the parcel first and the exception does not fire.

The parcel after the escape is the entire next instruction word. On a little-endian host the low half travels first. On a big-endian host the high half travels first. SH-2 and SH-4 share the escape halfword and differ in that order. Saturn code is big-endian. Dreamcast code is little-endian. Mixing the two framed binaries will execute a byte-swapped instruction.

`rs1` and `rs2` in a host image are host indices, not RV ABI numbers. The checked-in harness files still contain the RV numbers, with the escape inserted in front of every parcel, scalar parcels included. A ROM re-assembles the address fields and writes the branch in the host instruction set. The folder README says which file is which.

The adaptor stalls the host in order until the SVM accepts the parcel. An illegal opcode or an illegal SEW becomes the host's illegal-instruction exception, or an F-line exception on the 68000. `vill` itself does not change.

The vector register file is only the SVM file. Host general registers are addresses and scalar results, never vector elements.

Leave these alone. The A-line trap is how Amiga libraries are called. The SH-4 `Fxxx` space is the existing FPU and graphic unit. COP2 on the R3000A is the GTE, and COP2 on the Emotion Engine is VU0. SVM-8 and SVM-16 do not set `misa`.V. An empty `ED` slot is not part of the MSX standard; `ED 00` is the escape because the published map leaves it unused, and because MSX padding is the single byte `00`. SH-4 on BX-16 is an integer injection. ELEN is 16. A Dreamcast-class floating-point transform uses the same escape in front of BX-32.

The cost of the escape is the host's fetch of the prefix and of the parcel. The vector instruction then takes the same lane cycles it takes on the RV elaboration.

The bytes of every acceptance kernel are in `kernels/`, produced by `kernels/encode_kernels.py`. The hex in the machine chapters is that output.

Chapter 2

Machines That Remain MSX and Amiga

2.1 The place of the original machines

The MSX and the Amiga solved real-time picture and sound with small memories, narrow buses, and specialised peripheral chips. Those chips, and the way software talks to them, are the part worth keeping in register-transfer form. A student who can read a video display processor written as rules will understand a later display controller. A student who can read a shared sixteen-bit bus with several masters will understand a later DMA fabric.

BX-8 is the MSX-compatible member. Its slots, video display processor, programmable sound generator, keyboard matrix, and cartridge signalling follow the MSX map closely enough that a board can sit where an MSX sat: same kind of picture, same kind of sound, same habit of I/O ports. The processor is an RV32I hart with an eight-bit data path and the SVM-8 vector profile.

BX-16 is the Amiga-compatible member. Its data bus is sixteen bits wide. Memory is shared among the processor, a blitter-class mover, a video fetch, and an audio path, in the spirit of the Amiga chipset. The processor is an RV32I hart with a sixteen-bit data path and the SVM-16 vector profile. It can take the place of an Amiga in the same practical sense: a single FPGA image that runs a sixteen-bit desktop-class peripheral set and a vector unit sized for blits, mixing, and small transforms.

2.2 What compatible means in RTL

Compatibility, here, is a list you can tick on a board, not a promise that every original binary image will execute.

- Address ranges and register layouts of the kept peripherals match the reference machines within documented differences.
- Bus width matches: eight bits on BX-8, sixteen bits on BX-16.
- Cycle behaviour of the peripherals is specified and tested against the reference documents you choose to follow.

- Software that is written for the BX hart, in RV32I and in the vector profile, drives those peripherals. This is the default elaboration, `host = RV32`.
- A thin compatibility layer, if you write one, translates or interprets guest code. It is software on the hart.
- A host elaboration attaches the same SVM to a softcore you already have: R800 on BX-8, 68000 or SuperH on BX-16, MIPS on BX-32. The adaptor recognises an escape and presents one OP-V parcel. It is not a second decoder hidden inside the RV32 pipeline. Each elaboration has one fetch path.

The original processors remain the reference for what those machines could do with a given number of cycles. In the RV32 elaboration they are not instantiated beside the hart. In a host elaboration the softcore is the scalar CPU, and the SVM is the vector unit beside it.

2.3 Why the peripherals survive the change of processor

A video display processor does not become simpler because the CPU is RISC-V. Sprites, tiles, blanking, and a shared memory port are the same problem on an MSX-compatible board as they were on the original. Writing them in Bluespec forces the arbitration into rules that the compiler schedules. That is the infrastructure BX-32 and BX-64 reuse when the picture is no longer a tilemap and the bus is no longer eight bits wide.

The same is true of the sixteen-bit machine. A blitter is a programmed DMA with a small arithmetic step. SVM-16 can perform the arithmetic in the vector unit, or the blitter can remain a peripheral master. Both organisations are legal on BX-16. The book recommends keeping the peripheral master for compatibility of timing, and using the vector unit for new software.

2.4 The machine contract

Table 2.1: What each machine is for.

Machine	Can replace	Vector contract
BX-8	an MSX	SVM-8, integer, EEW 8
BX-16	an Amiga	SVM-16, integer, EEW 8 and 16
BX-32	—	Zve32f, with FP32
BX-64	—	V or Zve64d, with FP64
BX-128	—	V or Zve64d, VLEN 128, ELEN 64
BX-512	—	V or Zve64d, VLEN 512, ELEN 64

BX-32 and BX-64 keep the organisational lessons of the Saturn, the Dreamcast, the PlayStation 2, and the Nintendo 64: more than one hart, local memory, and a vector unit wide enough for geometry. BX-128 and BX-512 keep that vector contract and pin the register length. RV128I, Chapter 16, is an integer base with XLEN 128. It has no row in this table and no vector profile. The default compatibility target from BX-32 upward is the RISC-V specification. A host elaboration can sit an SH-2, SH-4, R3000A, VR4300, or Emotion Engine softcore in front of the same SVM. That is an instruction port. It is not a claim that the FPGA image is the console, and it is not a cartridge port.

Chapter 3

The RISC-V Ladder, from an 8-bit Bus to a 64-bit Element

3.1 Datapath width is not XLEN

The frozen bases RV32I and RV64I, Chapters 8 and 12, define integer registers of thirty-two and sixty-four bits. RV128I, Chapter 16, is an unfrozen base whose integer register is 128 bits, so an address can be 128 bits. No frozen base defines eight-bit or sixteen-bit registers. BX-8 and BX-16 honour the frozen bases. XLEN is 32. The instructions in memory are the ordinary thirty-two-bit RV32I encodings. The ALU and the data bus are narrower than XLEN, so a word operation takes several cycles.

On BX-8 a load word is four bus beats, a load halfword is two, and a load byte is one. Shifts and adds iterate over bytes, low byte first, with the carry kept in a one-bit register between beats. The architectural result in the integer register is identical to a single-cycle RV32I implementation. Only the latency changes.

On BX-16 the beat is a halfword. A load word is two beats. The vector element width of SVM-16 matches the bus: eight or sixteen bits. A unit-stride vector load moves VLEN/16 halfwords when SEW is 16, and the bus can supply one halfword per beat.

BX-32 is the first member whose datapath, bus, and XLEN are all thirty-two bits. BX-64 widens XLEN itself to 64 and implements RV64I, together with the floating-point and vector extensions its profile names. BX-128 and BX-512 keep that XLEN. A 128-bit integer register is Chapter 16.

3.2 One decode for scalar and vector

OP-V is the only vector encoding. The decoder classifies a parcel as integer, load, store, configuration, or vector arithmetic, and steers it. A host adaptor may recognise a foreign escape and then present that same parcel. It does not invent a private vector opcode, and it does not glue a second opcode map onto the integer decode of the RV hart. On the narrow machines the vector lanes are as narrow as the bus. A VLEN of 32 bits with eight-bit elements is four lanes, and four lanes of eight bits fit in a small FPGA without a second clock domain.

This is why a kernel written for SVM-8 is still RVV. The assembler emits `vsetvli` and `vadd.vv`. BX-32 will execute that parcel if the requested SEW is legal there. The ladder is a sequence of legal SEWs, not a sequence of source translators.

3.3 Latency you should publish

Publish, next to the profile card, the cycle count of the operations a programmer will feel.

Operation	BX-8	BX-16	BX-32
ALU on XLEN bits	4 beats	2 beats	1
Unit-stride beat	1 byte	1 halfword	1 word
`vsetvl`	1	1	1

Vector arithmetic latency is a parameter of the lane pipeline, not of XLEN. Keep `vsetvl` single-cycle. Configuration is how software discovers VLMAX, and hiding it in a long pipeline teaches the wrong lesson.

3.4 What moves upward unchanged

A strip-mined loop that only uses EEW=8 is legal on every machine in the book. A loop that uses EEW=16 is legal from BX-16 upward. A loop that uses EEW=32 or FP32 is legal from BX-32 upward. A loop that uses EEW=64 or FP64 is legal on BX-64, BX-128, and BX-512.

The source does not change. The elaborated hardware refuses, at `vsetvl`, any SEW above its ELEN. That refusal is the entire compatibility story between profiles. There is no mode bit that widens a machine beyond the profile on its card.

Chapter 4

SVM Profiles and the Ratified Vector Extensions

4.1 The names

SVM stands for scalar, vector, and matrix. In this book the scalar part is the RISC-V integer hart, the vector part is a profile of RVV, and the matrix part is an optional custom extension called Xsvm. Xsvm is not implemented unless the elaboration parameter asks for it. The profiles themselves do not include matrix instructions.

Table 4.1: The ladder as a contract.

	BX-8	BX-16	BX-32	BX-64
Name	SVM-8	SVM-16	Zve32f	V or Zve64d
XLEN	32	32	32	64
Datapath	8	16	32	64
ELEN	8	16	32	64
VLEN	32	128	≥ 128	≥ 128
Legal EEW	8	8, 16	8, 16, 32	8, 16, 32, 64
FP	no	no	FP32	FP32 and FP64
Ratified claim	no	no	yes	yes

VLEN on BX-32 and BX-64 may be 256 or 512. The minimum is 128 because that is what the application extension requires, and because Zve32f implementations in this book are built to the same floor so that a BX-32 image can later be compared with a BX-64 image on the same vector length. BX-128 and BX-512 pin that length at 128 and 512. ELEN on both is 64. The ladder of element widths ends at 64. RV128I, Chapter 16, widens the integer register and leaves this element ladder where it is.

4.2 Why SVM-8 is not Zve32x

Zve32x requires element widths 8, 16, and 32, a minimum VLEN of at least ELEN, and the integer vector instruction set that goes with those widths. SVM-8 implements one of those

widths. Reporting Zve32x from BX-8 would tell a compiler that `vsetvl` with SEW=32 is legal. It is not. The machine raises an illegal-instruction exception.

SVM-16 fails the same test one step higher. SEW=32 is illegal. SEW=8 and SEW=16 are legal. The machine also implements the saturating and widening integer operations used by sixteen-bit kernels, still under the OP-V encoding.

From BX-32 the claim is allowed. Zve32f adds single-precision floating point on top of the integer widths through 32. BX-64 claims V when VLEN is at least 128 and the double-precision vector operations are present; otherwise it claims Zve64d and says so.

4.3 The `misa` rule

On BX-8 and BX-16, `misa` reports RV32I and the extensions you have actually implemented among M, A, and the CSR set. It does not set the V bit. Software discovers SVM-8 or SVM-16 through a documented custom CSR, or through an illegal-instruction probe of SEW, whichever you publish in the profile card. Pick one and test it.

On BX-32, `misa` may set the bits for the extensions you implement, and the vector discovery path is the standard one for Zve32f. On BX-64 the V bit is set only if you implement V, not merely a subset you hope to finish later.

4.4 Xsvm

Two operations recur in the geometry the sixteen-bit machine is asked to run: a four-by-four matrix times a vector, and a three-element cross product. RVV expresses both as short sequences. BX-16 may also offer them as custom opcodes in the custom-0 or custom-1 space, under the name Xsvm.

If Xsvm is present it is an addition, not a substitute. A kernel that uses only standard OP-V must run on a BX-16 built without Xsvm. Document the opcodes in the same header that documents the discovery CSR. Do not alias them onto standard vector opcodes.

4.5 Fixed point without a private lane type

Sixteen-bit pixels and samples are ordinary SEW=16 integers. A product that needs more than sixteen bits is a widening multiply into a thirty-two-bit element group. A Q16.16 value, if software wants one, is that product followed by a shift. The hardware does not have a third element type for fixed point. Rounding and saturation use `vxrm` and `vxsat` where the specification already defines them, and only on the machines that implement those instructions.

Chapter 5

Vector State

5.1 The register file

Every machine in this book has thirty-two vector registers, numbered 0 through 31. Register 0 is also the mask. When an instruction is masked, bit i of `v0` says whether element i is active. Implementing a subset of the registers would change the encoding, so the file is always thirty-two deep even when each register is only thirty-two bits wide, as on BX-8. That file is 128 bytes. On BX-16, VLEN is 128 and the file is 512 bytes. Both sizes are small next to a video memory.

Registers are not grouped in the SVM-8 profile. LMUL stays 1. SVM-16 allows LMUL in $\{1, 2, 4, 8\}$ for widening destinations, because a sixteen-bit multiply that widens needs a thirty-two-bit place to land, and that place is a register group. If you would rather forbid LMUL greater than 1 on SVM-16, widening stores its high half in an explicit second destination named by the instruction you implement. Pick one, write it on the profile card, and do not mix them.

5.2 Configuration registers

The programmer-visible configuration is the standard one.

- `vtype` holds the selected element width, LMUL, and the tail and mask policy bits.
- `vl` holds the number of elements to process.
- `vstart` holds the element at which a partially completed instruction would resume. On these machines instructions either complete or trap before they write, and `vstart` is zero whenever software is allowed to read it. State that in the profile card. It is a legal simplification if you never resume a trapped vector instruction in the middle.
- `vxrm` and `vxsat` exist on any machine that implements fixed-point rounding or saturation. SVM-8 may omit them if it omits those instructions.

`vsetvl` and `vsetvli` are the only way to change `vtype` and `vl`. The instruction writes the chosen `vl` to the integer destination register as well as to the CSR. That write is how a strip-mined loop learns how many elements it was granted.

5.3 Legality at configuration time

The decoder of `vsetvl` checks SEW against ELEN and LMUL against the profile. An illegal request sets the `vill` bit of `vtype` and does not change `vl`, following the specification's rule for an invalid configuration. Subsequent vector instructions then trap as illegal. Do not leave `vill` clear and fail later inside a lane. Software cannot recover a clean meaning from a lane that has already consumed a beat of the bus.

On SVM-8, SEW must be 8 and LMUL must be 1.
On SVM-16, SEW must be 8 or 16.
On Zve32f, SEW must be 8, 16, or 32.
On V or Zve64d, SEW must be 8, 16, 32, or 64.

5.4 Tail and mask policy

The specification allows undisturbed and agnostic policies for inactive elements and for the tail past `vl`. Agnostic is easier in hardware: inactive lanes may write anything. Undisturbed is what a careful blit wants, because the tail of a scanline must stay what it was.

Implement undisturbed for the body of this book. It costs a read of the old destination element and a select. On an eight-bit lane that read is cheap. Publishing agnostic as well is optional and must be tested; a single policy, undisturbed, is the one the exercises assume.

5.5 Precise traps

A vector load that faults does so with `vstart` naming the first element that did not complete, or the instruction does not start the bus transaction until the whole access is known to be mapped. The second option is the one these machines take on BX-8 and BX-16, where the address map is a flat peripheral space without demand paging. BX-64, if it runs with a page table, uses the first option and a documented `vstart`. The profile card says which.

Chapter 6

The Instruction Classes

6.1 A minimum that is still RVV

Each profile implements a rectangle of the vector specification, not a sketch of it. Inside that rectangle every instruction you claim is the standard one: same encoding, same register numbers, same exception conditions. Outside the rectangle the instruction is illegal.

The rectangle grows as follows.

6.2 Configuration

Both `vsetvl` and `vsetvli` are required on every machine. `vsetivli` is required from SVM-16 upward. Immediate configuration is how small kernels avoid a spare integer register, and BX-8 may include it too if the decoder is already wide enough. If BX-8 omits it, the profile card says so.

6.3 Loads and stores

Unit-stride loads and stores of the legal element widths are required everywhere. The address is a byte address in the integer register, as in the specification. On BX-8 the bus moves one byte per beat, so a unit-stride load of `vl` elements occupies `vl` beats. On BX-16 a SEW=16 unit-stride load occupies `vl` halfword beats when the address is aligned, and two byte beats per element when it is not. Alignment faults follow the scalar policy of the hart: if scalar misaligned accesses are allowed, vector ones are allowed, and they take the extra beats.

Constant-stride loads are required from SVM-16 upward. Indexed loads are required from BX-32 upward. They are the gathers a texture or a sparse blit actually uses. Leaving them out of SVM-8 keeps the first load unit a counter and an address adder.

Segment loads are not required in any profile in this book. A compiler that emits them will trap. Say so next to the profile card.

6.4 Integer arithmetic

For every legal EEW the machine implements vector add, subtract, and, or, xor, and the shifts, in both vector-vector and vector-scalar forms. Compare and min/max are required from

SVM-16 upward. SVM-8 includes compare, because a mask is the natural way to clip a byte.

Widening multiply is required on SVM-16 for SEW=8 producing EEW=16, and for SEW=16 producing EEW=32. The thirty-two-bit destination is the reason SVM-16 allows a grouped destination even though ELEN is 16. Narrowing shifts are required on the same machine so that a widened product can return to a sixteen-bit pixel.

Saturating add and subtract are required on SVM-16 and on BX-32. SVM-8 includes saturating add only. Saturation sets `vxsat` when that CSR is implemented.

6.5 Reductions, masks, and permutes

An integer add reduction is required from SVM-16 upward. It is how a dot product of sixteen-bit samples finishes without a scalar loop over lanes. Slides by one element are required from SVM-16 upward. Gather-permute within a register is required from BX-32 upward.

Mask logical operations and mask population count are required wherever compare is required. The mask is `v0`. There is no second mask file.

6.6 Floating point

No floating-point vector instruction is legal on BX-8 or BX-16. The scalar hart on those machines also omits the F and D extensions, so a compiler for the replacement MSX or Amiga is an integer compiler plus the vector profile.

BX-32 implements the single-precision vector operations of Zve32f: add, subtract, multiply, fused multiply-add, min, max, and compare, plus the conversions between SEW=32 integers and FP32. BX-64 adds the double-precision set required by Zve64d, and the rest of V if that is the claim.

6.7 Custom geometry

Xsvm, when elaborated, adds two instructions in custom-0:

- a four-by-four matrix held in `v0`–`v3` times a vector in `vs2`, result in `vd`;
- a three-element cross product of `vs2` and `vs1` into `vd`.

Both operate at the current SEW and honour `vl`. They are illegal on a build that did not ask for Xsvm. They are illegal on BX-8 even if someone sets the parameter, because a four-by-four of bytes is not the operation the extension is for. The parameter is honoured from BX-16 upward only.

6.8 Instructions you will be tempted to skip

Division, square root, and the transcendental estimates are not in SVM-8 or SVM-16. They are optional on BX-32 and required on BX-64 only to the extent the claimed extension requires them. A reciprocal estimate with a Newton step in software is an acceptable published gap on BX-32 if the profile card lists it. It is not an acceptable gap if you have set the V bit.

Chapter 7

Lanes, Register Files, and Memory on an FPGA

7.1 Three widths

These machines have two register files. XLEN sizes the integer file and the address. VLEN sizes one vector register. SEW is the element width software selects inside that vector register. ELEN is the widest SEW the implementation accepts. At LMUL 1, VLMAX is VLEN/SEW.

Name	Where it lives	What it counts
XLEN	integer register	Bits of one GPR, and of an address.
VLEN	vector register	Bits of one of the thirty-two vector registers.
ELEN	implementation	Widest legal SEW.
SEW	`vtype`	Bits of one element, chosen by software.
VLMAX	derived	Elements in one register at LMUL 1.

BX-128 pins VLEN at 128. BX-512 pins VLEN at 512. Both keep ELEN at 64, so the legal element widths are 8, 16, 32, and 64. SEW 128 traps on BX-128. SEW 512 traps on BX-512. The default hart on both pins is RV64, so XLEN is 64. Chapter 16 is an RV128I hart, XLEN 128, with that same vector unit attached. The vector file stays the size the pin names.

	RV64 hart		RV128 hart	
	BX-128	BX-512	BX-128	BX-512
GPR, bits	32×64	32×64	32×128	32×128
GPR, bytes	256	256	512	512
Vector register, bits	32×128	32×512	32×128	32×512
VRF, bytes	512	2048	512	2048
VLMAX at SEW 32	4	16	4	16
VLMAX at SEW 64	2	8	2	8
Legal lane counts	1, 2	1, 2, 4, 8	1, 2	1, 2, 4, 8
Address adder, bits	64	64	128	128

Chapters 14 and 15 build the RV64 columns, on the hart of Chapter 12. The two RV128 columns are the same vector unit beside the integer base of Chapter 16. The OP-V parcel is 32 bits on every row. `vsetvli` has no XLEN field and no VLEN field. `vsetvli t0, zero, e32,`

`m1, tu, mu` is the word `002072D7` on all four compositions. `t0` receives 4 when VLEN is 128 and 16 when VLEN is 512.

`vmv.x.s` copies element 0 into a GPR. When SEW is narrower than XLEN, the value is sign-extended to XLEN. On RV64 a SEW-32 element fills bits 63:32 with the sign. On RV128 that element sign-extends through bit 127, and a SEW-64 element sign-extends through bit 127. `vmv.s.x` copies the low SEW bits of the GPR into element 0 and leaves the bits above SEW unused. A vector-scalar integer operation reads those same low bits. The floating-point scalar comes from an F register. RV128I leaves that file at 32 or 64 bits, so the floating-point scalar port stays the width of FLEN.

`vle64.v` with `vl` equal to 2 writes 16 bytes into one BX-128 vector register. `lq` writes 16 bytes into one RV128 GPR. The data port in Chapters 14 and 15 is 64 bits, so both moves take two beats. The bytes arrive in different files.

7.2 The Intel register ladder

Intel widened the vector register in four steps and, in long mode, left the general-purpose register at 64 bits. That is an RV64 hart beside a pinned vector file. The width column is VLEN. The opcode names the element width, which is SEW. The source for the register pictures is the Intel manual in the references.

Extension	Registers	Width	At SEW 32	At SEW 64	GPR
MMX	8 `mm`	64	2	—	32 or 64
SSE, SSE2	16 `xmm`	128	4	2	64
AVX, AVX2	16 `ymm`	256	8	4	64
AVX-512	32 `zmm`	512	16	8	64

A 32-bit SSE program has eight `xmm` registers. Long mode has sixteen, and that is the count the kernel tables are assembled against. AVX-512 raises the architectural count to thirty-two `zmm` registers, which is the number of vector registers on every machine in this book, including BX-8. MMX stops at 32-bit elements. `paddq` is an SSE2 instruction on `xmm`. SSE proper is four packed singles. SSE2 adds the integer element widths and two packed doubles. AVX2 is what carries the integer element widths up to `ymm`. AVX itself is the floating-point widening.

One BX-128 register at SEW 32 and `vl`=4 is four packed singles, an `xmm`. `gemm32` is that width. The same register at SEW 64 and `vl`=2 is two packed doubles, and `gemm64` is that width. One BX-512 register at SEW 32 and `vl`=16 is sixteen packed singles, a `zmm`. `wide32` is that update, and its Intel column uses `zmm`. `wide64` is eight packed doubles. A BX-64 image may set VLEN to 256, which is a `ymm`. BX-128 is pinned at 128, so its architectural register is the `xmm`, and BX-512 is the `zmm`.

MMX is eight architectural registers of 64 bits. Two lanes of BX-128 also cut the vector file into 64-bit slices. The lane is a slice of one 128-bit register. `mm0` is the register the instruction names.

AVX-512 adds eight mask registers, `k0` through `k7`, each 64 bits. RVV keeps the mask in `v0`, one bit per element. On BX-512 at SEW 8, VLMAX is 64, so the live mask is 64 bits wide and still resides in `v0`. An `xmm` instruction writes the whole register. RVV writes `vl` elements and keeps the tail under the tail policy. `ymm0` overlaps `xmm0`: the low 128 bits are one register viewed at two widths. An LMUL group names distinct registers. The bits of `v0` and the bits of `v2` do not overlap.

x86-64 widened the general-purpose register from 32 bits to 64 and left `xmm` at 128. That change is XLEN. AVX widened the vector register and left `rax` at 64. That change is VLEN. RV128I widens the general-purpose register to 128 and leaves the vector pin where Chapters 14 and 15 set it.

7.3 How many lanes

A lane is a slice of the vector register file plus the functional units that operate on one element group per cycle. The number of lanes does not change the architecture. It changes how many elements retire per cycle. The same OP-V program is the acceptance test for one lane and for several. The Hart column is the integer file the row is built beside: RV32 in Chapter 8, RV64 in Chapter 12, and RV128 in Chapter 16.

Machine	Hart	VLEN	Suggested lanes	Reason
BX-8	RV32	32	1 or 4	Four eight-bit lanes finish a register in one cycle.
BX-16	RV32	128	4	Four sixteen-bit lanes match a modest DSP column.
BX-32	RV32	128	4	One FP32 unit per lane is already a large board image.
BX-64	RV64	128	1 or 2	SEW 64 holds two elements, so four lanes do not divide.
BX-64	RV64	256	2 or 4	Four lanes are legal once a register holds four doubles.
BX-128	RV64	128	2	Each lane holds 64 bits: one FP64, or two FP32.
BX-128	RV128	128	2	Same lanes. The address adder and the GPR are 128 bits.
BX-512	RV64	512	4	Each lane holds 128 bits and retires two doubles.
BX-512	RV128	512	4	Same lanes. The address adder and the GPR are 128 bits.

Start with one lane on the first elaboration of each profile. Widen after the tests in Chapter 19 pass. `NLANES` must divide VLEN/SEW for every legal SEW, because each lane owns a fixed slice of bits and that slice must contain a whole number of elements at every SEW. On SVM-16, VLEN is 128 and the narrowest VLMAX is 8, so `NLANES` divides 8. Four is the comfortable choice.

On BX-128 the narrowest VLMAX is 2, so the legal counts are 1 and 2. Two lanes give each lane 64 bits: one FP64, two FP32, four halfwords, or eight bytes per cycle. On BX-512 the narrowest VLMAX is 8, so the legal counts are 1, 2, 4, and 8. Four is the image to build first. Eight lanes retire a register in one cycle at every legal SEW and require eight FP64 units. The RV128 rows repeat the BX-128 and BX-512 counts. Only the address adder and the GPR port grow.

7.4 The register file

Each lane owns a fixed slice of every vector register, the same bits at every SEW. With four lanes and VLEN 128, a lane holds 32 bits of each register. With two lanes and VLEN 128, a lane holds 64 bits. With four lanes and VLEN 512, a lane holds 128 bits. Reads for a vector-vector instruction are two ports; the write back is a third. The mask bit of element i is bit i of `v0`, while the element's data begins at byte $i \times \mathrm{SEW}/8$. The mask port follows the element index. It is a fourth, narrower port, or a bit collected across slices.

On an FPGA, implement the file as distributed memory for SVM-8 and as block RAM for SVM-16 and above if the port requirement fits the RAM primitive. If the primitive is dual-port and you need three accesses, bank the file by element and accept a one-cycle bubble when two reads and a write collide, or replicate the file. Replication is the smaller surprise on a teaching design. State the choice in the profile card under "microarchitecture", not under "architecture". Software must not be able to tell, except by cycle count.

7.5 Chaining

Chaining means a consumer starts when the first element of its producer is ready, rather than when the whole vector has retired. It is worth building on BX-32, BX-64, BX-128, and BX-512, where floating-point latency is several cycles and kernels are long. The hart width does not remove that latency. It is not worth building on BX-8. A four-element register finishes before a chain controller pays for itself.

The in-order rule for BX-8 and BX-16 is: one vector instruction in the arithmetic unit at a time, loads and arithmetic allowed to overlap only when the load target is not a source of the arithmetic instruction. Scoreboard those three register numbers. That is enough hazard logic for the first board.

7.6 Memory width

Match the vector memory port to the bus you already have for the peripherals, not to VLEN.

On the MSX-compatible machine the bus is eight bits. The vector load unit is a byte counter. On the Amiga-compatible machine the bus is sixteen bits and is shared with video and blitter masters. The vector load unit requests halfwords through the same arbiter as those masters. Do not add a private wide path from the vector file to memory on BX-16; it would no longer be the bus an Amiga-compatible board has.

On BX-32 and BX-64 a dedicated wider port is appropriate, because those machines are not standing in for an eight-bit or sixteen-bit chipset. Even there, size the port to a power of two beats of the element width you care about, and measure utilisation with the fill test before widening it again.

BX-128 and BX-512 keep the 64-bit data port of their board pictures. A 128-bit vector register then takes two beats, and a 512-bit register takes eight. An RV128 hart widens the address adder to 128 bits and leaves this port alone. A 128-bit data port, the wide choice Chapter 16 allows for `lq`, retires a BX-128 register in one beat and a BX-512 register in four.

7.7 What to read in other people's cores

Ara shows a lane-based file, a central sequencer, and a slide unit, on a full application vector extension, in a process where a lane's area is a published number. Spatz shows that a small file and a shared scratchpad can be enough when the data are already local. Vicuna shows a coprocessor interface and a configurable set of pipelines aimed at Zve32x.

None of them is a Bluespec description of SVM-8, and none of them is an MSX or Amiga bus. Read them for the shape of a lane and for the list of hazards. Write your own rules against the profile card. Importing a large SystemVerilog vector unit beside an eight-bit peripheral fabric will not produce a machine that can replace an MSX.

Chapter 8

RV32: The 32-bit Hart

BX-8, BX-16, and BX-32 share this integer file. The vector unit is what changes from chapter to chapter. Lane counts for those machines are the RV32 rows of Section 7.3. This chapter is the scalar side the next three boards attach.

8.1 The integer file

Thirty-two general-purpose registers, each 32 bits. The file is 128 bytes. `x0` reads as zero. Every parcel is 32 bits, and every OP-V word is one of those parcels.

The data path may be narrower than the register. On BX-8 the ALU and the bus are eight bits, so one word takes four beats. On BX-16 the bus is sixteen bits, so one word takes two beats. On BX-32 the data path is 32 bits and one word takes one beat. The value written back is the 32-bit architectural result on all three. Chapters 9, 10, and 11 are those boards.

8.2 The address

The address adder is 32 bits. A vector load takes its base from one of these registers. BX-16 keeps VLEN at 128 on this adder. BX-32 keeps the same adder and raises ELEN to 32. The address width is XLEN.

8.3 What crosses into the vector unit

Three values leave this file for the vector unit. The base address of a load or a store is the whole GPR. The scalar operand of a `.vx` instruction is the low SEW bits of a GPR. Element 0 comes back through `vmv.x.s`.

`vmv.x.s` writes element 0 into a GPR. When SEW is narrower than 32, the element is sign-extended to 32 bits. On BX-8, SEW is 8, so a byte sign-fills the register. On BX-16, SEW is 8 or 16. On BX-32 the widest element is 32 bits, which equals XLEN, so the element fills the GPR.

`vmv.s.x` writes a GPR into element 0. When SEW is narrower than 32, the low SEW bits are copied and the bits above SEW stay out of the element. A `.vx` instruction uses those same low bits. On these three machines SEW is at most 32, so the GPR holds the whole element.

BX-8 and BX-16 have no floating-point register file. On BX-32 the floating-point scalar is an F register of 32 bits, and `OPFVF` reads that register.

8.4 Fetch

The default elaboration fetches RV32 on a 32-bit instruction port. OP-V uses that port.

A host elaboration puts an escape in front of one OP-V parcel. The integer pipeline belongs to the softcore you bring. BX-8 uses the R800 escape `ED 00`, little-endian. BX-16 uses the 68000 escape `F400`, the SH-2 escape `0000` in big-endian, and the SH-4 escape `0000` in little-endian. BX-32 uses the COP3 word `4C000000`. The adaptor record is Chapter 1. The register maps are in the machine chapters. A harness file prefixes the escape before every RV parcel and still contains RV register numbers. A ROM escapes only OP-V, writes the branch in the host instruction set, and reassembles `rs1` as a host index.

Chapter 9

BX-8: An MSX-Compatible Machine with SVM-8

9.1 The board-level picture

BX-8 is a single FPGA image that can replace an MSX in the narrow sense of Chapter 2. The parts a cartridge and a display see are the slot signals, the video display processor, the sound generator, the keyboard and joystick ports, and the memory map. The part that executes instructions is an RV32I hart. Its data path is eight bits. Its vector unit is SVM-8. Chapter 8 is that integer file.

A word in an integer register is still thirty-two bits. The program counter advances by four when the parcel is an ordinary instruction, because the encoding is thirty-two bits even though the data bus is not. Instruction memory may be a separate thirty-two-bit port. Data memory and I/O share the eight-bit bus with the video processor. That split is the usual way to keep fetch off the peripheral bus, and it does not change the architectural width of a load.

9.2 The hart

Fetch reads a thirty-two-bit parcel. Decode recognises RV32I and the OP-V opcodes of SVM-8. Execute for an integer instruction runs four byte beats through an eight-bit ALU, with a carry flop between beats. Writeback commits the whole register or, on a fault, commits nothing.

The default image is an RV32I hart. R800 software runs on that image only if you have written an interpreter or a translator in RV32I. The peripherals do not know which program is running. They answer to port writes in the MSX map.

A second elaboration sets `host = R800` and attaches SVM-8 to an R800 softcore you already have. The book specifies the adaptor, not the R800 pipeline. The escape, the register map, and the harness files are in the host section below.

The multiplier, if you include the M extension, is a sequenced eight-bit multiplier, not a DSP that produces a thirty-two-bit product in one cycle. Leaving M out of the first image is acceptable. Vector byte adds cover the inner loops that would otherwise have wanted a fast scalar multiply.

9.3 SVM-8

ELEN = 8, VLEN = 32, thirty-two registers, LMUL = 1.
Legal SEW = 8 only.
Required: `vsetvl`, `vsetvli`, unit-stride load and store,
add, subtract, logic, shifts, compare, saturating add, splat (`vmv.v.x`),
vector-vector and vector-scalar forms.
Undisturbed tail and mask.
No floating point, no indexed load, no Xsvm.

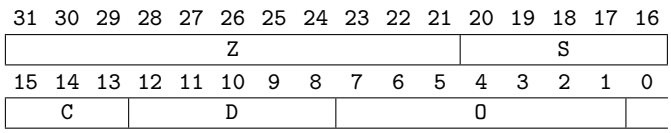
Four elements fill a register. With four lanes, a vector add retires in one cycle after the sources are read. With one lane, it retires in four. Either organisation passes the same architectural tests.

The load unit takes the base address from an integer register, checks that `vl` bytes lie in a mapped region, and then consumes `vl` bus beats when the arbiter grants them. Video refresh has priority on that arbiter. A vector load that loses the bus for a scanline simply waits. That wait is the MSX-compatible behaviour worth preserving: the picture does not tear because a copy loop held the bus.

9.4 Opcode map for SVM-8

All encodings are 32-bit. The diagrams use the approved two-line rigid matrix with single-letter field codes. Only the forms legal under ELEN = 8 and LMUL = 1 are shown. Every other OP-V encoding traps.

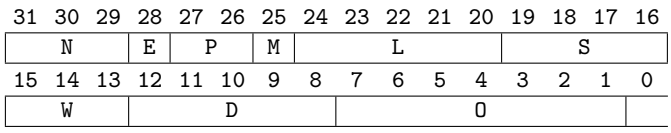
9.4.1 Configuration — `vsetvli`



Z = zimm[10:0] (vtype immediate)
S = rs1
C = 111 (OPCFG)
D = rd
O = 1010111

`vsetvl` replaces the zimm field with rs2. `vsetivli` (if implemented) uses a 5-bit immediate in place of rs1.

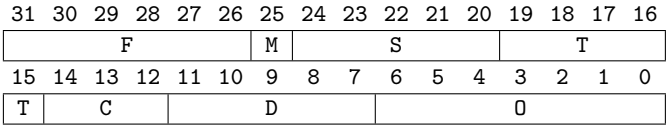
9.4.2 Unit-stride load — `vle8.v`



N = nf (normally 000)
E = mew
P = mop (00 = unit-stride)
M = vm
L = lumop (00000 = unit-stride)
S = rs1 (base)
W = width (000 = 8-bit)
D = vd
O = 0000111

Store form `vse8.v` uses major opcode 0100111 and vs3 in the D field.

9.4.3 Integer arithmetic (OPIVV / OPIVX)



F = funct6
M = vm
S = vs2
T = vs1 (vv) or rs1 (vx)
C = funct3 (000 = OPIVV, 100 = OPIVX)
D = vd
O = 1010111

Instruction	funct6	form
`vadd`	000000	vv / vx
`vsub`	000010	vv / vx
`vrsub`	000011	vx only
`vand`	001001	vv / vx
`vor`	001010	vv / vx
`vxor`	001011	vv / vx
`vsll`	100101	vv / vx
`vsrl`	101000	vv / vx
`vsra`	101001	vv / vx
`vsadd`	100001	vv / vx
`vssub`	100011	vv / vx (optional)

9.4.4 Compare (mask result)

Same layout as integer arithmetic. funct3 = 000 (OPIVV) or 100 (OPIVX). Result is written to the mask register `v0` when vd = 0, or to the named mask destination.

Instruction	funct6	form
`vmseq`	011000	vv / vx
`vmsne`	011001	vv / vx
`vmsltu`	011010	vv / vx
`vmslt`	011011	vv / vx
`vmsleu`	011100	vv / vx
`vmsle`	011101	vv / vx
`vmsgtu`	011110	vx only
`vmsgt`	011111	vx only

9.4.5 Move and splat

Instruction	encoding notes
`vmv.v.v`	funct6 = 010111, vs1 = vs2, OPIVV
`vmv.v.x`	funct6 = 010111, OPIVX
`vmv.v.i`	funct6 = 010111, OPIVI
`vmv.x.s`	whole-register scalar extract
`vmv.s.x`	whole-register scalar insert

9.4.6 Mask logicals

Instruction	funct6
`vmand`	011001
`vmnand`	011101
`vmandn`	011000
`vmor`	011010
`vmnor`	011110
`vmorn`	011100
`vmxor`	011011
`vmxnor`	011111
`vcpop`	010000 (OPMVV)
`vfirst`	010001 (OPMVV)

No widening, no narrowing, no stride, no indexed, no reduction, no slide, no floating-point, and no custom geometry opcodes are legal on SVM-8.

9.5 R800 fetch

The R800 elaboration keeps SVM-8 and replaces the RV32 fetch with an R800 softcore. The adaptor record is the one in Chapter 1. This section fills the R800 column and the wiring.

The escape is the two bytes `ED 00`. The next four bytes are one OP-V parcel, low byte first. `ED C1` and its multiples are MULUB. `ED C3`, `ED D3`, `ED E3`, and `ED F3` are MULUW. Those stay. `ED 00` is the first empty slot in the `ED` map. On a real R800 that slot behaves as a slow NOP, so a binary that used it as padding would change meaning. MSX padding is the byte `00`.

rs1[2:0]	R800 pair
0	BC
1	DE
2	HL
3	IX
4	IY
5	SP
6, 7	reserved, illegal

An address-bearing vector instruction uses the selected pair as a byte address. `vsetvl` writes `vl` into the destination pair. The vector file is `v0`–`v31` inside the SVM. An illegal parcel raises the R800 illegal-instruction exception.

The harness files in `kernels/bx8/r800/` put `ED 00` in front of every RV parcel, scalar words included, and they still contain RV register numbers. A ROM uses the escape only on OP-V and writes the branch in R800 instructions. One strip of a byte copy, with `rs1` already reassembled so that 0 is BC, 1 is DE, and 2 is HL, looks like this. `BC` holds the remaining count, `HL` the source, `DE` the destination.

```
; BC = n, HL = src, DE = dst
    ld   a, b
    or   c
    ret  z
    ; vsetvli  t-pair, BC, e8, m1, tu, mu
    defb 0xED, 0x00, <parcel, low byte first>
    ; vle8.v v1, (HL)
    defb 0xED, 0x00, <parcel>
    ; vse8.v v1, (DE)
    defb 0xED, 0x00, <parcel>
```

The softcore stalls from the escape until the SVM accepts the parcel. Video refresh still wins the bus. The picture rule in the SVM-8 section does not change because the fetch path changed.

9.6 Assembler kernels that must run

These routines are the acceptance set for every BX-8 image. They use only the opcodes listed above and must produce bit-identical results to a scalar reference model. The hex is the RV hart image. The R800 file next to it is the harness frame.

Table 9.1: Kernel analogues on this machine.

Kernel	RVV	Intel	ARM
fill8	`vse8.v`	`movdqu`	`st1`
copy8	`vle8.v`	`movdqu`	`ld1`
mask8	`vmsltu.vx`	`psubusb`	`cmhi`
add8	`vsadd.vv`	`paddusb`	`uqadd`

9.6.1 Where these kernels run

Each comparison table is three assembled strips. The columns are the ISA, the address, the encoding, and the instruction. RVV addresses are byte offsets in the acceptance image, and the hex is the instruction word. Intel bytes are x86-64 in memory order, with no spaces between bytes. They are assembled by clang with AVX2 and FMA, and without AVX-512, except `wide32`

Table 9.2: Programs that run the BX-8 kernels.

Kernel	Work	Where it ships
fill8	byte fill	C `memset`. Edge padding of a picture in FFmpeg. Silence in 8-bit PCM. A cleared tile.
copy8	byte move	C `memcpy`. Prediction copy in MPEG-2, MPEG-4, H.264, HEVC, VC-1, and AV1: x264, x265, dav1d, SVT-AV1, FFmpeg. A blitter rectangle.
mask8	threshold	A chroma-key. The predicate inside a deblocking filter. A byte kept or replaced against a limit.
add8	saturating add	The clip of a residual back into a byte. 8-bit PCM that saturates. VC-1 overlap smoothing. Intel `paddusb`.

and `wide64`, whose Intel strip is EVEX because that register file is 512 bits. A splat from a general register is `movd` and a shuffle, or `vmovq` and `vpbroadcastq` from an `xmm`. ARM bytes are AArch64 NEON words, assembled by clang. SVE is not in these bytes. An indexed load on NEON is a scalar `ldr` and `str`. `psubusb` then `pcmpeqb` is the unsigned compare. `pcmpgtb` is signed and is not used. `pmullw` alone is the low half of a product. The `vwmul16` Intel strip is `pmullw`, `pmulhw`, unpack, and `packssdw`. The ARM strip is `smull` and `smull2`, then `sqxtn` and `sqxtn2`. `vfmadd231ps` and AArch64 `fmla` are fused. A separate multiply and add is not. The GEMM strip is one rank-1 update: a vector load, a scalar broadcast, and a fused multiply-add into each accumulator. `vbroadcastss`, `vmovddup`, and `ld1r` are that broadcast. `wide32` and `wide64` are that update on a 512-bit register, four columns, stride `vl`. A 24 by 8 tile is still wider than those four accumulators. The ARM column of that strip stays NEON. A label shares its row with the following instruction. On `asum` and `iamax` the Intel mask in `xmm7` is the absolute-value mask, the same role as the byte mask in `mask8`. The index itself is the RVV `vfirst.m`. `quad`, `add128`, `addd16`, `zext64`, and `sh64` are RV128I integer strips. Their first column is labeled RV128. `lq`, `sq`, `ldu`, and `addid` use the decoder placement in Chapter 16. `addi` is the ordinary XLEN add. `slli` by 64 uses the seven-bit immediate. The Intel and ARM columns of those strips keep 64-bit general registers and write the high half explicitly, except `quad`, which moves 16 bytes through an `xmm` and through NEON. SVE and AVX-512 are not in those bytes. `sad16` is a halfword absolute difference and a horizontal sum. `pabsw` is the SSSE3 absolute value. `psadbw` sums bytes into words and is a different instruction. `sw8` is `paddusb` and `pmaxub`. `wmul32`, `bfly32`, and `cmul32` are `vmulps`, `vaddps`, and `vsubps`. Those three are not fused.

9.6.2 fill8 — fill a region with a byte

`kernels/bx8/fill8.sew8.lmul1.tu.mu.vlen32.xlen32.le.bin`

9.6.3 copy8 — unit-stride byte copy

`kernels/bx8/copy8.sew8.lmul1.tu.mu.vlen32.xlen32.le.bin`

9.6.4 mask8 — selective fill under a generated mask

`kernels/bx8/mask8.sew8.lmul1.tu.mu.vlen32.xlen32.le.bin`

9.6.5 add8 — element-wise byte add with saturation

`kernels/bx8/add8.sew8.lmul1.tu.mu.vlen32.xlen32.le.bin`

9.7 What you elaborate

Table 9.3: Kernel comparison: fill8

`a0 = n, a1 = fill byte, a2 = dst`

ISA	Address	Hex	Assembly
RVV	0000	00050E63	`fill8: beqz a0, end`
RVV	0004	000572D7	`loop: vsetvli t0, a0, e8, m1, tu, mu`
RVV	0008	5E05C0D7	`vmv.v.x v1, a1`
RVV	000C	020600A7	`vse8.v v1, (a2)`
RVV	0010	00560633	`add a2, a2, t0`
RVV	0014	40550533	`sub a0, a0, t0`
RVV	0018	FE0516E3	`bnez a0, loop`
RVV	001C	00008067	`end: ret`
Intel	0000	F30F7F07	`fill8: movdqu [rdi], xmm0`
Intel	0004	4883C710	`add rdi, 16`
Intel	0008	4883EE10	`sub rsi, 16`
Intel	000C	77F2	`ja fill8`
ARM	0000	4E010C20	`fill8: dup v0.16b, w1`
ARM	0004	4C007040	`st1 {v0.16b}, [x2]`
ARM	0008	91004042	`add x2, x2, #16`
ARM	000C	F1004000	`subs x0, x0, #16`
ARM	0010	54FFFF88	`b.hi fill8`

Table 9.4: Kernel comparison: copy8

`a0 = n, a1 = src, a2 = dst`

ISA	Address	Hex	Assembly
RVV	0000	02050063	`copy8: beqz a0, end`
RVV	0004	000572D7	`loop: vsetvli t0, a0, e8, m1, tu, mu`
RVV	0008	02058087	`vle8.v v1, (a1)`
RVV	000C	020600A7	`vse8.v v1, (a2)`
RVV	0010	005585B3	`add a1, a1, t0`
RVV	0014	00560633	`add a2, a2, t0`
RVV	0018	40550533	`sub a0, a0, t0`
RVV	001C	FE0514E3	`bnez a0, loop`
RVV	0020	00008067	`end: ret`
Intel	0000	F30F6F06	`copy8: movdqu xmm0, [rsi]`
Intel	0004	F30F7F07	`movdqu [rdi], xmm0`
Intel	0008	4883C610	`add rsi, 16`
Intel	000C	4883C710	`add rdi, 16`
Intel	0010	4883EA10	`sub rdx, 16`
Intel	0014	77EA	`ja copy8`
ARM	0000	4C407020	`copy8: ld1 {v0.16b}, [x1]`
ARM	0004	4C007040	`st1 {v0.16b}, [x2]`
ARM	0008	91004021	`add x1, x1, #16`
ARM	000C	91004042	`add x2, x2, #16`
ARM	0010	F1004000	`subs x0, x0, #16`
ARM	0014	54FFFF68	`b.hi copy8`

Table 9.5: Kernel comparison: mask8

`a0 = n, a1 = fill, a2 = dst, a3 = threshold`

ISA	Address	Hex	Assembly
RVV	0000	02050263	`mask8: beqz a0, end`
RVV	0004	000572D7	`loop: vsetvli t0, a0, e8, m1, tu, mu`
RVV	0008	02060087	`vle8.v v1, (a2)`
RVV	000C	6A16C057	`vmsltu.vx v0, v1, a3`
RVV	0010	5E05C157	`vmv.v.x v2, a1`
RVV	0014	00060127	`vse8.v v2, (a2), v0.t`
RVV	0018	00560633	`add a2, a2, t0`
RVV	001C	40550533	`sub a0, a0, t0`
RVV	0020	FE0512E3	`bnez a0, loop`
RVV	0024	00008067	`end: ret`
Intel	0000	F30F6F07	`mask8: movdqu xmm0, [rdi]`
Intel	0004	660F6FCE	`movdqa xmm1, xmm6`
Intel	0008	660FD8C8	`psubusb xmm1, xmm0`
Intel	000C	660FEFD2	`pxor xmm2, xmm2`
Intel	0010	660F74CA	`pcmpeqb xmm1, xmm2`
Intel	0014	660FDBC1	`pand xmm0, xmm1`
Intel	0018	660FDFCD	`pandn xmm1, xmm5`
Intel	001C	660FEBC1	`por xmm0, xmm1`
Intel	0020	F30F7F07	`movdqu [rdi], xmm0`
ARM	0000	4C407040	`mask8: ld1 {v0.16b}, [x2]`
ARM	0004	6E203422	`cmhi v2.16b, v1.16b, v0.16b`
ARM	0008	6EA21C60	`bit v0.16b, v3.16b, v2.16b`
ARM	000C	4C007040	`st1 {v0.16b}, [x2]`

Table 9.6: Kernel comparison: add8

`a0 = n, a1 = src0, a2 = src1, a3 = dst`

ISA	Address	Hex	Assembly
RVV	0000	02050663	`add8: beqz a0, end`
RVV	0004	000572D7	`loop: vsetvli t0, a0, e8, m1, tu, mu`
RVV	0008	02058087	`vle8.v v1, (a1)`
RVV	000C	02060107	`vle8.v v2, (a2)`
RVV	0010	861101D7	`vsadd.vv v3, v1, v2`
RVV	0014	020681A7	`vse8.v v3, (a3)`
RVV	0018	005585B3	`add a1, a1, t0`
RVV	001C	00560633	`add a2, a2, t0`
RVV	0020	005686B3	`add a3, a3, t0`
RVV	0024	40550533	`sub a0, a0, t0`
RVV	0028	FC051EE3	`bnez a0, loop`
RVV	002C	00008067	`end: ret`
Intel	0000	F30F6F06	`add8: movdqu xmm0, [rsi]`
Intel	0004	F30F6F0A	`movdqu xmm1, [rdx]`
Intel	0008	660FDCC1	`paddusb xmm0, xmm1`
Intel	000C	F30F7F07	`movdqu [rdi], xmm0`
ARM	0000	4C407020	`add8: ld1 {v0.16b}, [x1]`
ARM	0004	4C407041	`ld1 {v1.16b}, [x2]`
ARM	0008	6E210C00	`uqadd v0.16b, v0.16b, v1.16b`
ARM	000C	4C007060	`st1 {v0.16b}, [x3]`

```
typedef enum { SVM8, SVM16, ZVE32F, V64 } Profile;
Profile profile = SVM8;
Integer elen = 8;
Integer vlen = 32;
Integer lanes = 1;          // or 4
Bool xsvm = False;
Bool vsetivli = False;      // True only if decoder already has it
```

The same package, with different parameters, is BX-16 and the later machines. BX-8 is the parameter set in which most of the package is illegal-instruction logic rather than arithmetic.

Chapter 10

BX-16: An Amiga-Compatible Machine with SVM-16

10.1 The board-level picture

BX-16 can replace an Amiga in the same practical sense that BX-8 can replace an MSX. The bus is sixteen bits wide. Several masters share it: the hart, a blitter-class block mover, video fetch, and audio DMA. Chip memory and a larger local memory sit on that bus. The display timing can follow a programmed list of fetches, in the role a copper list fills on the original machine. The hart that runs the list is RV32I with a sixteen-bit data path. Chapter 8 is that integer file.

Integer registers remain thirty-two bits. A load word takes two halfword beats. Instruction fetch is again allowed its own port so that video DMA does not stall the decode of every parcel.

10.2 SVM-16

ELEN = 16, VLEN = 128, thirty-two registers.
Legal SEW = 8 or 16.
LMUL may be greater than 1 only for a widening destination.
Required, beyond SVM-8: strides, widening multiply,
narrowing shift, saturating add and subtract,
integer add reduction, slide by one, `vsetivli`.
No floating point. Xsvm optional.

A register holds sixteen elements of eight bits or eight elements of sixteen bits. That is enough for a short span of a scanline or a frame of samples. Longer spans strip-mine, and the bus arbiter inserts the video and audio masters between strips. Write the arbiter so that a maximum latency for the audio master is enforced. A vector copy that can lock out audio is not an Amiga-compatible bus, however correct its RVV tests are.

10.3 Widening, not a private fixed-point format

A sixteen-bit multiply that keeps the full product uses `vwmul.vv`. The destination elements are thirty-two bits wide, in a register group. A following narrowing shift brings a saturated

sixteen-bit pixel back to SEW=16. Software that wants a Q16.16 convention shifts by sixteen. The lane does not test a mode bit named Q16.16.

This is the operation a blitter-class scaler and a small transform both need. Implementing it as a special coprocessor instruction would make the kernel unreadable on BX-32. Implementing it as `vwmul` means the same source runs on Zve32f.

10.4 Xsvm

When the elaboration parameter `xsvm` is true, two custom-0 instructions are decoded.

The matrix instruction reads four vector registers starting at a fixed base, `v8` through `v11`, as the rows of a four-by-four matrix, multiplies them by `vs2`, and writes `vd`. Using `v8` rather than `v0` keeps the mask register free. The cross-product instruction reads two registers and writes one, three elements, at the current SEW, with `vl` forced to a minimum of three or the instruction is illegal.

Both instructions are undisturbed outside `vl`. Both are absent from the decoder when `xsvm` is false, and the parcels trap. The exercises that must pass on every BX-16 image do not use them.

10.5 Opcode map for SVM-16

Everything legal on SVM-8 remains legal. The additional classes required by SVM-16 are shown with the same rigid two-line matrix. SEW may be 8 or 16. The complete cross-profile contract and full opcode summary appear after the machines, in Chapter 17.

10.5.1 Configuration

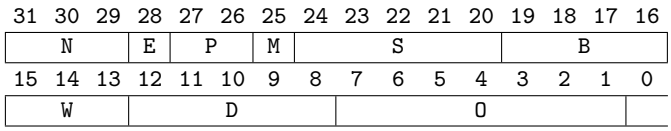
Identical to SVM-8. `vsetivli` is now required.

10.5.2 Unit-stride load/store (EEW = 8 and 16)

Same layout as SVM-8. Width field:

- 000 = 8-bit (`vle8.v`/`vse8.v`)
- 101 = 16-bit (`vle16.v`/`vse16.v`)

10.5.3 Constant-stride load — `vlse16.v`



N = nf
E = mew
P = mop (10 = strided)
M = vm
S = rs2 (stride)
B = rs1 (base)
W = width (101 = 16-bit)
D = vd
O = 0000111

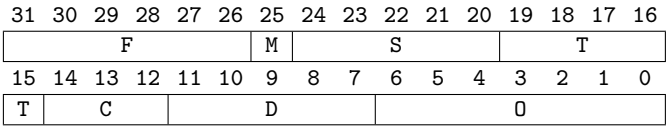
Store form uses major opcode 0100111.

10.5.4 Integer arithmetic (SEW = 8, 16)

Same OP-V layout as SVM-8. Additional instructions:

Instruction	funct6	notes
`vminu/vmin`	000100 / 000101	vv / vx
`vmaxu/vmax`	000110 / 000111	vv / vx
`vsaddu/vsadd`	100000 / 100001	saturating
`vssubu/vssub`	100010 / 100011	saturating

10.5.5 Widening multiply and narrowing



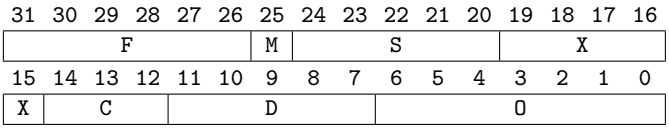
F = funct6
M = vm
S = vs2
T = vs1 / rs1
C = funct3 (OPIVV / OPIVX)
D = vd (group for widening)
O = 1010111

Instruction	funct6	width rule
`vwmul`	111011	SEW → 2×SEW
`vwmulu`	111000	unsigned
`vwmulsu`	111010	mixed
`vnsrl`	101100	narrowing shift
`vnsra`	101101	narrowing arithmetic
`vnclipu/vnclip`	101110 / 101111	narrowing clip

10.5.6 Reduction and slide

Instruction	funct6 / notes
`vredsum.vs`	000000 (OPMVV)
`vslide1up.vx`	001110 (OPIVX)
`vslide1down.vx`	001111 (OPIVX)

10.5.7 Optional Xsvm (custom-0)



F = custom funct6
M = vm
S = vs2
X = unused / fixed
C = custom funct3
D = vd
O = 0001011 (custom-0)

Matrix and cross-product are distinguished by the custom funct6 / funct3 values documented in the profile card. They are illegal when `xsvm = False`.

10.6 68000, SH-2, and SH-4 fetch

The adaptor is the record in Chapter 1. BX-16 fills three columns. It does not restate the mechanism.

Host	Escape	Endian	`rs1`
68000	`F400`	big, high halfword first	D0–D7, then A0–A7
SH-2	`0000`	big, high halfword first	R0–R15
SH-4	`0000`	little, low halfword first	R0–R15

`F400` is F-line, coprocessor id 2, general type, effective-address field zero. Coprocessor id 0 is the PMMU slot. Coprocessor id 1 is the 68881. The two extension words are the OP-V parcel.

`0000` is undefined on SH-2 and SH-4. The defined NOP is `0009`. Saturn fetch is big-endian, so the SH-2 frame and the SH-4 frame are different files. Dreamcast fetch is little-endian.

Do not claim these words. An A-line word is the Amiga library trap. An SH-4 `Fxxx` word is the FPU, including `FTRV` and `FIPR`. SVM-16 does not set `misa`.V. SH-4 on this machine is an integer injection: ELEN is 16, and there is no FP32. A floating-point transform uses the same escape in front of BX-32.

The 68000 exception for a parcel the SVM rejects is F-line. The SuperH exception is illegal instruction. The blitter remains a peripheral master. It does not fetch OP-V.

Harness files live in `kernels/bx16/m68k/`, `kernels/bx16/sh2/`, and `kernels/bx16/sh4/`. They frame every RV parcel. A ROM re-assembles `rs1` and writes the branch in the host ISA.

10.7 Assembler kernels that must run

In addition to every kernel of Chapter 9, the following must pass on every BX-16 image (Xsvm kernels are excluded from the travelling suite). `redsum16` strip-mines. A single `vsetvli` would keep only `VLMAX` elements, which is 8 at SEW 16, LMUL 1, VLEN 128. The per-strip sum is a wrapped SEW-16 result; `a2` accumulates those results in XLEN.

Table 10.1: Kernel analogues on this machine.

Kernel	RVV	Intel	ARM
sew16	`vse16.v`	`movdqu`	`st1`
vwmul16	`vwmul.vv`	`pmullw`	`smull`
stride16	`vlse16.v`	`pinsrw`	`ld1`
redsum16	`vredsum.vs`	`phaddw`	`addp`
sad16	`vmax.vv`	`pabsw`	`sabd`
sw8	`vsaddu.vv`	`pmaxub`	`umax`

10.7.1 Where these kernels run

Table 10.2: Programs that run the BX-16 kernels.

Kernel	Work	Where it ships
sew16	halfword fill	Silence in 16-bit PCM. A cleared H.264 residual buffer.
vwmul16	Q15 multiply, then saturate	CMSIS-DSP `arm_scale_q15`. A blitter scaler. Fixed-point gain in sox and in FFmpeg's integer resampler.
stride16	strided halfword gather	Deinterleave of stereo S16, stride 4 bytes. One column of a 16-bit image. One channel of interleaved int16 IQ.
redsum16	halfword sum	Block energy after a Q15 square. The DC of a small residual. The horizontal add under `sad16`.
sad16	sum of absolute differences	Motion estimation. x264 `pixel_sad`, x265, dav1d, SVT-AV1, FFmpeg. MPEG-2, MPEG-4, H.264, HEVC, VC-1, AV1.
sw8	saturating add, then max	One stripe of Farrar Smith–Waterman. parasail, SSW, SWIPE.

The comparison table under each kernel is one row per instruction: RVV, then Intel, then ARM. Chapter 9 says which encodings those bytes are.

10.7.2 sew16 — halfword fill

Table 10.3: Kernel comparison: sew16

`a0 = n (elements), a1 = fill halfword, a2 = dst`

ISA	Address	Hex	Assembly
RVV	0000	02050063	`sew16:  beqz a0, end`
RVV	0004	001572D7	`loop:  vsetvli t0, a0, e16, m1, tu, mu`
RVV	0008	5E05C0D7	`vmv.v.x v1, a1`
RVV	000C	020650A7	`vse16.v v1, (a2)`
RVV	0010	00129313	`slli t1, t0, 1`
RVV	0014	00660633	`add a2, a2, t1`
RVV	0018	40550533	`sub a0, a0, t0`
RVV	001C	FE0514E3	`bnez a0, loop`
RVV	0020	00008067	`end:  ret`
Intel	0000	660F6EC6	`sew16:  movd xmm0, esi`
Intel	0004	F20F70C000	`pshuflw xmm0, xmm0, 0`
Intel	0009	660F6CC0	`punpcklqdq xmm0, xmm0`
Intel	000D	F30F7F07	`movdqu [rdi], xmm0`
Intel	0011	4883C710	`add rdi, 16`
Intel	0015	4883EA08	`sub rdx, 8`
Intel	0019	77E5	`ja sew16`
ARM	0000	4E020C20	`sew16:  dup v0.8h, w1`
ARM	0004	4C007440	`st1 {v0.8h}, [x2]`
ARM	0008	91004042	`add x2, x2, #16`
ARM	000C	F1002000	`subs x0, x0, #8`
ARM	0010	54FFFF88	`b.hi sew16`

`kernels/bx16/sew16.sew16.lmul1.tu.mu.vlen128.xlen32.le.bin`

10.7.3 vwmul16 — widening multiply then narrow with saturation

`kernels/bx16/vwmul16.sew16.lmul1.tu.mu.vlen128.xlen32.le.bin`

10.7.4 stride16 — constant-stride gather of halfwords

The base in `a1` is not advanced. The table re-reads the same strip when `n` exceeds `VLMAX`. An acceptance test uses `n` at most `VLMAX`, or the reader adds the byte advance `vl * stride`. That

Table 10.4: Kernel comparison: vwmul16

`a0 = n, a1 = src0, a2 = src1, a3 = dst.  Wide product, then clip.`

ISA	Address	Hex	Assembly
RVV	0000	02050C63	vwmul16: beqz a0, end
RVV	0004	001572D7	loop: vsetvli t0, a0, e16, m1, tu, mu
RVV	0008	0205D107	vle16.v v2, (a1)
RVV	000C	02065207	vle16.v v4, (a2)
RVV	0010	EE222457	vwmul.vv v8, v2, v4
RVV	0014	001572D7	vsetvli t0, a0, e16, m1, tu, mu
RVV	0018	BE8030D7	vnclip.wi v1, v8, 0
RVV	001C	0206D0A7	vse16.v v1, (a3)
RVV	0020	00129313	slli t1, t0, 1
RVV	0024	006585B3	add a1, a1, t1
RVV	0028	00660633	add a2, a2, t1
RVV	002C	006686B3	add a3, a3, t1
RVV	0030	40550533	sub a0, a0, t0
RVV	0034	FC0518E3	bnez a0, loop
RVV	0038	00008067	end: ret
Intel	0000	660F6FD0	vwmul16: movdqa xmm2, xmm0
Intel	0004	660FD5C1	pmullw xmm0, xmm1
Intel	0008	660FE5D1	pmulhw xmm2, xmm1
Intel	000C	660F6FD8	movdqa xmm3, xmm0
Intel	0010	660F61C2	punpcklwd xmm0, xmm2
Intel	0014	660F69DA	punpckhwd xmm3, xmm2
Intel	0018	660F6BC3	packssdw xmm0, xmm3
ARM	0000	0E61C002	vwmul16: smull v2.4s, v0.4h, v1.4h
ARM	0004	4E61C003	smull2 v3.4s, v0.8h, v1.8h
ARM	0008	0E614844	sqxtn v4.4h, v2.4s
ARM	000C	4E614864	sqxtn2 v4.8h, v3.4s

advance is a multiply. M is optional on this machine, so the printed kernel does not depend on it.

Table 10.5: Kernel comparison: stride16

`a0 = n, a1 = base, a2 = stride (bytes), a3 = dst.  Base stays.`

ISA	Address	Hex	Assembly
RVV	0000	02050063	stride16: beqz a0, end
RVV	0004	001572D7	loop: vsetvli t0, a0, e16, m1, tu, mu
RVV	0008	0AC5D087	vlse16.v v1, (a1), a2
RVV	000C	0206D0A7	vse16.v v1, (a3)
RVV	0010	00129313	slli t1, t0, 1
RVV	0014	006686B3	add a3, a3, t1
RVV	0018	40550533	sub a0, a0, t0
RVV	001C	FE0514E3	bnez a0, loop
RVV	0020	00008067	end: ret
Intel	0000	660FC40600	stride16: pinsrw xmm0, [rsi], 0
Intel	0005	4801D6	add rsi, rdx
Intel	0008	660FC40601	pinsrw xmm0, [rsi], 1
Intel	000D	4801D6	add rsi, rdx
Intel	0010	660FC40602	pinsrw xmm0, [rsi], 2
Intel	0015	4801D6	add rsi, rdx
Intel	0018	660FC40603	pinsrw xmm0, [rsi], 3
Intel	001D	F30F7F07	movdqu [rdi], xmm0
ARM	0000	0DC24020	stride16: ld1 {v0.h}[0], [x1], x2
ARM	0004	0DC24820	ld1 {v0.h}[1], [x1], x2
ARM	0008	0DC25020	ld1 {v0.h}[2], [x1], x2
ARM	000C	0DC25820	ld1 {v0.h}[3], [x1], x2

`kernels/bx16/stride16.sew16.lmul1.tu.mu.vlen128.xlen32.le.bin`

10.7.5 redsum16 — integer reduction

Table 10.6: Kernel comparison: redsum16

a0 = n, a1 = src, a2 = sum out. Each strip wraps in 16 bits.

ISA	Address	Hex	Assembly
RVV	0000	00000613	redsum16: li a2, 0
RVV	0004	02050663	beqz a0, end
RVV	0008	001572D7	loop: vsetvli t0, a0, e16, m1, tu, mu
RVV	000C	0205D087	vle16.v v1, (a1)
RVV	0010	5E003157	vmv.v.i v2, 0
RVV	0014	02112157	vredsum.vs v2, v1, v2
RVV	0018	42202357	vmv.x.s t1, v2
RVV	001C	00660633	add a2, a2, t1
RVV	0020	00129313	slli t1, t0, 1
RVV	0024	006585B3	add a1, a1, t1
RVV	0028	40550533	sub a0, a0, t0
RVV	002C	FC051EE3	bnez a0, loop
RVV	0030	00008067	end: ret
Intel	0000	660F3801C0	redsum16: phaddw xmm0, xmm0
Intel	0005	660F3801C0	phaddw xmm0, xmm0
Intel	000A	660F3801C0	phaddw xmm0, xmm0
Intel	000F	660FC5C000	pextrw eax, xmm0, 0
ARM	0000	4E60BC00	redsum16: addp v0.8h, v0.8h, v0.8h
ARM	0004	4E60BC00	addp v0.8h, v0.8h, v0.8h
ARM	0008	4E60BC00	addp v0.8h, v0.8h, v0.8h
ARM	000C	0E023C00	umov w0, v0.h[0]

`kernels/bx16/redsum16.sew16.lmul1.tu.mu.vlen128.xlen32.le.bin`

10.7.6 sad16 — sum of absolute differences

Signed halfwords. An 8-bit pixel promoted into a halfword has a difference from −255 to 255, and eight of those fit in the signed strip sum. Full-scale 16-bit audio can wrap inside the strip. `a3` accumulates the sign-extended piece, the same rule as `redsum16`. `pabsw` is the SSSE3 halfword absolute value. `psadbw` sums bytes into words.

`kernels/bx16/sad16.sew16.lmul1.tu.mu.vlen128.xlen32.le.bin`

10.7.7 sw8 — one Smith–Waterman stripe

The profile bytes are already biased into the unsigned range a saturating add expects. The stripe adds that profile into the running score and takes the unsigned maximum against the gap vector. Farrar's layout makes the cells of one stripe independent. The zero floor is the same `vmaxu` against a register of zeros. A seed hit in BLAST or minimap2 is the gather `index32`.

`kernels/bx16/sw8.sew8.lmul1.tu.mu.vlen128.xlen32.le.bin`

10.8 Harts

Scalar parcels and vector parcels share decode, the scoreboard, and writeback. The blitter, if it is a bus master, is not a second hart and does not fetch OP-V. It is a peripheral with a pointer, a count, and a small arithmetic step, programmed by stores to its registers. New software may ignore it and use SVM-16 instead. Old software that you have translated may still program it. Keeping both is how the machine remains Amiga-compatible while the new kernels are ordinary RVV.

Table 10.7: Kernel comparison: sad16

```
a0 = n, a1 = src0, a2 = src1, a3 = sum.  Sum of |a-b|.  Each strip wraps in 16 bits.
```

ISA	Address	Hex	Assembly
RVV	0000	00000693	sad16: li a3, 0
RVV	0004	04050063	beqz a0, end
RVV	0008	001572D7	loop: vsetvli t0, a0, e16, m1, tu, mu
RVV	000C	0205D087	vle16.v v1, (a1)
RVV	0010	02065107	vle16.v v2, (a2)
RVV	0014	0A1101D7	vsub.vv v3, v1, v2
RVV	0018	0A208257	vsub.vv v4, v2, v1
RVV	001C	1E3201D7	vmax.vv v3, v3, v4
RVV	0020	5E003257	vmv.v.i v4, 0
RVV	0024	02322257	vredsum.vs v4, v3, v4
RVV	0028	42402357	vmv.x.s t1, v4
RVV	002C	006686B3	add a3, a3, t1
RVV	0030	00129313	slli t1, t0, 1
RVV	0034	006585B3	add a1, a1, t1
RVV	0038	00660633	add a2, a2, t1
RVV	003C	40550533	sub a0, a0, t0
RVV	0040	FC0514E3	bnez a0, loop
RVV	0044	00008067	end: ret
Intel	0000	F30F6F06	sad16: movdqu xmm0, [rsi]
Intel	0004	F30F6F0A	movdqu xmm1, [rdx]
Intel	0008	660FF9C1	psubw xmm0, xmm1
Intel	000C	660F381DC0	pabsw xmm0, xmm0
Intel	0011	660F3801C0	phaddw xmm0, xmm0
Intel	0016	660F3801C0	phaddw xmm0, xmm0
Intel	001B	660F3801C0	phaddw xmm0, xmm0
Intel	0020	660FC5C000	pextrw eax, xmm0, 0
ARM	0000	4C407420	sad16: ld1 {v0.8h}, [x1]
ARM	0004	4C407441	ld1 {v1.8h}, [x2]
ARM	0008	4E617400	sabd v0.8h, v0.8h, v1.8h
ARM	000C	4E60BC00	addp v0.8h, v0.8h, v0.8h
ARM	0010	4E60BC00	addp v0.8h, v0.8h, v0.8h
ARM	0014	4E60BC00	addp v0.8h, v0.8h, v0.8h
ARM	0018	0E023C00	umov w0, v0.h[0]

10.9 Elaboration

```
Profile profile = SVM16;
Integer elen = 16;
Integer vlen = 128;
Integer lanes = 4;
Bool xsvm = False; // True only on a geometry image
```

Four lanes of sixteen bits retire four elements per cycle. A full SEW=16 register then takes two arithmetic cycles, plus the read and write of the file. Publish that number. Programmers of audio mixing will plan their strip length around it.

Table 10.8: Kernel comparison: sw8

`a0 = n, a1 = H, a2 = profile, a3 = E. Saturating add, then unsigned max.`

ISA	Address	Hex	Assembly
RVV	0000	02050A63	`sw8: beqz a0, end`
RVV	0004	000572D7	`loop: vsetvli t0, a0, e8, m1, tu, mu`
RVV	0008	02058087	`vle8.v v1, (a1)`
RVV	000C	02060107	`vle8.v v2, (a2)`
RVV	0010	821100D7	`vsaddu.vv v1, v1, v2`
RVV	0014	02068187	`vle8.v v3, (a3)`
RVV	0018	1A1180D7	`vmaxu.vv v1, v1, v3`
RVV	001C	020580A7	`vse8.v v1, (a1)`
RVV	0020	005585B3	`add a1, a1, t0`
RVV	0024	00560633	`add a2, a2, t0`
RVV	0028	005686B3	`add a3, a3, t0`
RVV	002C	40550533	`sub a0, a0, t0`
RVV	0030	FC051AE3	`bnez a0, loop`
RVV	0034	00008067	`end: ret`
Intel	0000	F30F6F06	`sw8: movdqu xmm0, [rsi]`
Intel	0004	F30F6F0A	`movdqu xmm1, [rdx]`
Intel	0008	660FDCC1	`paddusb xmm0, xmm1`
Intel	000C	F30F6F09	`movdqu xmm1, [rcx]`
Intel	0010	660FDEC1	`pmaxub xmm0, xmm1`
Intel	0014	F30F7F06	`movdqu [rsi], xmm0`
ARM	0000	4C407020	`sw8: ld1 {v0.16b}, [x1]`
ARM	0004	4C407041	`ld1 {v1.16b}, [x2]`
ARM	0008	6E210C00	`uqadd v0.16b, v0.16b, v1.16b`
ARM	000C	4C407062	`ld1 {v2.16b}, [x3]`
ARM	0010	6E226400	`umax v0.16b, v0.16b, v2.16b`
ARM	0014	4C007020	`st1 {v0.16b}, [x1]`

Chapter 11

BX-32: Embedded Floating Point with Zve32f

11.1 The architecture

BX-32 is an RV32 hart whose data path is thirty-two bits, with the scalar extensions you need for a hosted compiler, and with Zve32f. Chapter 8 is that integer file. This is the first machine in the book that is allowed to tell a standard toolchain “vector, with single-precision float”. The discovery path is the standard one. The custom CSR used by SVM-8 and SVM-16 may remain, returning a code that means “superset of SVM-16”, but `misa` and the vector CSRs are authoritative.

ELEN = 32. VLEN is at least 128.
Legal SEW = 8, 16, and 32.
FP32 vector arithmetic, conversions, and fused multiply-add.
Indexed loads and stores.
Integer behaviour of SVM-16 remains legal.

A loop assembled for SVM-8, using only EEW=8, runs. A loop assembled for SVM-16, using EEW=16 and `vwmul`, runs. A loop that sets SEW=32 runs for the first time.

11.2 Floating point in the lane

Each lane that supports FP32 contains a single-precision adder and multiplier, or a fused unit if the FPGA’s DSP blocks map cleanly onto one. Fused multiply-add is required by the extension. Rounding follows the dynamic rounding mode of the hart.

Do not share one FP unit across all lanes by silent time-multiplexing unless the cycle count is in the profile card. A programmer who reads Zve32f will expect four lanes to retire four FP32 elements per cycle once the pipeline is full. If your first image has one FP unit and four integer lanes, say “integer lanes: 4, FP lanes: 1” on the card, and do not claim that the peak is four times the FP unit.

11.3 Where SVM-16 software feels the change

The change a programmer notices is not the wider bus. It is that SEW=32 no longer traps, and that a matrix kernel can stay in floating point instead of in widened integers. Xsvm, if it is still elaborated, must use the same SEW as the surrounding code. A four-by-four of FP32 values in `v8`–`v11` is the useful form. The integer form remains for SVM-16 compatibility.

Prefer, in new BX-32 software, the standard sequence of fused multiply-adds over Xsvm. Keep Xsvm for kernels you have measured, and for comparison with the BX-16 geometry image.

11.4 Opcode map for Zve32f (BX-32)

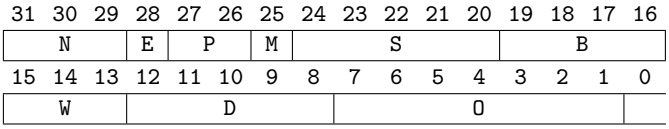
All opcodes of SVM-16 remain legal. The additional classes required by Zve32f use the same rigid two-line matrix. SEW may be 8, 16 or 32.

11.4.1 Configuration

Identical to SVM-16.

11.4.2 Unit-stride, stride and indexed load/store

Unit-stride and stride use the layouts already shown. Indexed load (`vluxei32.v` / `vloxei32.v`):



N = nf
E = mew
P = mop (01 = indexed unordered, 11 = ordered)
M = vm
S = vs2 (index vector)
B = rs1 (base)
W = width (110 = 32-bit)
D = vd
O = 0000111

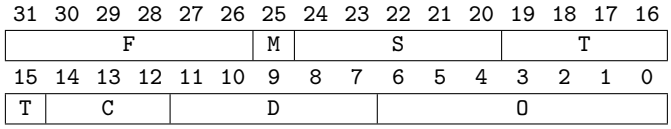
Store forms use major opcode 0100111.

11.4.3 Integer arithmetic (SEW = 8, 16, 32)

Same OP-V layout. All integer instructions of SVM-16 plus the SEW=32 variants and gather:

Instruction	notes
`vrgather.vv/vx`	register gather
`vcompress.vm`	compress under mask

11.4.4 Floating-point arithmetic (SEW = 32)



F = funct6
M = vm
S = vs2
T = vs1 / rs1 / fs1
C = funct3 (001 = OPFVV, 101 = OPFVF)
D = vd
O = 1010111

Instruction	funct6	form
`vfadd`	000000	vv / vf
`vfsub`	000010	vv / vf
`vfmul`	100100	vv / vf
`vfmacc`	101100	vv / vf
`vfnmacc`	101101	vv / vf
`vfmsac`	101110	vv / vf
`vfnmsac`	101111	vv / vf
`vfmadd`	101000	vv / vf
`vfnmadd`	101001	vv / vf
`vfmsub`	101010	vv / vf
`vfnmsub`	101011	vv / vf
`vfmin`	000100	vv / vf
`vfmax`	000110	vv / vf
`vfsqrt`	010011	v
`vfsgnj*`	001000..001010	vv / vf
`vfcvt.*`	01000x	conversions
`vmfeq..`	011000..	FP compare

11.4.5 Mask and move

Full mask logical set of SVM-16 plus `vfmv.v.f`, `vfmv.f.s`, `vfmv.s.f`.

Xsvm remains optional and is not part of the Zve32f claim.

11.5 MIPS fetch

Three MIPS softcores share one escape because none of them assigns coprocessor 3. The word is `4C000000`: opcode COP3, every other field zero. The following 32-bit word is the OP-V parcel.

Host	Machine	Endian	Left in place
R3000A	PlayStation	little	COP2, the GTE
VR4300	Nintendo 64	big	COP1, the FPU
Emotion Engine	PlayStation 2	little	COP2, which is VU0

`rs1` selects R0–R31. On the Emotion Engine the architectural GPR is 128 bits wide; this elaboration uses the low 32 bits as the address. The adaptor sits on the instruction bus and claims the COP3 word before Coprocessor Unusable is taken.

The Nintendo 64 vector unit that programmers mean is the RSP, a separate memory and a separate instruction set, not a VR4300 opcode. The PlayStation 2 vector units are VU0 and VU1, with their own 64-bit instruction words. Those units stay. The adaptor feeds the MIPS side.

SH-4 floating point, which BX-16 cannot host, uses the SH-4 escape of Chapter 10 in front of this machine. ELEN 32 is what `FTRV` was doing with a 4-by-4 of single-precision values. The escape halfword does not move into the `Fxxx` space.

Harness files are `kernels/bx32/r3000/`, `kernels/bx32/vr4300/`, and `kernels/bx32/ee/`.

11.6 Assembler kernels that must run

In addition to every kernel of Chapters 9 and 10, the following must pass.

Table 11.1: Kernel analogues on this machine.

Kernel	RVV	Intel	ARM
sew32	`vse32.v`	`movdqu`	`st1`
fma32	`vfmacc.vv`	`vfmadd231ps`	`fmla`
gemm32	`vfmacc.vf`	`vbroadcastss`	`ld1r`
index32	`vluxei32.v`	`vpgatherdd`	`ldr`
wmul32	`vfmul.vv`	`vmulps`	`fmul`
bfly32	`vfadd.vv`	`vaddps`	`fadd`
cmul32	`vfmul.vf`	`vmulps`	`fmul`

The comparison table under each kernel is one row per instruction: RVV, then Intel, then ARM. Chapter 9 says which encodings those bytes are.

11.6.1 sew32 — word fill

The element is 32 bits. The byte step is `vl` shifted left by 2.
`kernels/bx32/sew32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`

11.6.2 fma32 — fused multiply-add strip

`kernels/bx32/fma32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`

11.6.3 gemm32 — outer product of a 4-wide panel

`fma32` updates one vector. A matrix product reuses that vector against several scalars from the other matrix. One step loads four singles from the packed A panel, reads each of four packed B scalars with `flw`, and accumulates with `vfmacc.vf`. Four accumulators make a 4 by 4 tile, because VLEN 128 holds four SEW-32 elements. `K` counts those rank-1 updates. Alpha is 1. When beta is not 1, scale C before entering.

`flw` is LOAD-FP with funct3 010. It shares the opcode byte with `vle32.v`. The width field separates the scalar load from the vector load.

A and B arrive packed, so the vector load is unit stride. Packing is a copy into a contiguous panel. The loop that chooses the panel size sits outside this image.

This is the inner kernel of the GEMM note cited in the references. That note writes the update in double precision, then widens the tile to four doubles in a `ymm` and to 24 by 8 in a

Table 11.2: Kernel comparison: sew32

`a0 = n, a1 = fill word, a2 = dst`

ISA	Address	Hex	Assembly
RVV	0000	02050063	`sew32: beqz a0, end`
RVV	0004	002572D7	`loop: vsetvli t0, a0, e32, m1, tu, mu`
RVV	0008	5E05C0D7	`vmv.v.x v1, a1`
RVV	000C	020660A7	`vse32.v v1, (a2)`
RVV	0010	00229313	`slli t1, t0, 2`
RVV	0014	00660633	`add a2, a2, t1`
RVV	0018	40550533	`sub a0, a0, t0`
RVV	001C	FE0514E3	`bnez a0, loop`
RVV	0020	00008067	`end: ret`
Intel	0000	660F6EC6	`sew32: movd xmm0, esi`
Intel	0004	660F70C000	`pshufd xmm0, xmm0, 0`
Intel	0009	F30F7F07	`movdqu [rdi], xmm0`
Intel	000D	4883C710	`add rdi, 16`
Intel	0011	4883EA04	`sub rdx, 4`
Intel	0015	77E9	`ja sew32`
ARM	0000	4E040C20	`sew32: dup v0.4s, w1`
ARM	0004	4C007840	`st1 {v0.4s}, [x2]`
ARM	0008	91004042	`add x2, x2, #16`
ARM	000C	F1001000	`subs x0, x0, #4`
ARM	0010	54FFFF88	`b.hi sew32`

Table 11.3: Kernel comparison: fma32

`a0 = n, a1 = a, a2 = b, a3 = c. c = c + a*b`

ISA	Address	Hex	Assembly
RVV	0000	02050A63	`fma32: beqz a0, end`
RVV	0004	002572D7	`loop: vsetvli t0, a0, e32, m1, tu, mu`
RVV	0008	0205E087	`vle32.v v1, (a1)`
RVV	000C	02066107	`vle32.v v2, (a2)`
RVV	0010	0206E187	`vle32.v v3, (a3)`
RVV	0014	B22091D7	`vfmacc.vv v3, v1, v2`
RVV	0018	0206E1A7	`vse32.v v3, (a3)`
RVV	001C	00229313	`slli t1, t0, 2`
RVV	0020	006585B3	`add a1, a1, t1`
RVV	0024	00660633	`add a2, a2, t1`
RVV	0028	006686B3	`add a3, a3, t1`
RVV	002C	40550533	`sub a0, a0, t0`
RVV	0030	FC051AE3	`bnez a0, loop`
RVV	0034	00008067	`end: ret`
Intel	0000	C5FC1006	`fma32: vmovups ymm0, [rsi]`
Intel	0004	C5FC100A	`vmovups ymm1, [rdx]`
Intel	0008	C5FC1011	`vmovups ymm2, [rcx]`
Intel	000C	C4E27DB8D1	`vfmadd231ps ymm2, ymm0, ymm1`
Intel	0011	C5FC1111	`vmovups [rcx], ymm2`
ARM	0000	4C407820	`fma32: ld1 {v0.4s}, [x1]`
ARM	0004	4C407841	`ld1 {v1.4s}, [x2]`
ARM	0008	4C407862	`ld1 {v2.4s}, [x3]`
ARM	000C	4E21CC02	`fmla v2.4s, v0.4s, v1.4s`
ARM	0010	4C007862	`st1 {v2.4s}, [x3]`

zmm. The Intel column here is four singles in an xmm, with vbroadcastss and vfmadd231ps. The ARM column is ld1r and fmla.

Table 11.4: Kernel comparison: gemm32

a0 = K, a1 = A panel, a2 = B panel, a3 = C. 4-wide outer product.

ISA	Address	Hex	Assembly
RVV	0000	06050A63	gemm32: beqz a0, end
RVV	0004	00207057	vsetvli zero, zero, e32, m1, tu, mu
RVV	0008	0206E407	vle32.v v8, (a3)
RVV	000C	01068293	addi t0, a3, 16
RVV	0010	0202E487	vle32.v v9, (t0)
RVV	0014	02068293	addi t0, a3, 32
RVV	0018	0202E507	vle32.v v10, (t0)
RVV	001C	03068293	addi t0, a3, 48
RVV	0020	0202E587	vle32.v v11, (t0)
RVV	0024	0205E087	loop: vle32.v v1, (a1)
RVV	0028	00062007	flw ft0, 0(a2)
RVV	002C	B2105457	vfmacc.vf v8, ft0, v1
RVV	0030	00462007	flw ft0, 4(a2)
RVV	0034	B21054D7	vfmacc.vf v9, ft0, v1
RVV	0038	00862007	flw ft0, 8(a2)
RVV	003C	B2105557	vfmacc.vf v10, ft0, v1
RVV	0040	00C62007	flw ft0, 12(a2)
RVV	0044	B21055D7	vfmacc.vf v11, ft0, v1
RVV	0048	01058593	addi a1, a1, 16
RVV	004C	01060613	addi a2, a2, 16
RVV	0050	FFF50513	addi a0, a0, -1
RVV	0054	FC0518E3	bnez a0, loop
RVV	0058	0206E427	vse32.v v8, (a3)
RVV	005C	01068293	addi t0, a3, 16
RVV	0060	0202E4A7	vse32.v v9, (t0)
RVV	0064	02068293	addi t0, a3, 32
RVV	0068	0202E527	vse32.v v10, (t0)
RVV	006C	03068293	addi t0, a3, 48
RVV	0070	0202E5A7	vse32.v v11, (t0)
RVV	0074	00008067	end: ret
Intel	0000	C5F81006	gemm32: vmovups xmm0, [rsi]
Intel	0004	C4E279180A	vbroadcastss xmm1, dword ptr [rdx]
Intel	0009	C4E279B8E1	vfmadd231ps xmm4, xmm0, xmm1
Intel	000E	C4E279184A04	vbroadcastss xmm1, dword ptr [rdx+4]
Intel	0014	C4E279B8E9	vfmadd231ps xmm5, xmm0, xmm1
Intel	0019	C4E279184A08	vbroadcastss xmm1, dword ptr [rdx+8]
Intel	001F	C4E279B8F1	vfmadd231ps xmm6, xmm0, xmm1
Intel	0024	C4E279184A0C	vbroadcastss xmm1, dword ptr [rdx+12]
Intel	002A	C4E279B8F9	vfmadd231ps xmm7, xmm0, xmm1
Intel	002F	4883C610	add rsi, 16
Intel	0033	4883C210	add rdx, 16
Intel	0037	4883E901	sub rcx, 1
Intel	003B	75C3	jnz gemm32
ARM	0000	4CDF7820	gemm32: ld1 {v0.4s}, [x1], #16
ARM	0004	4DDFC841	ld1r {v1.4s}, [x2], #4
ARM	0008	4E21CC04	fmla v4.4s, v0.4s, v1.4s
ARM	000C	4DDFC841	ld1r {v1.4s}, [x2], #4
ARM	0010	4E21CC05	fmla v5.4s, v0.4s, v1.4s
ARM	0014	4DDFC841	ld1r {v1.4s}, [x2], #4
ARM	0018	4E21CC06	fmla v6.4s, v0.4s, v1.4s
ARM	001C	4DDFC841	ld1r {v1.4s}, [x2], #4
ARM	0020	4E21CC07	fmla v7.4s, v0.4s, v1.4s
ARM	0024	F1000400	subs x0, x0, #1
ARM	0028	54FFFEC1	b.ne gemm32

kernels/bx32/gemm32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin

11.6.4 index32 — indexed load (gather)

kernels/bx32/index32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin

Table 11.5: Kernel comparison: index32

a0 = n, a1 = base, a2 = index vector, a3 = dst

ISA	Address	Hex	Assembly
RVV	0000	02050463	index32: beqz a0, end
RVV	0004	002572D7	loop: vsetvli t0, a0, e32, m1, tu, mu
RVV	0008	02066107	vle32.v v2, (a2)
RVV	000C	0625E087	vluxei32.v v1, (a1), v2
RVV	0010	0206E0A7	vse32.v v1, (a3)
RVV	0014	00229313	slli t1, t0, 2
RVV	0018	00660633	add a2, a2, t1
RVV	001C	006686B3	add a3, a3, t1
RVV	0020	40550533	sub a0, a0, t0
RVV	0024	FE0510E3	bnez a0, loop
RVV	0028	00008067	end: ret
Intel	0000	C4E26D90040E	index32: vpgatherdd ymm0, [rsi + ymm1], ymm2
ARM	0000	BC637820	index32: ldr s0, [x1, x3, lsl #2]
ARM	0004	BD000040	str s0, [x2]
ARM	0008	91001063	add x3, x3, #4
ARM	000C	91001042	add x2, x2, #4
ARM	0010	F1000400	subs x0, x0, #1
ARM	0014	54FFFF61	b.ne index32

11.6.5 wmul32 — element-wise window

A vector times a second vector. AAC inside an MP4 or M4A file, Opus, Vorbis, AC-3, and MP3 apply this window before the transform. A raised-cosine pulse shaper in GNU Radio and liquid-dsp is the same product. A spectrometer calls it the apodization window.

Table 11.6: Kernel comparison: wmul32

a0 = n, a1 = x, a2 = window, a3 = dst. Element-wise product.

ISA	Address	Hex	Assembly
RVV	0000	02050863	wmul32: beqz a0, end
RVV	0004	002572D7	loop: vsetvli t0, a0, e32, m1, tu, mu
RVV	0008	0205E087	vle32.v v1, (a1)
RVV	000C	02066107	vle32.v v2, (a2)
RVV	0010	921110D7	vfmul.vv v1, v1, v2
RVV	0014	0206E0A7	vse32.v v1, (a3)
RVV	0018	00229313	slli t1, t0, 2
RVV	001C	006585B3	add a1, a1, t1
RVV	0020	00660633	add a2, a2, t1
RVV	0024	006686B3	add a3, a3, t1
RVV	0028	40550533	sub a0, a0, t0
RVV	002C	FC051CE3	bnez a0, loop
RVV	0030	00008067	end: ret
Intel	0000	C5F81006	wmul32: vmovups xmm0, [rsi]
Intel	0004	C5F8100A	vmovups xmm1, [rdx]
Intel	0008	C5F859C1	vmulps xmm0, xmm0, xmm1
Intel	000C	C5F81107	vmovups [rdi], xmm0
ARM	0000	4C407820	wmul32: ld1 {v0.4s}, [x1]
ARM	0004	4C407841	ld1 {v1.4s}, [x2]
ARM	0008	6E21DC00	fmul v0.4s, v0.4s, v1.4s
ARM	000C	4C007860	st1 {v0.4s}, [x3]

`kernels/bx32/wmul32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`

11.6.6 bfly32 — radix-2 butterfly

One real butterfly with a scalar twiddle, in place: $a + wb$ and $a - wb$. The multiply and the add are separate instructions. `fma32` is the fused product. FFTW, PocketFFT, KissFFT, vDSP, and muFFT are this step. A complex FFT runs the body on the real array and again on the

imaginary array. AAC, Opus, and Vorbis build the MDCT on that FFT. The double-precision form is the same parcels with `e64`, which is the width FFTW runs by default.

Table 11.7: Kernel comparison: bfly32

`a0 = n, a1 = a, a2 = b, a3 = twiddle.  Real radix-2 butterfly, in place.`

ISA	Address	Hex	Assembly
RVV	0000	0006A007	`bfly32:  flw ft0, 0(a3)`
RVV	0004	02050C63	`beqz a0, end`
RVV	0008	002572D7	`loop:  vsetvli t0, a0, e32, m1, tu, mu`
RVV	000C	0205E087	`vle32.v v1, (a1)`
RVV	0010	02066107	`vle32.v v2, (a2)`
RVV	0014	922051D7	`vfmul.vf v3, v2, ft0`
RVV	0018	02119257	`vfadd.vv v4, v1, v3`
RVV	001C	0A1192D7	`vfsub.vv v5, v1, v3`
RVV	0020	0205E227	`vse32.v v4, (a1)`
RVV	0024	020662A7	`vse32.v v5, (a2)`
RVV	0028	00229313	`slli t1, t0, 2`
RVV	002C	006585B3	`add a1, a1, t1`
RVV	0030	00660633	`add a2, a2, t1`
RVV	0034	40550533	`sub a0, a0, t0`
RVV	0038	FC0518E3	`bnez a0, loop`
RVV	003C	00008067	`end:  ret`
Intel	0000	C5F81007	`bfly32:  vmovups xmm0, [rdi]`
Intel	0004	C5F8100E	`vmovups xmm1, [rsi]`
Intel	0008	C4E2791812	`vbroadcastss xmm2, dword ptr [rdx]`
Intel	000D	C5F059CA	`vmulps xmm1, xmm1, xmm2`
Intel	0011	C5F858D9	`vaddps xmm3, xmm0, xmm1`
Intel	0015	C5F85CC1	`vsubps xmm0, xmm0, xmm1`
Intel	0019	C5F8111F	`vmovups [rdi], xmm3`
Intel	001D	C5F81106	`vmovups [rsi], xmm0`
ARM	0000	4C407820	`bfly32:  ld1 {v0.4s}, [x1]`
ARM	0004	4C407841	`ld1 {v1.4s}, [x2]`
ARM	0008	4D40C862	`ld1r {v2.4s}, [x3]`
ARM	000C	6E22DC21	`fmul v1.4s, v1.4s, v2.4s`
ARM	0010	4E21D403	`fadd v3.4s, v0.4s, v1.4s`
ARM	0014	4EA1D400	`fsub v0.4s, v0.4s, v1.4s`
ARM	0018	4C007823	`st1 {v3.4s}, [x1]`
ARM	001C	4C007840	`st1 {v0.4s}, [x2]`

`kernels/bx32/bfly32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`

11.6.7 cmul32 — complex multiply by one scalar

Split real and imaginary arrays times one scalar complex (w_r, w_i). The real part is $a_r w_r - a_i w_i$. The imaginary part is $a_r w_i + a_i w_r$. A software mixer and a shared local oscillator use one scalar against the vector: GNU Radio VOLK and liquid-dsp. A per-lane twiddle is `wmul32` on each plane, then this add and this subtract. A complex product `cgemm` is `gemm32` on each plane plus this cross term.

`kernels/bx32/cmul32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`

11.7 Matrices

A matrix is memory. Storage is column-major. Element (i, j) is at base $+ i + j \times$ lda, which is the address the GEMM note writes as `A(i,j) = A[(i)+(j)*LDA]`. There is no matrix register. LMUL groups vector registers. A group is still a vector.

The acceptance microkernels do not take `lda`. The caller packs a panel, and the vector load is unit stride. Packing is a copy, or a stride. Alpha is 1 in these images. When beta is not 1, scale C before the kernel.

Table 11.8: Kernel comparison: cmul32

a0 = n, a1 = re, a2 = im, a3 = (wr, wi). Split complex times one scalar.

ISA	Address	Hex	Assembly
RVV	0000	0006A007	cmul32: flw ft0, 0(a3)
RVV	0004	0046A087	flw ft1, 4(a3)
RVV	0008	04050263	beqz a0, end
RVV	000C	002572D7	loop: vsetvli t0, a0, e32, m1, tu, mu
RVV	0010	0205E087	vle32.v v1, (a1)
RVV	0014	02066107	vle32.v v2, (a2)
RVV	0018	921051D7	vfmul.vf v3, v1, ft0
RVV	001C	9220D257	vfmul.vf v4, v2, ft1
RVV	0020	0A3212D7	vfsub.vv v5, v3, v4
RVV	0024	9210D1D7	vfmul.vf v3, v1, ft1
RVV	0028	92205257	vfmul.vf v4, v2, ft0
RVV	002C	02321357	vfadd.vv v6, v3, v4
RVV	0030	0205E2A7	vse32.v v5, (a1)
RVV	0034	02066327	vse32.v v6, (a2)
RVV	0038	00229313	slli t1, t0, 2
RVV	003C	006585B3	add a1, a1, t1
RVV	0040	00660633	add a2, a2, t1
RVV	0044	40550533	sub a0, a0, t0
RVV	0048	FC0512E3	bnez a0, loop
RVV	004C	00008067	end: ret
Intel	0000	C5F81007	cmul32: vmovups xmm0, [rdi]
Intel	0004	C5F8100E	vmovups xmm1, [rsi]
Intel	0008	C4E2791812	vbroadcastss xmm2, dword ptr [rdx]
Intel	000D	C4E279185A04	vbroadcastss xmm3, dword ptr [rdx+4]
Intel	0013	C5F859E2	vmulps xmm4, xmm0, xmm2
Intel	0017	C5F059EB	vmulps xmm5, xmm1, xmm3
Intel	001B	C5D85CE5	vsubps xmm4, xmm4, xmm5
Intel	001F	C5F859EB	vmulps xmm5, xmm0, xmm3
Intel	0023	C5F059C2	vmulps xmm0, xmm1, xmm2
Intel	0027	C5D058E8	vaddps xmm5, xmm5, xmm0
Intel	002B	C5F81127	vmovups [rdi], xmm4
Intel	002F	C5F8112E	vmovups [rsi], xmm5
ARM	0000	4C407820	cmul32: ld1 {v0.4s}, [x1]
ARM	0004	4C407841	ld1 {v1.4s}, [x2]
ARM	0008	4D40C862	ld1r {v2.4s}, [x3]
ARM	000C	BD400463	ldr s3, [x3, #4]
ARM	0010	4E040463	dup v3.4s, v3.s[0]
ARM	0014	6E22DC04	fmul v4.4s, v0.4s, v2.4s
ARM	0018	6E23DC25	fmul v5.4s, v1.4s, v3.4s
ARM	001C	4EA5D484	fsub v4.4s, v4.4s, v5.4s
ARM	0020	6E23DC05	fmul v5.4s, v0.4s, v3.4s
ARM	0024	6E22DC26	fmul v6.4s, v1.4s, v2.4s
ARM	0028	4E26D4A5	fadd v5.4s, v5.4s, v6.4s
ARM	002C	4C007824	st1 {v4.4s}, [x1]
ARM	0030	4C007845	st1 {v5.4s}, [x2]

Complex BLAS is two real arrays. There is no complex opcode. Triangular, symmetric, packed, and banded routines use the same arithmetic body. The scalar address walk is what changes. A wider VLEN holds a taller column slice of that memory.

11.8 BLAS names

Each standard name below is one of the bodies in this chapter, or the same body with the address rule in the third column. The `s` forms are SEW 32. Chapter 13 repeats the bodies at SEW 64 for the `d` forms. `gemm32` is the product. The bytes are the comparison tables.

Table 11.9: BLAS names on this machine.

Name	Body	Address rule
sscal	scal32	one vector, one scalar
scopy	copy32	two unit-stride vectors
saxpy	axpy32	
sdot	dot32	
sdsdot, dsdot	dot32	convert, then the reduction
snrm2	nrm232	square, reduce, square root
sasum	asum32	absolute value, then a sum
sswap	swap32	
srot	rot32	
srotm	rot32	the flag selects c and s
srotg, srotmg	scalar	no vector body
isamax	iamax32	one strip, $n \leq$ VLMAX
isamin	iamax32	a min reduction
sgemv	gemv32	rows = VLMAX, lda = VLMAX
sgbmv, ssymv, ssbmv, sspmv, strmv	gemv32	a different walk
strsv, stbsv, stpsv	dot32	the remaining row, then one scalar store
sger	ger32	
ssyr, sspr, ssyr2, sspr2	ger32	on and above the diagonal
sgemm	gemm32	a packed panel
ssymm, ssyrk, ssyr2k, strmm	gemm32	the same product
strsm	trsv	one panel of the triangular solve
c*, z*	the real body	two real arrays

Softmax is a max, a subtract, an exponential, a sum, and a divide. The exponential is a scalar sequence, or a later extension. ReLU is a max against zero. An embedding is `index32`. A convolution is a strided copy of patches, then `gemm32`. Attention is gemm, softmax, gemm.

`cblas_dgemv` in the GEMM note is the matrix-vector row. `cblas_dgemm` is the product. The packing routines in that note are the copy that makes A and B unit stride.

`kernels/bx32/scal32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`
`kernels/bx32/copy32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`
`kernels/bx32/axpy32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`
`kernels/bx32/dot32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`
`kernels/bx32/nrm232.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`
`kernels/bx32/asum32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`
`kernels/bx32/swap32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`
`kernels/bx32/rot32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`
`kernels/bx32/gemv32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`
`kernels/bx32/ger32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`
`kernels/bx32/iamax32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin`

Table 11.10: Programs that run the BX-32 kernels.

Kernel	Work	Where it ships
sew32	word fill	A zeroed float buffer. A cleared embedding row.
fma32	element $c + ab$	One step of Horner's rule. A biquad across channels. vDSP `vDSP_vma`. A small update in Eigen.
gemm32	4-wide rank-1	`sgemm` in OpenBLAS, BLIS, and Eigen. The GEMM note. im2col plus this product in Caffe and oneDNN.
index32	gather	CSR sparse matrix-vector in PETSc. An embedding row. A wavetable read by phase. A stencil that has gathered its neighbours.
scal32	scale	Volume. LAPACK scaling. The normaliser after `snrm2`.
copy32	copy	`scopy`. Packing a GEMM panel.
axpy32	scaled add	CG and GMRES. A mix of float audio stems. OpenBLAS `saxpy`.
dot32	inner product	`sdot`. A FIR sum: CMSIS `arm_dot_prod_f32`, GNU Radio `fir_filter`, FFmpeg `afir`. Cosine similarity.
nrm232	Euclidean length	`snrm2`. RMS. The vector length in a least-squares solver.
asum32	L1 norm	`sasum`. Mean absolute amplitude.
swap32	exchange	`sswap`. A LAPACK pivot exchange.
rot32	Givens rotation	`srot`. QR and Jacobi in LAPACK.
gemv32	matrix-vector	`sgemv`. A fully connected layer of height VLMAX. One row of a DCT against a cosine matrix.
ger32	rank-1	`sger`. A covariance update.
iamax32	index of max abs	`isamax`. A LAPACK pivot. A peak picker. One strip.
wmul32	window	MDCT window for AAC, Opus, Vorbis, AC-3, MP3. Pulse shaping. Apodization.
bfly32	radix-2 FFT step	FFTW, PocketFFT, KissFFT, vDSP. The FFT under an IMDCT.
cmul32	complex mixer	GNU Radio VOLK. liquid-dsp NCO. One local oscillator against a vector of IQ.

Table 11.11: Kernel comparison: scal32

`a0 = n, a1 = x, a2 = alpha.  x *= alpha`

ISA	Address	Hex	Assembly
RVV	0000	00062007	`scal32:  flw ft0, 0(a2)`
RVV	0004	02050263	`beqz a0, end`
RVV	0008	002572D7	`loop:  vsetvli t0, a0, e32, m1, tu, mu`
RVV	000C	0205E087	`vle32.v v1, (a1)`
RVV	0010	921050D7	`vfmul.vf v1, v1, ft0`
RVV	0014	0205E0A7	`vse32.v v1, (a1)`
RVV	0018	00229313	`slli t1, t0, 2`
RVV	001C	006585B3	`add a1, a1, t1`
RVV	0020	40550533	`sub a0, a0, t0`
RVV	0024	FE0512E3	`bnez a0, loop`
RVV	0028	00008067	`end:  ret`
Intel	0000	C4E279180E	`scal32:  vbroadcastss xmm1, dword ptr [rsi]`
Intel	0005	C5F81007	`vmovups xmm0, [rdi]`
Intel	0009	C5F859C1	`vmulps xmm0, xmm0, xmm1`
Intel	000D	C5F81107	`vmovups [rdi], xmm0`
Intel	0011	4883C710	`add rdi, 16`
Intel	0015	4883EA04	`sub rdx, 4`
Intel	0019	75E5	`jnz scal32`
ARM	0000	4D40C841	`scal32:  ld1r {v1.4s}, [x2]`
ARM	0004	4C407820	`ld1 {v0.4s}, [x1]`
ARM	0008	6E21DC00	`fmul v0.4s, v0.4s, v1.4s`
ARM	000C	4C007820	`st1 {v0.4s}, [x1]`
ARM	0010	91004021	`add x1, x1, #16`
ARM	0014	F1001000	`subs x0, x0, #4`
ARM	0018	54FFFF41	`b.ne scal32`

Table 11.12: Kernel comparison: copy32

a0 = n, a1 = src, a2 = dst

ISA	Address	Hex	Assembly
RVV	0000	02050263	copy32: beqz a0, end
RVV	0004	002572D7	loop: vsetvli t0, a0, e32, m1, tu, mu
RVV	0008	0205E087	vle32.v v1, (a1)
RVV	000C	020660A7	vse32.v v1, (a2)
RVV	0010	00229313	slli t1, t0, 2
RVV	0014	006585B3	add a1, a1, t1
RVV	0018	00660633	add a2, a2, t1
RVV	001C	40550533	sub a0, a0, t0
RVV	0020	FE0512E3	bnez a0, loop
RVV	0024	00008067	end: ret
Intel	0000	C5F81006	copy32: vmovups xmm0, [rsi]
Intel	0004	C5F81107	vmovups [rdi], xmm0
Intel	0008	4883C610	add rsi, 16
Intel	000C	4883C710	add rdi, 16
Intel	0010	4883EA04	sub rdx, 4
Intel	0014	75EA	jnz copy32
ARM	0000	4C407820	copy32: ld1 {v0.4s}, [x1]
ARM	0004	4C007840	st1 {v0.4s}, [x2]
ARM	0008	91004021	add x1, x1, #16
ARM	000C	91004042	add x2, x2, #16
ARM	0010	F1001000	subs x0, x0, #4
ARM	0014	54FFFF61	b.ne copy32

Table 11.13: Kernel comparison: axpy32

a0 = n, a1 = x, a2 = y, a3 = alpha. y += alpha*x

ISA	Address	Hex	Assembly
RVV	0000	0006A007	axpy32: flw ft0, 0(a3)
RVV	0004	02050663	beqz a0, end
RVV	0008	002572D7	loop: vsetvli t0, a0, e32, m1, tu, mu
RVV	000C	0205E087	vle32.v v1, (a1)
RVV	0010	02066107	vle32.v v2, (a2)
RVV	0014	B2105157	vfmacc.vf v2, ft0, v1
RVV	0018	02066127	vse32.v v2, (a2)
RVV	001C	00229313	slli t1, t0, 2
RVV	0020	006585B3	add a1, a1, t1
RVV	0024	00660633	add a2, a2, t1
RVV	0028	40550533	sub a0, a0, t0
RVV	002C	FC051EE3	bnez a0, loop
RVV	0030	00008067	end: ret
Intel	0000	C4E2791811	axpy32: vbroadcastss xmm2, dword ptr [rcx]
Intel	0005	C5F81006	vmovups xmm0, [rsi]
Intel	0009	C5F8100F	vmovups xmm1, [rdi]
Intel	000D	C4E279B8CA	vfmadd231ps xmm1, xmm0, xmm2
Intel	0012	C5F8110F	vmovups [rdi], xmm1
Intel	0016	4883C610	add rsi, 16
Intel	001A	4883C710	add rdi, 16
Intel	001E	4883EA04	sub rdx, 4
Intel	0022	75DC	jnz axpy32
ARM	0000	4D40C862	axpy32: ld1r {v2.4s}, [x3]
ARM	0004	4C407820	ld1 {v0.4s}, [x1]
ARM	0008	4C407841	ld1 {v1.4s}, [x2]
ARM	000C	4E22CC01	fmla v1.4s, v0.4s, v2.4s
ARM	0010	4C007841	st1 {v1.4s}, [x2]
ARM	0014	91004021	add x1, x1, #16
ARM	0018	91004042	add x2, x2, #16
ARM	001C	F1001000	subs x0, x0, #4
ARM	0020	54FFFF01	b.ne axpy32

Table 11.14: Kernel comparison: dot32

a0 = n, a1 = x, a2 = y, a3 = sum out

ISA	Address	Hex	Assembly
RVV	0000	5E003257	dot32: vmv.v.i v4, 0
RVV	0004	02050663	beqz a0, end
RVV	0008	002572D7	loop: vsetvli t0, a0, e32, m1, tu, mu
RVV	000C	0205E087	vle32.v v1, (a1)
RVV	0010	02066107	vle32.v v2, (a2)
RVV	0014	921110D7	vfmul.vv v1, v1, v2
RVV	0018	06121257	vfredusum.vsv4, v1, v4
RVV	001C	00229313	slli t1, t0, 2
RVV	0020	006585B3	add a1, a1, t1
RVV	0024	00660633	add a2, a2, t1
RVV	0028	40550533	sub a0, a0, t0
RVV	002C	FC051EE3	bnez a0, loop
RVV	0030	42401057	end: vfmv.f.s ft0, v4
RVV	0034	0006A027	fsw ft0, 0(a3)
RVV	0038	00008067	ret
Intel	0000	C5F81006	dot32: vmovups xmm0, [rsi]
Intel	0004	C5F8100A	vmovups xmm1, [rdx]
Intel	0008	C5F859C1	vmulps xmm0, xmm0, xmm1
Intel	000C	F20F7CC0	haddps xmm0, xmm0
Intel	0010	F20F7CC0	haddps xmm0, xmm0
ARM	0000	4C407820	dot32: ld1 {v0.4s}, [x1]
ARM	0004	4C407841	ld1 {v1.4s}, [x2]
ARM	0008	6E21DC00	fmul v0.4s, v0.4s, v1.4s
ARM	000C	6E20D400	faddp v0.4s, v0.4s, v0.4s
ARM	0010	6E20D400	faddp v0.4s, v0.4s, v0.4s

Table 11.15: Kernel comparison: nrm232

a0 = n, a1 = x, a2 = norm out. sqrt of sum of squares

ISA	Address	Hex	Assembly
RVV	0000	5E003257	nrm232: vmv.v.i v4, 0
RVV	0004	02050263	beqz a0, end
RVV	0008	002572D7	loop: vsetvli t0, a0, e32, m1, tu, mu
RVV	000C	0205E087	vle32.v v1, (a1)
RVV	0010	921090D7	vfmul.vv v1, v1, v1
RVV	0014	06121257	vfredusum.vsv4, v1, v4
RVV	0018	00229313	slli t1, t0, 2
RVV	001C	006585B3	add a1, a1, t1
RVV	0020	40550533	sub a0, a0, t0
RVV	0024	FE0512E3	bnez a0, loop
RVV	0028	42401057	end: vfmv.f.s ft0, v4
RVV	002C	58007053	fsqrt.s ft0, ft0
RVV	0030	00062027	fsw ft0, 0(a2)
RVV	0034	00008067	ret
Intel	0000	C5F81006	nrm232: vmovups xmm0, [rsi]
Intel	0004	C5F859C0	vmulps xmm0, xmm0, xmm0
Intel	0008	F20F7CC0	haddps xmm0, xmm0
Intel	000C	F20F7CC0	haddps xmm0, xmm0
Intel	0010	F30F51C0	sqrtss xmm0, xmm0
ARM	0000	4C407820	nrm232: ld1 {v0.4s}, [x1]
ARM	0004	6E20DC00	fmul v0.4s, v0.4s, v0.4s
ARM	0008	6E20D400	faddp v0.4s, v0.4s, v0.4s
ARM	000C	6E20D400	faddp v0.4s, v0.4s, v0.4s
ARM	0010	1E21C000	fsqrt s0, s0

Table 11.16: Kernel comparison: asum32

a0 = n, a1 = x, a2 = sum of absolute values

ISA	Address	Hex	Assembly
RVV	0000	5E003257	asum32: vmv.v.i v4, 0
RVV	0004	02050263	beqz a0, end
RVV	0008	002572D7	loop: vsetvli t0, a0, e32, m1, tu, mu
RVV	000C	0205E087	vle32.v v1, (a1)
RVV	0010	2A1090D7	vfsgnjx.vv v1, v1, v1
RVV	0014	06121257	vfredusum.vsv4, v1, v4
RVV	0018	00229313	slli t1, t0, 2
RVV	001C	006585B3	add a1, a1, t1
RVV	0020	40550533	sub a0, a0, t0
RVV	0024	FE0512E3	bnez a0, loop
RVV	0028	42401057	end: vfmv.f.s ft0, v4
RVV	002C	00062027	fsw ft0, 0(a2)
RVV	0030	00008067	ret
Intel	0000	C5F81006	asum32: vmovups xmm0, [rsi]
Intel	0004	C5F854C7	vandps xmm0, xmm0, xmm7
Intel	0008	F20F7CC0	haddps xmm0, xmm0
Intel	000C	F20F7CC0	haddps xmm0, xmm0
ARM	0000	4C407820	asum32: ld1 {v0.4s}, [x1]
ARM	0004	4EA0F800	fabs v0.4s, v0.4s
ARM	0008	6E20D400	faddp v0.4s, v0.4s, v0.4s
ARM	000C	6E20D400	faddp v0.4s, v0.4s, v0.4s

Table 11.17: Kernel comparison: swap32

a0 = n, a1 = x, a2 = y

ISA	Address	Hex	Assembly
RVV	0000	02050663	swap32: beqz a0, end
RVV	0004	002572D7	loop: vsetvli t0, a0, e32, m1, tu, mu
RVV	0008	0205E087	vle32.v v1, (a1)
RVV	000C	02066107	vle32.v v2, (a2)
RVV	0010	0205E127	vse32.v v2, (a1)
RVV	0014	020660A7	vse32.v v1, (a2)
RVV	0018	00229313	slli t1, t0, 2
RVV	001C	006585B3	add a1, a1, t1
RVV	0020	00660633	add a2, a2, t1
RVV	0024	40550533	sub a0, a0, t0
RVV	0028	FC051EE3	bnez a0, loop
RVV	002C	00008067	end: ret
Intel	0000	C5F81006	swap32: vmovups xmm0, [rsi]
Intel	0004	C5F8100F	vmovups xmm1, [rdi]
Intel	0008	C5F8110E	vmovups [rsi], xmm1
Intel	000C	C5F81107	vmovups [rdi], xmm0
Intel	0010	4883C610	add rsi, 16
Intel	0014	4883C710	add rdi, 16
Intel	0018	4883EA04	sub rdx, 4
Intel	001C	75E2	jnz swap32
ARM	0000	4C407820	swap32: ld1 {v0.4s}, [x1]
ARM	0004	4C407841	ld1 {v1.4s}, [x2]
ARM	0008	4C007821	st1 {v1.4s}, [x1]
ARM	000C	4C007840	st1 {v0.4s}, [x2]
ARM	0010	91004021	add x1, x1, #16
ARM	0014	91004042	add x2, x2, #16
ARM	0018	F1001000	subs x0, x0, #4
ARM	001C	54FFFF21	b.ne swap32

Table 11.18: Kernel comparison: rot32

`a0 = n, a1 = x, a2 = y, a3 = (c, s)`

ISA	Address	Hex	Assembly
RVV	0000	0006A007	`rot32:  flw ft0, 0(a3)`
RVV	0004	0046A087	`flw ft1, 4(a3)`
RVV	0008	04050263	`beqz a0, end`
RVV	000C	002572D7	`loop:  vsetvli t0, a0, e32, m1, tu, mu`
RVV	0010	0205E087	`vle32.v v1, (a1)`
RVV	0014	02066107	`vle32.v v2, (a2)`
RVV	0018	921051D7	`vfmul.vf v3, v1, ft0`
RVV	001C	9220D257	`vfmul.vf v4, v2, ft1`
RVV	0020	023212D7	`vfadd.vv v5, v3, v4`
RVV	0024	922051D7	`vfmul.vf v3, v2, ft0`
RVV	0028	9210D257	`vfmul.vf v4, v1, ft1`
RVV	002C	0A321357	`vfsub.vv v6, v3, v4`
RVV	0030	0205E2A7	`vse32.v v5, (a1)`
RVV	0034	02066327	`vse32.v v6, (a2)`
RVV	0038	00229313	`slli t1, t0, 2`
RVV	003C	006585B3	`add a1, a1, t1`
RVV	0040	00660633	`add a2, a2, t1`
RVV	0044	40550533	`sub a0, a0, t0`
RVV	0048	FC0512E3	`bnez a0, loop`
RVV	004C	00008067	`end:  ret`
Intel	0000	C4E2791811	`rot32:  vbroadcastss xmm2, dword ptr [rcx]`
Intel	0005	C4E279185904	`vbroadcastss xmm3, dword ptr [rcx+4]`
Intel	000B	C5F81006	`vmovups xmm0, [rsi]`
Intel	000F	C5F8100F	`vmovups xmm1, [rdi]`
Intel	0013	C5F859E2	`vmulps xmm4, xmm0, xmm2`
Intel	0017	C4E271B8E3	`vfmadd231ps xmm4, xmm1, xmm3`
Intel	001C	C5F059EA	`vmulps xmm5, xmm1, xmm2`
Intel	0020	C4E279BCEB	`vfnmadd231ps xmm5, xmm0, xmm3`
Intel	0025	C5F81126	`vmovups [rsi], xmm4`
Intel	0029	C5F8112F	`vmovups [rdi], xmm5`
ARM	0000	4D40C862	`rot32:  ld1r {v2.4s}, [x3]`
ARM	0004	91001063	`add x3, x3, #4`
ARM	0008	4D40C863	`ld1r {v3.4s}, [x3]`
ARM	000C	4C407820	`ld1 {v0.4s}, [x1]`
ARM	0010	4C407841	`ld1 {v1.4s}, [x2]`
ARM	0014	6E22DC04	`fmul v4.4s, v0.4s, v2.4s`
ARM	0018	4E23CC24	`fmla v4.4s, v1.4s, v3.4s`
ARM	001C	6E22DC25	`fmul v5.4s, v1.4s, v2.4s`
ARM	0020	4EA3CC05	`fmls v5.4s, v0.4s, v3.4s`
ARM	0024	4C007824	`st1 {v4.4s}, [x1]`
ARM	0028	4C007845	`st1 {v5.4s}, [x2]`

11.9 Harts

A second RV32 hart is optional. It shares memory through an arbiter you already know how to write, because BX-16 has several masters. The second hart is for a concurrent system, not a different instruction set. Both harts, if both are present, implement Zve32f or the second is a scalar only and the profile card says so. Do not build one hart with FP32 vector and another with a private opcode map.

Local scratchpads are ordinary memories in the address map, marked cacheable or not according to your platform. They are not a third address space with a private load instruction. Vector loads to them use the same unit-stride encoding as vector loads to the rest of memory.

Table 11.19: Kernel comparison: gemv32

`a0 = columns, a1 = A, a2 = x, a3 = y.  rows = VLMAX, lda = VLMAX`

ISA	Address	Hex	Assembly
RVV	0000	02050863	`gemv32:  beqz a0, end`
RVV	0004	002072D7	`vsetvli t0, zero, e32, m1, tu, mu`
RVV	0008	00229313	`slli t1, t0, 2`
RVV	000C	0206E407	`vle32.v v8, (a3)`
RVV	0010	0205E087	`loop:  vle32.v v1, (a1)`
RVV	0014	00062007	`flw ft0, 0(a2)`
RVV	0018	B2105457	`vfmacc.vf v8, ft0, v1`
RVV	001C	006585B3	`add a1, a1, t1`
RVV	0020	00460613	`addi a2, a2, 4`
RVV	0024	FFF50513	`addi a0, a0, -1`
RVV	0028	FE0514E3	`bnez a0, loop`
RVV	002C	0206E427	`vse32.v v8, (a3)`
RVV	0030	00008067	`end:  ret`
Intel	0000	C5F81006	`gemv32:  vmovups xmm0, [rsi]`
Intel	0004	C4E279180A	`vbroadcastss xmm1, dword ptr [rdx]`
Intel	0009	C4E279B8E1	`vfmadd231ps xmm4, xmm0, xmm1`
Intel	000E	4883C610	`add rsi, 16`
Intel	0012	4883C204	`add rdx, 4`
Intel	0016	4883E901	`sub rcx, 1`
Intel	001A	75E4	`jnz gemv32`
ARM	0000	4CDF7820	`gemv32:  ld1 {v0.4s}, [x1], #16`
ARM	0004	4DDFC841	`ld1r {v1.4s}, [x2], #4`
ARM	0008	4E21CC04	`fmla v4.4s, v0.4s, v1.4s`
ARM	000C	F1000400	`subs x0, x0, #1`
ARM	0010	54FFFF81	`b.ne gemv32`

Table 11.20: Kernel comparison: ger32

`a0 = columns, a1 = x, a2 = y, a3 = A. A += x y^T, lda = VLMAX`

ISA	Address	Hex	Assembly
RVV	0000	02050863	`ger32:  beqz a0, end`
RVV	0004	002072D7	`vsetvli t0, zero, e32, m1, tu, mu`
RVV	0008	00229313	`slli t1, t0, 2`
RVV	000C	0205E087	`vle32.v v1, (a1)`
RVV	0010	0206E407	`loop:  vle32.v v8, (a3)`
RVV	0014	00062007	`flw ft0, 0(a2)`
RVV	0018	B2105457	`vfmacc.vf v8, ft0, v1`
RVV	001C	0206E427	`vse32.v v8, (a3)`
RVV	0020	006686B3	`add a3, a3, t1`
RVV	0024	00460613	`addi a2, a2, 4`
RVV	0028	FFF50513	`addi a0, a0, -1`
RVV	002C	FE0512E3	`bnez a0, loop`
RVV	0030	00008067	`end:  ret`
Intel	0000	C5F8100E	`ger32:  vmovups xmm1, [rsi]`
Intel	0004	C5F81001	`vmovups xmm0, [rcx]`
Intel	0008	C4E2791812	`vbroadcastss xmm2, dword ptr [rdx]`
Intel	000D	C4E271B8C2	`vfmadd231ps xmm0, xmm1, xmm2`
Intel	0012	C5F81101	`vmovups [rcx], xmm0`
Intel	0016	4883C110	`add rcx, 16`
Intel	001A	4883C204	`add rdx, 4`
Intel	001E	4883E801	`sub rax, 1`
Intel	0022	75DC	`jnz ger32`
ARM	0000	4C407821	`ger32:  ld1 {v1.4s}, [x1]`
ARM	0004	4C407860	`ld1 {v0.4s}, [x3]`
ARM	0008	4DDFC842	`ld1r {v2.4s}, [x2], #4`
ARM	000C	4E22CC20	`fmla v0.4s, v1.4s, v2.4s`
ARM	0010	4C9F7860	`st1 {v0.4s}, [x3], #16`
ARM	0014	F1000400	`subs x0, x0, #1`
ARM	0018	54FFFF41	`b.ne ger32`

Table 11.21: Kernel comparison: iamax32

```
a0 = n (<= VLMAX), a1 = x, a2 = index out.  First index of max |x|
```

ISA	Address	Hex	Assembly
RVV	0000	02050263	iamax32: beqz a0, end
RVV	0004	002572D7	vsetvli t0, a0, e32, m1, tu, mu
RVV	0008	0205E087	vle32.v v1, (a1)
RVV	000C	2A1090D7	vfsgnjx.vv v1, v1, v1
RVV	0010	1E109157	vfredmax.vs v2, v1, v1
RVV	0014	42201057	vfmv.f.s ft0, v2
RVV	0018	621051D7	vmfeq.vf v3, v1, ft0
RVV	001C	4238A2D7	vfirst.m t0, v3
RVV	0020	00562023	sw t0, 0(a2)
RVV	0024	00008067	end: ret
Intel	0000	C5F81007	iamax32: vmovups xmm0, [rdi]
Intel	0004	C5F854C7	vandps xmm0, xmm0, xmm7
Intel	0008	0F12C8	movhlps xmm1, xmm0
Intel	000B	0F5FC1	maxps xmm0, xmm1
Intel	000E	660F70C801	pshufd xmm1, xmm0, 1
Intel	0013	F30F5FC1	maxss xmm0, xmm1
ARM	0000	4C407820	iamax32: ld1 {v0.4s}, [x1]
ARM	0004	4EA0F800	fabs v0.4s, v0.4s
ARM	0008	6E30F801	fmaxv s1, v0.4s
ARM	000C	4E040421	dup v1.4s, v1.s[0]
ARM	0010	6EA18C02	cmeq v2.4s, v0.4s, v1.4s

Chapter 12

RV64: The 64-bit Hart

BX-64, BX-128, and BX-512 share this integer file. XLEN is 64. VLEN is the length the next three chapters recommend or pin. Lane counts, and the same vector files beside an RV128 hart, are Chapter 7. Chapter 16 widens this integer file to 128 bits. The vector register stays at the length BX-128 or BX-512 already fixed.

12.1 The integer file

Thirty-two general-purpose registers, each 64 bits. The file is 256 bytes. The parcel stays 32 bits.

The RV64 `*W` operations read the low 32 bits and sign-extend bit 31 through bit 63. `addiw` is that add. The full-width add is `addi`. Chapter 16 keeps `*W` and continues the sign-extension through bit 127.

Chapters 13, 14, and 15 are the three boards on this file. BX-64 leaves VLEN at 128 or more, with 256 as the recommended elaboration. BX-128 pins VLEN at 128. BX-512 pins VLEN at 512. All three keep this hart.

12.2 The address

The address adder is 64 bits. A vector load takes its base from one of these registers. The address stays 64 bits when VLEN is pinned at 128 and when VLEN is pinned at 512. A BX-512 register still moves on the 64-bit data port of that chapter: eight beats for the whole register.

12.3 What crosses into the vector unit

The base address is the 64-bit GPR. `vmv.x.s` writes element 0 into a GPR. SEW 64 equals XLEN, so a double fills the register. A narrower element is sign-extended to 64 bits: a SEW-32 element fills bits 63:32 with the sign. Chapter 16 continues that sign-extension through bit 127.

`vmv.s.x` writes the low SEW bits of the GPR into element 0. A `.vx` instruction uses those low bits. The floating-point scalar comes from an F register. FLEN is 32 or 64, and `OPFVF` reads that register. The floating-point file stays at FLEN when the integer file later grows to 128 bits.

12.4 Fetch

The default elaboration fetches a native RV64 parcel. BX-64, BX-128, and BX-512 have no harness directory. A MIPS host, if you attach one, uses the COP3 word `4C000000` from Chapter 11. The address is the low 64 bits of the GPR.

Chapter 13

BX-64: Application Vectors with V and Zve64d

13.1 The architecture

BX-64 widens XLEN to 64. The base is RV64I. Chapter 12 is that integer file. The vector unit is Zve64d or the full application extension V. ELEN is 64. VLEN is at least 128, and 256 is the recommended elaboration when the block RAM budget allows a file of thirty-two registers times 256 bits, which is one kilobyte.

> Legal SEW = 8, 16, 32, and 64.
> FP32 and FP64, including fused multiply-add.
> If the claim is V: every instruction V requires, VLEN at least 128,
> and the V bit set in `misa`.
> If the claim is Zve64d: do not set the V bit.

The second sentence is the one that keeps the ladder honest. A machine that implements double-precision vector arithmetic and not yet the whole of V is a useful machine. It is not a V machine. Chapters 14 and 15 keep this element width and pin the register length. An integer register of 128 bits is Chapter 16.

13.2 What arrives with sixty-four-bit elements

SEW=64 is the element width a double-precision value needs, and the element width a widened thirty-two-bit product needs if you choose to widen that far. Both appear in geometry and in audio rate conversion. The lane's integer ALU grows a sixty-four-bit path, which on a sixteen-bit or thirty-two-bit DSP primitive is a short sequence of partial adds. That sequence is internal. The architectural latency is a published integer.

Masks, slides, and reductions at SEW=64 are required. Indexed accesses are required. `vstart` becomes architecturally visible if the machine has a page table and a vector load can fault on a later page than it started on. The simple machines avoided this by checking the whole access first. A sixty-four-bit hart with virtual memory cannot always do that cheaply. Implement `vstart`, test a fault on element 0 and a fault on a middle element, and document that restart is precise.

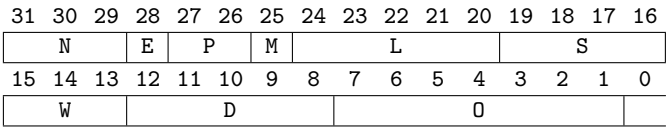
13.3 Opcode map for Zve64d / V (BX-64)

All opcodes of Zve32f remain legal. The additional element widths and FP64 operations use the same rigid two-line matrix. SEW may be 8, 16, 32 or 64. Every major class has an explicit entry diagram below or inherits the diagram from the lower profile.

13.3.1 Configuration

Identical to the lower profiles (see Chapter 9).

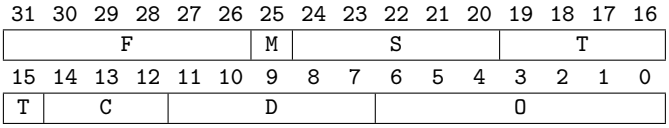
13.3.2 Unit-stride load at SEW=64 — `vle64.v`



N = nf
E = mew
P = mop (00 = unit-stride)
M = vm
L = lumop (00000)
S = rs1 (base)
W = width (111 = 64-bit)
D = vd
O = 0000111

Stride and indexed forms keep the same layout; only the mop and width fields change (width = 111 for EEW=64).

13.3.3 Integer / FP arithmetic (OP-V) at any SEW including 64



F = funct6
M = vm
S = vs2
T = vs1 / rs1 / fs1
C = funct3 (OPIVV / OPIVX / OPFVV / OPFVF ...)
D = vd
O = 1010111

All integer instructions of Zve32f are legal at SEW=64. All FP32 instructions remain legal; their FP64 counterparts use the same funct6 values with SEW=64.

Class	examples	notes
Integer	`vadd`, `vwmul`, `vredsum`, `vrgather`	full set
FP32	`vfadd`, `vfmacc`, `vfcvt`	from BX-32
FP64	`vfadd`, `vfmacc`, `vfcvt` at SEW=64	required for Zve64d/V

If the claim is the full application extension V, every additional instruction required by the V specification must be present and the V bit set in `misa`. A Zve64d implementation omits the V bit and may omit instructions that V adds beyond Zve64d.

13.3.4 Mask and move

Full mask set plus the FP move variants for both single and double.

Xsvm must be left off on any image that claims V.

13.4 Assembler kernels that must run

In addition to every kernel of the lower profiles, the following must pass on BX-64. The COP3 framing of Chapter 11 is the host pattern at this width too. The address is the low 64 bits of the GPR. There is no separate harness directory in this edition.

Table 13.1: Kernel analogues on this machine.

Kernel	RVV	Intel	ARM
sew64	`vse64.v`	`vmovupd`	`st1`
fma64	`vfmacc.vv`	`vfmadd231pd`	`fmla`
gemm64	`vfmacc.vf`	`vmovddup`	`ld1r`
index64	`vluxei64.v`	`vpgatherqq`	`ldr`

The comparison table under each kernel is one row per instruction: RVV, then Intel, then ARM. Chapter 9 says which encodings those bytes are.

13.4.1 sew64 — double-word fill

The element is 64 bits. VLEN 128 holds two of them.

Table 13.2: Kernel comparison: sew64

`a0 = n, a1 = fill, a2 = dst`

ISA	Address	Hex	Assembly
RVV	0000	02050063	`sew64: beqz a0, end`
RVV	0004	003572D7	`loop: vsetvli t0, a0, e64, m1, tu, mu`
RVV	0008	5E05C0D7	`vmv.v.x v1, a1`
RVV	000C	020670A7	`vse64.v v1, (a2)`
RVV	0010	00329313	`slli t1, t0, 3`
RVV	0014	00660633	`add a2, a2, t1`
RVV	0018	40550533	`sub a0, a0, t0`
RVV	001C	FE0514E3	`bnez a0, loop`
RVV	0020	00008067	`end: ret`
Intel	0000	C4E1F96EC6	`sew64: vmovq xmm0, rsi`
Intel	0005	C4E27959C0	`vpbroadcastq xmm0, xmm0`
Intel	000A	C5F91107	`vmovupd [rdi], xmm0`
ARM	0000	4E080C20	`sew64: dup v0.2d, x1`
ARM	0004	4C007C40	`st1 {v0.2d}, [x2]`

`kernels/bx64/sew64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`

13.4.2 fma64 — double-precision fused multiply-add

`kernels/bx64/fma64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`

Table 13.3: Kernel comparison: fma64

a0 = n, a1 = a, a2 = b, a3 = c. c = c + a*b

ISA	Address	Hex	Assembly
RVV	0000	02050A63	fma64: beqz a0, end
RVV	0004	003572D7	loop: vsetvli t0, a0, e64, m1, tu, mu
RVV	0008	0205F087	vle64.v v1, (a1)
RVV	000C	02067107	vle64.v v2, (a2)
RVV	0010	0206F187	vle64.v v3, (a3)
RVV	0014	B22091D7	vfmacc.vv v3, v1, v2
RVV	0018	0206F1A7	vse64.v v3, (a3)
RVV	001C	00329313	slli t1, t0, 3
RVV	0020	006585B3	add a1, a1, t1
RVV	0024	00660633	add a2, a2, t1
RVV	0028	006686B3	add a3, a3, t1
RVV	002C	40550533	sub a0, a0, t0
RVV	0030	FC051AE3	bnez a0, loop
RVV	0034	00008067	end: ret
Intel	0000	C5FD1006	fma64: vmovupd ymm0, [rsi]
Intel	0004	C5FD100A	vmovupd ymm1, [rdx]
Intel	0008	C5FD1011	vmovupd ymm2, [rcx]
Intel	000C	C4E2FDB8D1	vfmadd231pd ymm2, ymm0, ymm1
Intel	0011	C5FD1111	vmovupd [rcx], ymm2
ARM	0000	4C407C20	fma64: ld1 {v0.2d}, [x1]
ARM	0004	4C407C41	ld1 {v1.2d}, [x2]
ARM	0008	4C407C62	ld1 {v2.2d}, [x3]
ARM	000C	4E61CC02	fmla v2.2d, v0.2d, v1.2d
ARM	0010	4C007C62	st1 {v2.2d}, [x3]

13.4.3 gemm64 — outer product of a 2-wide panel

The BX-32 catalogue in Section 11.7 is the same list at SEW 64, with `fma64`, `index64`, and `gemm64`. VLEN 128 holds two doubles, so the acceptance tile is 2 by 4: one vector of A and four broadcast scalars from B. `fld` is LOAD-FP with funct3 011.

The GEMM note's AVX2 tile holds four doubles in a `ymm`. Its AVX-512 tile is 24 by 8, eight doubles in each `zmm`. The Intel column here is two doubles in an `xmm`. `vmovddup` writes both lanes from one memory double.

`kernels/bx64/gemm64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`

13.4.4 index64 — indexed load at SEW=64

The index EEW and the data EEW are both 64.

`kernels/bx64/index64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`

13.5 The same bodies at SEW 64

The names in Section 11.7 with a `d` prefix use these images. `ddot` is `dot64`. `dnrm2` is `nrm264`. `dasum` is `asum64`. `drotg` and `drotmg` stay scalar. `idamax` is one strip of `iamax64`. The product is `gemm64`. The Intel column of that product is two doubles, via `vmovddup`. A `ymm` holds four doubles. A 24 by 8 tile is `zmm`, and that register file is Chapter 15.

Reassemble `bfly32`, `wmul32`, and `cmul32` with `e64` when the samples are doubles. FFTW's default transform, NumPy's `fft`, and a double-precision channelizer are that butterfly. GNU Radio's fast path stays at SEW 32.

`kernels/bx64/scal64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`
`kernels/bx64/copy64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`
`kernels/bx64/axpy64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`
`kernels/bx64/dot64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`

Table 13.4: Kernel comparison: gemm64

a0 = K, a1 = A panel, a2 = B panel, a3 = C. 2-wide outer product.

ISA	Address	Hex	Assembly
RVV	0000	06050A63	gemm64: beqz a0, end
RVV	0004	00307057	vsetvli zero, zero, e64, m1, tu, mu
RVV	0008	0206F407	vle64.v v8, (a3)
RVV	000C	01068293	addi t0, a3, 16
RVV	0010	0202F487	vle64.v v9, (t0)
RVV	0014	02068293	addi t0, a3, 32
RVV	0018	0202F507	vle64.v v10, (t0)
RVV	001C	03068293	addi t0, a3, 48
RVV	0020	0202F587	vle64.v v11, (t0)
RVV	0024	0205F087	loop: vle64.v v1, (a1)
RVV	0028	00063007	fld ft0, 0(a2)
RVV	002C	B2105457	vfmacc.vf v8, ft0, v1
RVV	0030	00863007	fld ft0, 8(a2)
RVV	0034	B21054D7	vfmacc.vf v9, ft0, v1
RVV	0038	01063007	fld ft0, 16(a2)
RVV	003C	B2105557	vfmacc.vf v10, ft0, v1
RVV	0040	01863007	fld ft0, 24(a2)
RVV	0044	B21055D7	vfmacc.vf v11, ft0, v1
RVV	0048	01058593	addi a1, a1, 16
RVV	004C	02060613	addi a2, a2, 32
RVV	0050	FFF50513	addi a0, a0, -1
RVV	0054	FC0518E3	bnez a0, loop
RVV	0058	0206F427	vse64.v v8, (a3)
RVV	005C	01068293	addi t0, a3, 16
RVV	0060	0202F4A7	vse64.v v9, (t0)
RVV	0064	02068293	addi t0, a3, 32
RVV	0068	0202F527	vse64.v v10, (t0)
RVV	006C	03068293	addi t0, a3, 48
RVV	0070	0202F5A7	vse64.v v11, (t0)
RVV	0074	00008067	end: ret
Intel	0000	C5F91006	gemm64: vmovupd xmm0, [rsi]
Intel	0004	C5FB120A	vmovddup xmm1, qword ptr [rdx]
Intel	0008	C4E2F9B8E1	vfmadd231pd xmm4, xmm0, xmm1
Intel	000D	C5FB124A08	vmovddup xmm1, qword ptr [rdx+8]
Intel	0012	C4E2F9B8E9	vfmadd231pd xmm5, xmm0, xmm1
Intel	0017	C5FB124A10	vmovddup xmm1, qword ptr [rdx+16]
Intel	001C	C4E2F9B8F1	vfmadd231pd xmm6, xmm0, xmm1
Intel	0021	C5FB124A18	vmovddup xmm1, qword ptr [rdx+24]
Intel	0026	C4E2F9B8F9	vfmadd231pd xmm7, xmm0, xmm1
Intel	002B	4883C610	add rsi, 16
Intel	002F	4883C220	add rdx, 32
Intel	0033	4883E901	sub rcx, 1
Intel	0037	75C7	jnz gemm64
ARM	0000	4CDF7C20	gemm64: ld1 {v0.2d}, [x1], #16
ARM	0004	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0008	4E61CC04	fmla v4.2d, v0.2d, v1.2d
ARM	000C	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0010	4E61CC05	fmla v5.2d, v0.2d, v1.2d
ARM	0014	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0018	4E61CC06	fmla v6.2d, v0.2d, v1.2d
ARM	001C	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0020	4E61CC07	fmla v7.2d, v0.2d, v1.2d
ARM	0024	F1000400	subs x0, x0, #1
ARM	0028	54FFFEC1	b.ne gemm64

Table 13.5: Kernel comparison: index64

`a0 = n, a1 = base, a2 = index vector, a3 = dst`

ISA	Address	Hex	Assembly
RVV	0000	02050463	`index64: beqz a0, end`
RVV	0004	003572D7	`loop: vsetvli t0, a0, e64, m1, tu, mu`
RVV	0008	02067107	`vle64.v v2, (a2)`
RVV	000C	0625F087	`vluxei64.v v1, (a1), v2`
RVV	0010	0206F0A7	`vse64.v v1, (a3)`
RVV	0014	00329313	`slli t1, t0, 3`
RVV	0018	00660633	`add a2, a2, t1`
RVV	001C	006686B3	`add a3, a3, t1`
RVV	0020	40550533	`sub a0, a0, t0`
RVV	0024	FE0510E3	`bnez a0, loop`
RVV	0028	00008067	`end: ret`
Intel	0000	C4E2ED91040E	`index64: vpgatherqq ymm0, [rsi + ymm1*1], ymm2`
ARM	0000	FC637820	`index64: ldr d0, [x1, x3, lsl #3]`
ARM	0004	FD000040	`str d0, [x2]`
ARM	0008	91002063	`add x3, x3, #8`
ARM	000C	91002042	`add x2, x2, #8`
ARM	0010	F1000400	`subs x0, x0, #1`
ARM	0014	54FFFF61	`b.ne index64`

Table 13.6: Programs that run the BX-64 kernels.

Kernel	Work	Where it ships
sew64	doubleword fill	A zeroed double buffer.
fma64	element $c + ab$	A double polynomial step. The inner product of a compensated sum, written as a vector.
gemm64	2-wide rank-1	`dgemm` in OpenBLAS, BLIS, and LAPACK's callers. The GEMM note's double update, here two doubles wide.
index64	gather of doubles	Sparse matrix-vector with double values. A gather of double-precision samples.
scal64	scale	`dscal`. Column scaling in LAPACK.
copy64	copy	`dcopy`. Packing a double panel.
axpy64	scaled add	`daxpy`. CG in double precision.
dot64	inner product	`ddot`. A double correlation. SoXR's high-quality FIR.
nrm264	Euclidean length	`dnrm2`. A residual norm in a double least-squares solve.
asum64	L1 norm	`dasum`.
swap64	exchange	`dswap`. Partial pivoting.
rot64	Givens rotation	`drot`. Double QR.
gemv64	matrix-vector	`dgemv`. The GEMM note's `cblas_dgemv`, two doubles tall.
ger64	rank-1	`dger`. A double covariance update.
iamax64	index of max abs	`idamax`. The pivot of a double LU. One strip.

`kernels/bx64/nrm264.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`
`kernels/bx64/asum64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`
`kernels/bx64/swap64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`
`kernels/bx64/rot64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`
`kernels/bx64/gemv64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`
`kernels/bx64/ger64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`
`kernels/bx64/iamax64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin`

13.6 Harts

The organisation that justifies a second hart is the one the wide machines are for: one hart runs the operating system and short vector work, the other runs a longer kernel from a local memory. Both are RV64 with the same vector claim. A DMA engine moves blocks into that

Table 13.7: Kernel comparison: scal64

a0 = n, a1 = x, a2 = alpha. x *= alpha

ISA	Address	Hex	Assembly
RVV	0000	00063007	scal64: fld ft0, 0(a2)
RVV	0004	02050263	beqz a0, end
RVV	0008	003572D7	loop: vsetvli t0, a0, e64, m1, tu, mu
RVV	000C	0205F087	vle64.v v1, (a1)
RVV	0010	921050D7	vfmul.vf v1, v1, ft0
RVV	0014	0205F0A7	vse64.v v1, (a1)
RVV	0018	00329313	slli t1, t0, 3
RVV	001C	006585B3	add a1, a1, t1
RVV	0020	40550533	sub a0, a0, t0
RVV	0024	FE0512E3	bnez a0, loop
RVV	0028	00008067	end: ret
Intel	0000	C5FB120E	scal64: vmovddup xmm1, qword ptr [rsi]
Intel	0004	C5F91007	vmovupd xmm0, [rdi]
Intel	0008	C5F959C1	vmulpd xmm0, xmm0, xmm1
Intel	000C	C5F91107	vmovupd [rdi], xmm0
Intel	0010	4883C710	add rdi, 16
Intel	0014	4883EA02	sub rdx, 2
Intel	0018	75E6	jnz scal64
ARM	0000	4D40CC41	scal64: ld1r {v1.2d}, [x2]
ARM	0004	4C407C20	ld1 {v0.2d}, [x1]
ARM	0008	6E61DC00	fmul v0.2d, v0.2d, v1.2d
ARM	000C	4C007C20	st1 {v0.2d}, [x1]
ARM	0010	91004021	add x1, x1, #16
ARM	0014	F1000800	subs x0, x0, #2
ARM	0018	54FFFF41	b.ne scal64

Table 13.8: Kernel comparison: copy64

a0 = n, a1 = src, a2 = dst

ISA	Address	Hex	Assembly
RVV	0000	02050263	copy64: beqz a0, end
RVV	0004	003572D7	loop: vsetvli t0, a0, e64, m1, tu, mu
RVV	0008	0205F087	vle64.v v1, (a1)
RVV	000C	020670A7	vse64.v v1, (a2)
RVV	0010	00329313	slli t1, t0, 3
RVV	0014	006585B3	add a1, a1, t1
RVV	0018	00660633	add a2, a2, t1
RVV	001C	40550533	sub a0, a0, t0
RVV	0020	FE0512E3	bnez a0, loop
RVV	0024	00008067	end: ret
Intel	0000	C5F91006	copy64: vmovupd xmm0, [rsi]
Intel	0004	C5F91107	vmovupd [rdi], xmm0
Intel	0008	4883C610	add rsi, 16
Intel	000C	4883C710	add rdi, 16
Intel	0010	4883EA02	sub rdx, 2
Intel	0014	75EA	jnz copy64
ARM	0000	4C407C20	copy64: ld1 {v0.2d}, [x1]
ARM	0004	4C007C40	st1 {v0.2d}, [x2]
ARM	0008	91004021	add x1, x1, #16
ARM	000C	91004042	add x2, x2, #16
ARM	0010	F1000800	subs x0, x0, #2
ARM	0014	54FFFF61	b.ne copy64

Table 13.9: Kernel comparison: axpy64

a0 = n, a1 = x, a2 = y, a3 = alpha. y += alpha*x

ISA	Address	Hex	Assembly
RVV	0000	0006B007	axpy64: fld ft0, 0(a3)
RVV	0004	02050663	beqz a0, end
RVV	0008	003572D7	loop: vsetvli t0, a0, e64, m1, tu, mu
RVV	000C	0205F087	vle64.v v1, (a1)
RVV	0010	02067107	vle64.v v2, (a2)
RVV	0014	B2105157	vfmacc.vf v2, ft0, v1
RVV	0018	02067127	vse64.v v2, (a2)
RVV	001C	00329313	slli t1, t0, 3
RVV	0020	006585B3	add a1, a1, t1
RVV	0024	00660633	add a2, a2, t1
RVV	0028	40550533	sub a0, a0, t0
RVV	002C	FC051EE3	bnez a0, loop
RVV	0030	00008067	end: ret
Intel	0000	C5FB1211	axpy64: vmovddup xmm2, qword ptr [rcx]
Intel	0004	C5F91006	vmovupd xmm0, [rsi]
Intel	0008	C5F9100F	vmovupd xmm1, [rdi]
Intel	000C	C4E2F9B8CA	vfmadd231pd xmm1, xmm0, xmm2
Intel	0011	C5F9110F	vmovupd [rdi], xmm1
Intel	0015	4883C610	add rsi, 16
Intel	0019	4883C710	add rdi, 16
Intel	001D	4883EA02	sub rdx, 2
Intel	0021	75DD	jnz axpy64
ARM	0000	4D40CC62	axpy64: ld1r {v2.2d}, [x3]
ARM	0004	4C407C20	ld1 {v0.2d}, [x1]
ARM	0008	4C407C41	ld1 {v1.2d}, [x2]
ARM	000C	4E62CC01	fmla v1.2d, v0.2d, v2.2d
ARM	0010	4C007C41	st1 {v1.2d}, [x2]
ARM	0014	91004021	add x1, x1, #16
ARM	0018	91004042	add x2, x2, #16
ARM	001C	F1000800	subs x0, x0, #2
ARM	0020	54FFFF01	b.ne axpy64

Table 13.10: Kernel comparison: dot64

a0 = n, a1 = x, a2 = y, a3 = sum out

ISA	Address	Hex	Assembly
RVV	0000	5E003257	dot64: vmv.v.i v4, 0
RVV	0004	02050663	beqz a0, end
RVV	0008	003572D7	loop: vsetvli t0, a0, e64, m1, tu, mu
RVV	000C	0205F087	vle64.v v1, (a1)
RVV	0010	02067107	vle64.v v2, (a2)
RVV	0014	921110D7	vfmul.vv v1, v1, v2
RVV	0018	06121257	vfredusum.vsv4, v1, v4
RVV	001C	00329313	slli t1, t0, 3
RVV	0020	006585B3	add a1, a1, t1
RVV	0024	00660633	add a2, a2, t1
RVV	0028	40550533	sub a0, a0, t0
RVV	002C	FC051EE3	bnez a0, loop
RVV	0030	42401057	end: vfmv.f.s ft0, v4
RVV	0034	0006B027	fsd ft0, 0(a3)
RVV	0038	00008067	ret
Intel	0000	C5F91006	dot64: vmovupd xmm0, [rsi]
Intel	0004	C5F9100A	vmovupd xmm1, [rdx]
Intel	0008	C5F959C1	vmulpd xmm0, xmm0, xmm1
Intel	000C	660F7CC0	haddpd xmm0, xmm0
ARM	0000	4C407C20	dot64: ld1 {v0.2d}, [x1]
ARM	0004	4C407C41	ld1 {v1.2d}, [x2]
ARM	0008	6E61DC00	fmul v0.2d, v0.2d, v1.2d
ARM	000C	6E60D400	faddp v0.2d, v0.2d, v0.2d

Table 13.11: Kernel comparison: nrm264

`a0 = n, a1 = x, a2 = norm out.  sqrt of sum of squares`

ISA	Address	Hex	Assembly
RVV	0000	5E003257	`nrm264:  vmv.v.i v4, 0`
RVV	0004	02050263	`beqz a0, end`
RVV	0008	003572D7	`loop:  vsetvli t0, a0, e64, m1, tu, mu`
RVV	000C	0205F087	`vle64.v v1, (a1)`
RVV	0010	921090D7	`vfmul.vv v1, v1, v1`
RVV	0014	06121257	`vfredusum.vsv4, v1, v4`
RVV	0018	00329313	`slli t1, t0, 3`
RVV	001C	006585B3	`add a1, a1, t1`
RVV	0020	40550533	`sub a0, a0, t0`
RVV	0024	FE0512E3	`bnez a0, loop`
RVV	0028	42401057	`end:  vfmv.f.s ft0, v4`
RVV	002C	5A007053	`fsqrt.d ft0, ft0`
RVV	0030	00063027	`fsd ft0, 0(a2)`
RVV	0034	00008067	`ret`
Intel	0000	C5F91006	`nrm264:  vmovupd xmm0, [rsi]`
Intel	0004	C5F959C0	`vmulpd xmm0, xmm0, xmm0`
Intel	0008	660F7CC0	`haddpd xmm0, xmm0`
Intel	000C	F20F51C0	`sqrtsd xmm0, xmm0`
ARM	0000	4C407C20	`nrm264:  ld1 {v0.2d}, [x1]`
ARM	0004	6E60DC00	`fmul v0.2d, v0.2d, v0.2d`
ARM	0008	6E60D400	`faddp v0.2d, v0.2d, v0.2d`
ARM	000C	1E61C000	`fsqrt d0, d0`

Table 13.12: Kernel comparison: asum64

`a0 = n, a1 = x, a2 = sum of absolute values`

ISA	Address	Hex	Assembly
RVV	0000	5E003257	`asum64:  vmv.v.i v4, 0`
RVV	0004	02050263	`beqz a0, end`
RVV	0008	003572D7	`loop:  vsetvli t0, a0, e64, m1, tu, mu`
RVV	000C	0205F087	`vle64.v v1, (a1)`
RVV	0010	2A1090D7	`vfsgnjx.vv v1, v1, v1`
RVV	0014	06121257	`vfredusum.vsv4, v1, v4`
RVV	0018	00329313	`slli t1, t0, 3`
RVV	001C	006585B3	`add a1, a1, t1`
RVV	0020	40550533	`sub a0, a0, t0`
RVV	0024	FE0512E3	`bnez a0, loop`
RVV	0028	42401057	`end:  vfmv.f.s ft0, v4`
RVV	002C	00063027	`fsd ft0, 0(a2)`
RVV	0030	00008067	`ret`
Intel	0000	C5F91006	`asum64:  vmovupd xmm0, [rsi]`
Intel	0004	C5F954C7	`vandpd xmm0, xmm0, xmm7`
Intel	0008	660F7CC0	`haddpd xmm0, xmm0`
ARM	0000	4C407C20	`asum64:  ld1 {v0.2d}, [x1]`
ARM	0004	4EE0F800	`fabs v0.2d, v0.2d`
ARM	0008	6E60D400	`faddp v0.2d, v0.2d, v0.2d`

Table 13.13: Kernel comparison: swap64

a0 = n, a1 = x, a2 = y

ISA	Address	Hex	Assembly
RVV	0000	02050663	swap64: beqz a0, end
RVV	0004	003572D7	loop: vsetvli t0, a0, e64, m1, tu, mu
RVV	0008	0205F087	vle64.v v1, (a1)
RVV	000C	02067107	vle64.v v2, (a2)
RVV	0010	0205F127	vse64.v v2, (a1)
RVV	0014	020670A7	vse64.v v1, (a2)
RVV	0018	00329313	slli t1, t0, 3
RVV	001C	006585B3	add a1, a1, t1
RVV	0020	00660633	add a2, a2, t1
RVV	0024	40550533	sub a0, a0, t0
RVV	0028	FC051EE3	bnez a0, loop
RVV	002C	00008067	end: ret
Intel	0000	C5F91006	swap64: vmovupd xmm0, [rsi]
Intel	0004	C5F9100F	vmovupd xmm1, [rdi]
Intel	0008	C5F9110E	vmovupd [rsi], xmm1
Intel	000C	C5F91107	vmovupd [rdi], xmm0
Intel	0010	4883C610	add rsi, 16
Intel	0014	4883C710	add rdi, 16
Intel	0018	4883EA02	sub rdx, 2
Intel	001C	75E2	jnz swap64
ARM	0000	4C407C20	swap64: ld1 {v0.2d}, [x1]
ARM	0004	4C407C41	ld1 {v1.2d}, [x2]
ARM	0008	4C007C21	st1 {v1.2d}, [x1]
ARM	000C	4C007C40	st1 {v0.2d}, [x2]
ARM	0010	91004021	add x1, x1, #16
ARM	0014	91004042	add x2, x2, #16
ARM	0018	F1000800	subs x0, x0, #2
ARM	001C	54FFFF21	b.ne swap64

local memory. The DMA is a bus master programmed by stores, in the same style as the blitter on BX-16. It does not fetch vector instructions.

Local memory is mapped. Vector loads and stores name it with ordinary addresses. A kernel that ran on BX-32 in SEW=32 runs here in SEW=32 without change. A kernel that wants FP64 sets SEW=64 and uses the double-precision opcodes.

13.7 Xsvm on this machine

Leave Xsvm off on the image you use to claim V. A standard toolchain will not emit it, and a custom opcode in the middle of an otherwise clean V implementation confuses every test suite you did not write. A second elaboration, with Xsvm on, is how you compare a four-by-four custom instruction with the fused-multiply sequence. Publish both cycle counts. Keep the V-claiming image free of custom geometry.

Table 13.14: Kernel comparison: rot64

`a0 = n, a1 = x, a2 = y, a3 = (c, s)`

ISA	Address	Hex	Assembly
RVV	0000	0006B007	`rot64:  fld ft0, 0(a3)`
RVV	0004	0086B087	`fld ft1, 8(a3)`
RVV	0008	04050263	`beqz a0, end`
RVV	000C	003572D7	`loop:  vsetvli t0, a0, e64, m1, tu, mu`
RVV	0010	0205F087	`vle64.v v1, (a1)`
RVV	0014	02067107	`vle64.v v2, (a2)`
RVV	0018	921051D7	`vfmul.vf v3, v1, ft0`
RVV	001C	9220D257	`vfmul.vf v4, v2, ft1`
RVV	0020	023212D7	`vfadd.vv v5, v3, v4`
RVV	0024	922051D7	`vfmul.vf v3, v2, ft0`
RVV	0028	9210D257	`vfmul.vf v4, v1, ft1`
RVV	002C	0A321357	`vfsub.vv v6, v3, v4`
RVV	0030	0205F2A7	`vse64.v v5, (a1)`
RVV	0034	02067327	`vse64.v v6, (a2)`
RVV	0038	00329313	`slli t1, t0, 3`
RVV	003C	006585B3	`add a1, a1, t1`
RVV	0040	00660633	`add a2, a2, t1`
RVV	0044	40550533	`sub a0, a0, t0`
RVV	0048	FC0512E3	`bnez a0, loop`
RVV	004C	00008067	`end:  ret`
Intel	0000	C5FB1211	`rot64:  vmovddup xmm2, qword ptr [rcx]`
Intel	0004	C5FB125908	`vmovddup xmm3, qword ptr [rcx+8]`
Intel	0009	C5F91006	`vmovupd xmm0, [rsi]`
Intel	000D	C5F9100F	`vmovupd xmm1, [rdi]`
Intel	0011	C5F959E2	`vmulpd xmm4, xmm0, xmm2`
Intel	0015	C4E2F1B8E3	`vfmadd231pd xmm4, xmm1, xmm3`
Intel	001A	C5F159EA	`vmulpd xmm5, xmm1, xmm2`
Intel	001E	C4E2F9BCEB	`vfnmadd231pd xmm5, xmm0, xmm3`
Intel	0023	C5F91126	`vmovupd [rsi], xmm4`
Intel	0027	C5F9112F	`vmovupd [rdi], xmm5`
ARM	0000	4D40CC62	`rot64:  ld1r {v2.2d}, [x3]`
ARM	0004	91002063	`add x3, x3, #8`
ARM	0008	4D40CC63	`ld1r {v3.2d}, [x3]`
ARM	000C	4C407C20	`ld1 {v0.2d}, [x1]`
ARM	0010	4C407C41	`ld1 {v1.2d}, [x2]`
ARM	0014	6E62DC04	`fmul v4.2d, v0.2d, v2.2d`
ARM	0018	4E63CC24	`fmla v4.2d, v1.2d, v3.2d`
ARM	001C	6E62DC25	`fmul v5.2d, v1.2d, v2.2d`
ARM	0020	4EE3CC05	`fmls v5.2d, v0.2d, v3.2d`
ARM	0024	4C007C24	`st1 {v4.2d}, [x1]`
ARM	0028	4C007C45	`st1 {v5.2d}, [x2]`

Table 13.15: Kernel comparison: gemv64

`a0 = columns, a1 = A, a2 = x, a3 = y.  rows = VLMAX, lda = VLMAX`

ISA	Address	Hex	Assembly
RVV	0000	02050863	`gemv64:  beqz a0, end`
RVV	0004	003072D7	`vsetvli t0, zero, e64, m1, tu, mu`
RVV	0008	00329313	`slli t1, t0, 3`
RVV	000C	0206F407	`vle64.v v8, (a3)`
RVV	0010	0205F087	`loop:  vle64.v v1, (a1)`
RVV	0014	00063007	`fld ft0, 0(a2)`
RVV	0018	B2105457	`vfmacc.vf v8, ft0, v1`
RVV	001C	006585B3	`add a1, a1, t1`
RVV	0020	00860613	`addi a2, a2, 8`
RVV	0024	FFF50513	`addi a0, a0, -1`
RVV	0028	FE0514E3	`bnez a0, loop`
RVV	002C	0206F427	`vse64.v v8, (a3)`
RVV	0030	00008067	`end:  ret`
Intel	0000	C5F91006	`gemv64:  vmovupd xmm0, [rsi]`
Intel	0004	C5FB120A	`vmovddup xmm1, qword ptr [rdx]`
Intel	0008	C4E2F9B8E1	`vfmadd231pd xmm4, xmm0, xmm1`
Intel	000D	4883C610	`add rsi, 16`
Intel	0011	4883C208	`add rdx, 8`
Intel	0015	4883E901	`sub rcx, 1`
Intel	0019	75E5	`jnz gemv64`
ARM	0000	4CDF7C20	`gemv64:  ld1 {v0.2d}, [x1], #16`
ARM	0004	4DDFCC41	`ld1r {v1.2d}, [x2], #8`
ARM	0008	4E61CC04	`fmla v4.2d, v0.2d, v1.2d`
ARM	000C	F1000400	`subs x0, x0, #1`
ARM	0010	54FFFF81	`b.ne gemv64`

Table 13.16: Kernel comparison: ger64

`a0 = columns, a1 = x, a2 = y, a3 = A. A += x y^T, lda = VLMAX`

ISA	Address	Hex	Assembly
RVV	0000	02050863	`ger64:  beqz a0, end`
RVV	0004	003072D7	`vsetvli t0, zero, e64, m1, tu, mu`
RVV	0008	00329313	`slli t1, t0, 3`
RVV	000C	0205F087	`vle64.v v1, (a1)`
RVV	0010	0206F407	`loop:  vle64.v v8, (a3)`
RVV	0014	00063007	`fld ft0, 0(a2)`
RVV	0018	B2105457	`vfmacc.vf v8, ft0, v1`
RVV	001C	0206F427	`vse64.v v8, (a3)`
RVV	0020	006686B3	`add a3, a3, t1`
RVV	0024	00860613	`addi a2, a2, 8`
RVV	0028	FFF50513	`addi a0, a0, -1`
RVV	002C	FE0512E3	`bnez a0, loop`
RVV	0030	00008067	`end:  ret`
Intel	0000	C5F9100E	`ger64:  vmovupd xmm1, [rsi]`
Intel	0004	C5F91001	`vmovupd xmm0, [rcx]`
Intel	0008	C5FB1212	`vmovddup xmm2, qword ptr [rdx]`
Intel	000C	C4E2F1B8C2	`vfmadd231pd xmm0, xmm1, xmm2`
Intel	0011	C5F91101	`vmovupd [rcx], xmm0`
Intel	0015	4883C110	`add rcx, 16`
Intel	0019	4883C208	`add rdx, 8`
Intel	001D	4883E801	`sub rax, 1`
Intel	0021	75DD	`jnz ger64`
ARM	0000	4C407C21	`ger64:  ld1 {v1.2d}, [x1]`
ARM	0004	4C407C60	`ld1 {v0.2d}, [x3]`
ARM	0008	4DDFCC42	`ld1r {v2.2d}, [x2], #8`
ARM	000C	4E62CC20	`fmla v0.2d, v1.2d, v2.2d`
ARM	0010	4C9F7C60	`st1 {v0.2d}, [x3], #16`
ARM	0014	F1000400	`subs x0, x0, #1`
ARM	0018	54FFFF41	`b.ne ger64`

Table 13.17: Kernel comparison: iamax64

a0 = n (<= VLMAX), a1 = x, a2 = index out. First index of max |x|

ISA	Address	Hex	Assembly
RVV	0000	02050263	iamax64: beqz a0, end
RVV	0004	003572D7	vsetvli t0, a0, e64, m1, tu, mu
RVV	0008	0205F087	vle64.v v1, (a1)
RVV	000C	2A1090D7	vfsgnjx.vv v1, v1, v1
RVV	0010	1E109157	vfredmax.vs v2, v1, v1
RVV	0014	42201057	vfmv.f.s ft0, v2
RVV	0018	621051D7	vmfeq.vf v3, v1, ft0
RVV	001C	4238A2D7	vfirst.m t0, v3
RVV	0020	00562023	sw t0, 0(a2)
RVV	0024	00008067	end: ret
Intel	0000	C5F91007	iamax64: vmovupd xmm0, [rdi]
Intel	0004	C5F954C7	vandpd xmm0, xmm0, xmm7
Intel	0008	0F12C8	movhlps xmm1, xmm0
Intel	000B	660F5FC1	maxpd xmm0, xmm1
ARM	0000	4C407C20	iamax64: ld1 {v0.2d}, [x1]
ARM	0004	4EE0F800	fabs v0.2d, v0.2d
ARM	0008	7E70F801	fmaxp d1, v0.2d
ARM	000C	4E080421	dup v1.2d, v1.d[0]
ARM	0010	6EE18C02	cmeq v2.2d, v0.2d, v1.2d

Chapter 14

BX-128: A Pinned 128-bit Register

14.1 The board-level picture

BX-128 is the BX-64 board with the vector file pinned at 128 bits. It does not replace a console. Instruction fetch has its own 32-bit port. Data memory is a 64-bit port. The hart is RV64, Chapter 12. The vector file is thirty-two registers of 128 bits, 512 bytes. Chapter 16 is the integer base with XLEN 128. This board keeps the RV64 hart and pins the vector register.

A vector load at SEW 64 moves two elements from one register. A vector load at SEW 32 moves four. Lane counts, the two register files, and this pin beside an RV128 hart are Chapter 7.

XLEN = 64. ELEN = 64. VLEN = 128.
Thirty-two vector registers. VRF = 512 bytes.
Instruction port 32 bits. Data port 64 bits.

14.2 The architecture

Legal SEW = 8, 16, 32, and 64.
VLMAX = 16, 8, 4, and 2 at those widths.
FP32 and FP64, including fused multiply-add.
If the claim is V: every instruction V requires, and the V bit set in `misa`.
If the claim is Zve64d: do not set the V bit.
SEW 128 traps.

`vsetvli` with `rs1` equal to `x0` writes VLMAX into `rd`. VLEN is not an instruction field. The acceptance filename records 128, which is the length this image was accepted on.

14.3 Widening

Widening multiply and narrowing shift are the BX-64 operations. They are legal here because ELEN is 64. A 128-bit register holds more elements of that SEW than an eight-bit machine does, and the same number the BX-64 floor holds. In the matrix of Section 11.7 the extra elements are a taller column slice. LMUL groups registers. A group is still a vector.

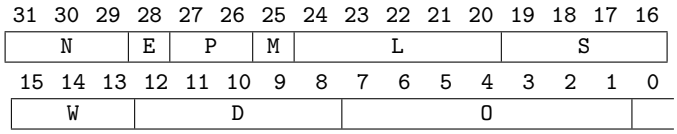
14.4 Opcode map for BX-128

The parcels are the BX-64 parcels. The diagrams below are the two fields this chapter uses. Integer, FP32, FP64, fused multiply-add, mask, and indexed forms are the map in Chapter 13.

14.4.1 Configuration

Identical to Chapter 9. `zimm` selects SEW and LMUL. Bit 31 stays clear. `vsetvli t0, zero, e32, m1, tu, mu` is the word `002072D7`. On this machine `t0` receives 4.

14.4.2 Unit-stride load at SEW=64 — `vle64.v`



N = nf
E = mew
P = mop (00 = unit-stride)
M = vm
L = lumop (00000)
S = rs1 (base)
W = width (111 = 64-bit)
D = vd
O = 0000111

EEW 8, 16, and 32 keep the width codes already printed: 000, 101, and 110.

14.5 RV64 and COP3 fetch

Path	Escape	Address
RV64	native OP-V	full 64-bit GPR
COP3 host	`4C000000`	low 64 bits of the GPR

The RV64 parcel is little-endian. The COP3 escape is the BX-32 escape, and the parcel endian is the host's. COP2 stays the host's own unit. There is no harness directory for this machine.

14.6 Assembler kernels that must run

Every BX-64 acceptance image is required here. The tables stay in Chapter 13. This chapter prints the tiles whose immediate stride is VLMAX at SEW 32. Offsets 16, 32, and 48 are that stride. `gemm64` is two doubles.

Table 14.1: Kernel analogues on this machine.

Kernel	RVV	Intel	ARM
gemm32	`vfmacc.vf`	`vbroadcastss`	`ld1r`
gemm64	`vfmacc.vf`	`vmovddup`	`ld1r`

gemm32 is the sgemm tile of Chapter 11: OpenBLAS, BLIS, Eigen, and a convolution after the patches are packed. gemm64 is dgemm at two doubles. The rest of that catalogue, including wmul32, bfly32, and cmul32, is required here at the same element width.

Table 14.2: Kernel comparison: gemm32

a0 = K, a1 = A panel, a2 = B panel, a3 = C. 4-wide outer product.

ISA	Address	Hex	Assembly
RVV	0000	06050A63	gemm32: beqz a0, end
RVV	0004	00207057	vsetvli zero, zero, e32, m1, tu, mu
RVV	0008	0206E407	vle32.v v8, (a3)
RVV	000C	01068293	addi t0, a3, 16
RVV	0010	0202E487	vle32.v v9, (t0)
RVV	0014	02068293	addi t0, a3, 32
RVV	0018	0202E507	vle32.v v10, (t0)
RVV	001C	03068293	addi t0, a3, 48
RVV	0020	0202E587	vle32.v v11, (t0)
RVV	0024	0205E087	loop: vle32.v v1, (a1)
RVV	0028	00062007	flw ft0, 0(a2)
RVV	002C	B2105457	vfmacc.vf v8, ft0, v1
RVV	0030	00462007	flw ft0, 4(a2)
RVV	0034	B21054D7	vfmacc.vf v9, ft0, v1
RVV	0038	00862007	flw ft0, 8(a2)
RVV	003C	B2105557	vfmacc.vf v10, ft0, v1
RVV	0040	00C62007	flw ft0, 12(a2)
RVV	0044	B21055D7	vfmacc.vf v11, ft0, v1
RVV	0048	01058593	addi a1, a1, 16
RVV	004C	01060613	addi a2, a2, 16
RVV	0050	FFF50513	addi a0, a0, -1
RVV	0054	FC0518E3	bnez a0, loop
RVV	0058	0206E427	vse32.v v8, (a3)
RVV	005C	01068293	addi t0, a3, 16
RVV	0060	0202E4A7	vse32.v v9, (t0)
RVV	0064	02068293	addi t0, a3, 32
RVV	0068	0202E527	vse32.v v10, (t0)
RVV	006C	03068293	addi t0, a3, 48
RVV	0070	0202E5A7	vse32.v v11, (t0)
RVV	0074	00008067	end: ret
Intel	0000	C5F81006	gemm32: vmovups xmm0, [rsi]
Intel	0004	C4E279180A	vbroadcastss xmm1, dword ptr [rdx]
Intel	0009	C4E279B8E1	vfmadd231ps xmm4, xmm0, xmm1
Intel	000E	C4E279184A04	vbroadcastss xmm1, dword ptr [rdx+4]
Intel	0014	C4E279B8E9	vfmadd231ps xmm5, xmm0, xmm1
Intel	0019	C4E279184A08	vbroadcastss xmm1, dword ptr [rdx+8]
Intel	001F	C4E279B8F1	vfmadd231ps xmm6, xmm0, xmm1
Intel	0024	C4E279184A0C	vbroadcastss xmm1, dword ptr [rdx+12]
Intel	002A	C4E279B8F9	vfmadd231ps xmm7, xmm0, xmm1
Intel	002F	4883C610	add rsi, 16
Intel	0033	4883C210	add rdx, 16
Intel	0037	4883E901	sub rcx, 1
Intel	003B	75C3	jnz gemm32
ARM	0000	4CDF7820	gemm32: ld1 {v0.4s}, [x1], #16
ARM	0004	4DDFC841	ld1r {v1.4s}, [x2], #4
ARM	0008	4E21CC04	fmla v4.4s, v0.4s, v1.4s
ARM	000C	4DDFC841	ld1r {v1.4s}, [x2], #4
ARM	0010	4E21CC05	fmla v5.4s, v0.4s, v1.4s
ARM	0014	4DDFC841	ld1r {v1.4s}, [x2], #4
ARM	0018	4E21CC06	fmla v6.4s, v0.4s, v1.4s
ARM	001C	4DDFC841	ld1r {v1.4s}, [x2], #4
ARM	0020	4E21CC07	fmla v7.4s, v0.4s, v1.4s
ARM	0024	F1000400	subs x0, x0, #1
ARM	0028	54FFFEC1	b.ne gemm32

kernels/bx32/gemm32.sew32.lmul1.tu.mu.vlen128.xlen32.le.bin

kernels/bx64/gemm64.sew64.lmul1.tu.mu.vlen128.xlen64.le.bin

Table 14.3: Kernel comparison: gemm64

a0 = K, a1 = A panel, a2 = B panel, a3 = C. 2-wide outer product.

ISA	Address	Hex	Assembly
RVV	0000	06050A63	gemm64: beqz a0, end
RVV	0004	00307057	vsetvli zero, zero, e64, m1, tu, mu
RVV	0008	0206F407	vle64.v v8, (a3)
RVV	000C	01068293	addi t0, a3, 16
RVV	0010	0202F487	vle64.v v9, (t0)
RVV	0014	02068293	addi t0, a3, 32
RVV	0018	0202F507	vle64.v v10, (t0)
RVV	001C	03068293	addi t0, a3, 48
RVV	0020	0202F587	vle64.v v11, (t0)
RVV	0024	0205F087	loop: vle64.v v1, (a1)
RVV	0028	00063007	fld ft0, 0(a2)
RVV	002C	B2105457	vfmacc.vf v8, ft0, v1
RVV	0030	00863007	fld ft0, 8(a2)
RVV	0034	B21054D7	vfmacc.vf v9, ft0, v1
RVV	0038	01063007	fld ft0, 16(a2)
RVV	003C	B2105557	vfmacc.vf v10, ft0, v1
RVV	0040	01863007	fld ft0, 24(a2)
RVV	0044	B21055D7	vfmacc.vf v11, ft0, v1
RVV	0048	01058593	addi a1, a1, 16
RVV	004C	02060613	addi a2, a2, 32
RVV	0050	FFF50513	addi a0, a0, -1
RVV	0054	FC0518E3	bnez a0, loop
RVV	0058	0206F427	vse64.v v8, (a3)
RVV	005C	01068293	addi t0, a3, 16
RVV	0060	0202F4A7	vse64.v v9, (t0)
RVV	0064	02068293	addi t0, a3, 32
RVV	0068	0202F527	vse64.v v10, (t0)
RVV	006C	03068293	addi t0, a3, 48
RVV	0070	0202F5A7	vse64.v v11, (t0)
RVV	0074	00008067	end: ret
Intel	0000	C5F91006	gemm64: vmovupd xmm0, [rsi]
Intel	0004	C5FB120A	vmovddup xmm1, qword ptr [rdx]
Intel	0008	C4E2F9B8E1	vfmadd231pd xmm4, xmm0, xmm1
Intel	000D	C5FB124A08	vmovddup xmm1, qword ptr [rdx+8]
Intel	0012	C4E2F9B8E9	vfmadd231pd xmm5, xmm0, xmm1
Intel	0017	C5FB124A10	vmovddup xmm1, qword ptr [rdx+16]
Intel	001C	C4E2F9B8F1	vfmadd231pd xmm6, xmm0, xmm1
Intel	0021	C5FB124A18	vmovddup xmm1, qword ptr [rdx+24]
Intel	0026	C4E2F9B8F9	vfmadd231pd xmm7, xmm0, xmm1
Intel	002B	4883C610	add rsi, 16
Intel	002F	4883C220	add rdx, 32
Intel	0033	4883E901	sub rcx, 1
Intel	0037	75C7	jnz gemm64
ARM	0000	4CDF7C20	gemm64: ld1 {v0.2d}, [x1], #16
ARM	0004	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0008	4E61CC04	fmla v4.2d, v0.2d, v1.2d
ARM	000C	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0010	4E61CC05	fmla v5.2d, v0.2d, v1.2d
ARM	0014	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0018	4E61CC06	fmla v6.2d, v0.2d, v1.2d
ARM	001C	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0020	4E61CC07	fmla v7.2d, v0.2d, v1.2d
ARM	0024	F1000400	subs x0, x0, #1
ARM	0028	54FFFEC1	b.ne gemm64

14.7 Harts

One RV64 hart is the default. An optional second hart is the BX-64 organisation: one hart for the short work, one for a longer kernel, a mapped local memory, and a DMA engine programmed by stores. Both harts see VLEN 128. The DMA does not fetch OP-V. Xsvm stays off on a V-claiming image.

Chapter 15

BX-512: A Pinned 512-bit Register

15.1 The board-level picture

BX-512 is the BX-64 board with the vector file pinned at 512 bits. It does not replace a console. Instruction fetch has its own 32-bit port. Data memory is a 64-bit port. The hart is RV64, Chapter 12. The vector file is thirty-two registers of 512 bits, 2048 bytes. Chapter 16 is the integer base with XLEN 128. This board keeps the RV64 hart.

A vector load at SEW 64 moves eight elements from one register. A vector load at SEW 32 moves sixteen. Lane counts, the two register files, and this pin beside an RV128 hart are Chapter 7.

XLEN = 64. ELEN = 64. VLEN = 512.
Thirty-two vector registers. VRF = 2048 bytes.
Instruction port 32 bits. Data port 64 bits.

15.2 The architecture

Legal SEW = 8, 16, 32, and 64.
VLMAX = 64, 32, 16, and 8 at those widths.
FP32 and FP64, including fused multiply-add.
If the claim is V: every instruction V requires, and the V bit set in `misa`.
If the claim is Zve64d: do not set the V bit.
SEW 512 traps.

`vsetvli` with `rs1` equal to `x0` writes VLMAX into `rd`. VLEN is not an instruction field. The acceptance filename records 512, which is the length this image was accepted on. The same parcel at VLEN 128 returns a smaller `vl`.

15.3 Widening

Widening multiply and narrowing shift are the BX-64 operations. They are legal here because ELEN is 64. A 512-bit register holds more elements of that SEW. In the matrix of Section 11.7 those elements are a taller column slice. LMUL groups registers. A group is still a vector.

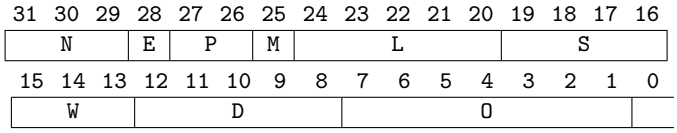
15.4 Opcode map for BX-512

The parcels are the BX-64 parcels. The diagrams below are the two fields this chapter uses. Integer, FP32, FP64, fused multiply-add, mask, and indexed forms are the map in Chapter 13.

15.4.1 Configuration

Identical to Chapter 9. `zimm` selects SEW and LMUL. Bit 31 stays clear. `vsetvli t0, zero, e32, m1, tu, mu` is the word `002072D7`. On this machine `t0` receives 16. That word is the configuration at the head of `wide32`.

15.4.2 Unit-stride load at SEW=64 — `vle64.v`



N = nf
E = mew
P = mop (00 = unit-stride)
M = vm
L = lumop (00000)
S = rs1 (base)
W = width (111 = 64-bit)
D = vd
O = 0000111

EEW 8, 16, and 32 keep the width codes already printed: 000, 101, and 110.

15.5 RV64 and COP3 fetch

Path	Escape	Address
RV64	native OP-V	full 64-bit GPR
COP3 host	`4C000000`	low 64 bits of the GPR

The RV64 parcel is little-endian. The COP3 escape is the BX-32 escape, and the parcel endian is the host's. COP2 stays the host's own unit. There is no harness directory for this machine.

15.6 Assembler kernels that must run

Every BX-64 acceptance image is required here. The tables stay in Chapter 13. `gemm32` uses the immediate offsets of a VLEN-128 tile. The acceptance product on this machine is `wide32` and `wide64`. The stride is `vl` after `vsetvli`, so the same parcels are legal at VLEN 128, and the filename records 512. The Intel column is `zmm`. The ARM column is AArch64 NEON.

`wide32` is the `sgemm` tile a `zmm` kernel asks for: oneDNN, a large BLIS kernel, and a training matmul, sixteen singles wide. `wide64` is the same product at eight doubles. The BX-32 catalogue still runs, and a wider register holds a taller column of the same panel.
`kernels/bx512/wide32.sew32.lmul1.tu.mu.vlen512.xlen64.le.bin`
`kernels/bx512/wide64.sew64.lmul1.tu.mu.vlen512.xlen64.le.bin`

Table 15.1: Kernel analogues on this machine.

Kernel	RVV	Intel	ARM
wide32	`vfmacc.vf`	`vfmadd231ps`	`fmla`
wide64	`vfmacc.vf`	`vfmadd231pd`	`fmla`

Table 15.2: Kernel comparison: wide32

`a0 = K, a1 = A, a2 = B, a3 = C. 4 columns, stride = vl.`

ISA	Address	Hex	Assembly
RVV	0000	06050C63	`wide32: beqz a0, end`
RVV	0004	002072D7	`vsetvli t0, zero, e32, m1, tu, mu`
RVV	0008	00229313	`slli t1, t0, 2`
RVV	000C	0206E407	`vle32.v v8, (a3)`
RVV	0010	006682B3	`add t0, a3, t1`
RVV	0014	0202E487	`vle32.v v9, (t0)`
RVV	0018	006282B3	`add t0, t0, t1`
RVV	001C	0202E507	`vle32.v v10, (t0)`
RVV	0020	006282B3	`add t0, t0, t1`
RVV	0024	0202E587	`vle32.v v11, (t0)`
RVV	0028	0205E087	`loop: vle32.v v1, (a1)`
RVV	002C	00062007	`flw ft0, 0(a2)`
RVV	0030	B2105457	`vfmacc.vf v8, ft0, v1`
RVV	0034	00462007	`flw ft0, 4(a2)`
RVV	0038	B21054D7	`vfmacc.vf v9, ft0, v1`
RVV	003C	00862007	`flw ft0, 8(a2)`
RVV	0040	B2105557	`vfmacc.vf v10, ft0, v1`
RVV	0044	00C62007	`flw ft0, 12(a2)`
RVV	0048	B21055D7	`vfmacc.vf v11, ft0, v1`
RVV	004C	006585B3	`add a1, a1, t1`
RVV	0050	01060613	`addi a2, a2, 16`
RVV	0054	FFF50513	`addi a0, a0, -1`
RVV	0058	FC0518E3	`bnez a0, loop`
RVV	005C	0206E427	`vse32.v v8, (a3)`
RVV	0060	006682B3	`add t0, a3, t1`
RVV	0064	0202E4A7	`vse32.v v9, (t0)`
RVV	0068	006282B3	`add t0, t0, t1`
RVV	006C	0202E527	`vse32.v v10, (t0)`
RVV	0070	006282B3	`add t0, t0, t1`
RVV	0074	0202E5A7	`vse32.v v11, (t0)`
RVV	0078	00008067	`end: ret`
Intel	0000	62F17C481006	`wide32: vmovups zmm0, [rsi]`
Intel	0006	62F27D48180A	`vbroadcastss zmm1, dword ptr [rdx]`
Intel	000C	62F27D48B8E1	`vfmadd231ps zmm4, zmm0, zmm1`
Intel	0012	62F27D48184A01	`vbroadcastss zmm1, dword ptr [rdx+4]`
Intel	0019	62F27D48B8E9	`vfmadd231ps zmm5, zmm0, zmm1`
Intel	001F	62F27D48184A02	`vbroadcastss zmm1, dword ptr [rdx+8]`
Intel	0026	62F27D48B8F1	`vfmadd231ps zmm6, zmm0, zmm1`
Intel	002C	62F27D48184A03	`vbroadcastss zmm1, dword ptr [rdx+12]`
Intel	0033	62F27D48B8F9	`vfmadd231ps zmm7, zmm0, zmm1`
Intel	0039	4883C640	`add rsi, 64`
Intel	003D	4883C210	`add rdx, 16`
Intel	0041	4883E901	`sub rcx, 1`
Intel	0045	75B9	`jnz wide32`
ARM	0000	4CDF7820	`wide32: ld1 {v0.4s}, [x1], #16`
ARM	0004	4DDFC841	`ld1r {v1.4s}, [x2], #4`
ARM	0008	4E21CC04	`fmla v4.4s, v0.4s, v1.4s`
ARM	000C	4DDFC841	`ld1r {v1.4s}, [x2], #4`
ARM	0010	4E21CC05	`fmla v5.4s, v0.4s, v1.4s`
ARM	0014	4DDFC841	`ld1r {v1.4s}, [x2], #4`
ARM	0018	4E21CC06	`fmla v6.4s, v0.4s, v1.4s`
ARM	001C	4DDFC841	`ld1r {v1.4s}, [x2], #4`
ARM	0020	4E21CC07	`fmla v7.4s, v0.4s, v1.4s`
ARM	0024	F1000400	`subs x0, x0, #1`
ARM	0028	54FFFEC1	`b.ne wide32`

Table 15.3: Kernel comparison: wide64

a0 = K, a1 = A, a2 = B, a3 = C. 4 columns, stride = vl.

ISA	Address	Hex	Assembly
RVV	0000	06050C63	wide64: beqz a0, end
RVV	0004	003072D7	vsetvli t0, zero, e64, m1, tu, mu
RVV	0008	00329313	slli t1, t0, 3
RVV	000C	0206F407	vle64.v v8, (a3)
RVV	0010	006682B3	add t0, a3, t1
RVV	0014	0202F487	vle64.v v9, (t0)
RVV	0018	006282B3	add t0, t0, t1
RVV	001C	0202F507	vle64.v v10, (t0)
RVV	0020	006282B3	add t0, t0, t1
RVV	0024	0202F587	vle64.v v11, (t0)
RVV	0028	0205F087	loop: vle64.v v1, (a1)
RVV	002C	00063007	fld ft0, 0(a2)
RVV	0030	B2105457	vfmacc.vf v8, ft0, v1
RVV	0034	00863007	fld ft0, 8(a2)
RVV	0038	B21054D7	vfmacc.vf v9, ft0, v1
RVV	003C	01063007	fld ft0, 16(a2)
RVV	0040	B2105557	vfmacc.vf v10, ft0, v1
RVV	0044	01863007	fld ft0, 24(a2)
RVV	0048	B21055D7	vfmacc.vf v11, ft0, v1
RVV	004C	006585B3	add a1, a1, t1
RVV	0050	02060613	addi a2, a2, 32
RVV	0054	FFF50513	addi a0, a0, -1
RVV	0058	FC0518E3	bnez a0, loop
RVV	005C	0206F427	vse64.v v8, (a3)
RVV	0060	006682B3	add t0, a3, t1
RVV	0064	0202F4A7	vse64.v v9, (t0)
RVV	0068	006282B3	add t0, t0, t1
RVV	006C	0202F527	vse64.v v10, (t0)
RVV	0070	006282B3	add t0, t0, t1
RVV	0074	0202F5A7	vse64.v v11, (t0)
RVV	0078	00008067	end: ret
Intel	0000	62F1FD481006	wide64: vmovupd zmm0, [rsi]
Intel	0006	62F2FD48190A	vbroadcastsd zmm1, qword ptr [rdx]
Intel	000C	62F2FD48B8E1	vfmadd231pd zmm4, zmm0, zmm1
Intel	0012	62F2FD48194A01	vbroadcastsd zmm1, qword ptr [rdx+8]
Intel	0019	62F2FD48B8E9	vfmadd231pd zmm5, zmm0, zmm1
Intel	001F	62F2FD48194A02	vbroadcastsd zmm1, qword ptr [rdx+16]
Intel	0026	62F2FD48B8F1	vfmadd231pd zmm6, zmm0, zmm1
Intel	002C	62F2FD48194A03	vbroadcastsd zmm1, qword ptr [rdx+24]
Intel	0033	62F2FD48B8F9	vfmadd231pd zmm7, zmm0, zmm1
Intel	0039	4883C640	add rsi, 64
Intel	003D	4883C220	add rdx, 32
Intel	0041	4883E901	sub rcx, 1
Intel	0045	75B9	jnz wide64
ARM	0000	4CDF7C20	wide64: ld1 {v0.2d}, [x1], #16
ARM	0004	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0008	4E61CC04	fmla v4.2d, v0.2d, v1.2d
ARM	000C	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0010	4E61CC05	fmla v5.2d, v0.2d, v1.2d
ARM	0014	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0018	4E61CC06	fmla v6.2d, v0.2d, v1.2d
ARM	001C	4DDFCC41	ld1r {v1.2d}, [x2], #8
ARM	0020	4E61CC07	fmla v7.2d, v0.2d, v1.2d
ARM	0024	F1000400	subs x0, x0, #1
ARM	0028	54FFFEC1	b.ne wide64

15.7 Harts

One RV64 hart is the default. An optional second hart is the BX-64 organisation: one hart for the short work, one for a longer kernel, a mapped local memory, and a DMA engine programmed by stores. Both harts see VLEN 512. The DMA does not fetch OP-V. Xsvm stays off on a V-claiming image.

Chapter 16

RV128I: A 128-bit Address

16.1 The board-level picture

RV128I is an integer base with a flat 128-bit address and an integer register of 128 bits. Version 1.7 of that chapter says the design is not frozen. The reason it gives for the width is address space. BX-128, Chapter 14, pins VLEN at 128 and keeps an RV64 hart. This chapter is the integer register.

Chapters 8 and 12 open the 32-bit and 64-bit integer files. BX-8, BX-16, and BX-32 follow the first. BX-64, BX-128, and BX-512 follow the second, and they remain on that hart: the 128 and 512 in their names are VLEN. This chapter opens the RV128 file. The BX chapters end with BX-512. A vector unit attached here is the BX-128 file or the BX-512 file, at the lane counts of Chapter 7. The vector register stays at 128 or 512 bits.

The board is not a console and it does not replace one. Instruction fetch is a 32-bit port: the parcel stays 32 bits. The data port is an implementation choice. A 128-bit port retires a quadword in one beat. A 64-bit port takes two beats, low address first, little-endian. The architectural result is the same either way, which is the lesson BX-8 already teaches for a word on an eight-bit bus.

Thirty-two integer registers, each 128 bits. There is no vector file in the base. A vector unit attached later is one of the profiles in this book: ELEN 64, VLEN whatever that profile pins, thirty-two vector registers. The integer register does not become a vector register.

RV128I, version 1.7. That text says it is not frozen.
XLEN = 128. Thirty-two integer registers.
Instruction port 32 bits. Data port 64 or 128 bits.
No vector file in the base. ELEN stays 64 if one is attached.

16.2 The architecture

Builds on RV64I. Most integer operations read and write XLEN bits.
The RV64 `*W` operations stay, and sign-extend from bit 31 to bit 127.
The `*D` operations read the low 64 bits and sign-extend from bit 63 to bit 127.
`LDU` zero-extends a doubleword. `LQ` and `SQ` move a quadword.
A shift immediate uses the low 7 bits of the I-immediate.
A shift by a register uses the low 7 bits of the source register.
Floating-point is unchanged. SEW 128 traps.

The Q extension, if you implement it, can add `FMV.X.Q`, `FMV.Q.X`, and conversions to and from a 128-bit integer. This book does not encode those words. RVV 1.0 still stops at an element of 64 bits. Attaching BX-128 or BX-512 to this hart changes VLEN and leaves ELEN at 64.

The unfrozen text also records a commentary, not a rule: the 64-bit add might later be renamed and the 128-bit add might take the old `OP-64` slot. The kernels below keep `addi` as the XLEN add.

16.3 Widening

`addiw` and `addw` still operate on the low 32 bits. The result is sign-extended through bit 127, where RV64 stopped the extension at bit 63. `addid` and `addd` operate on the low 64 bits and sign-extend bit 63 through bit 127. `addi` adds across all 128 bits, so a carry out of bit 63 is written into bit 64.

A shift immediate of 4 is the RV64 parcel: the extra shamt bit is clear. A shift immediate of 64 sets that bit. An RV64 decoder rejects the word. A variable shift uses the same R-type parcel as RV64; the amount is simply seven bits of the source register instead of six.

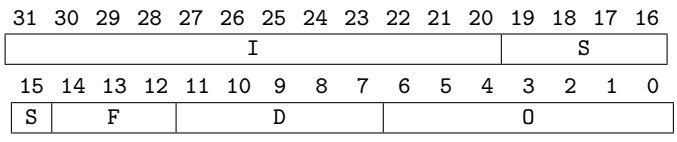
`ld` still sign-extends a doubleword into the 128-bit register. `ldu` zero-extends it. `lq` and `sq` move all 128 bits. The acceptance kernels use addresses aligned to the access: 8 bytes for `ldu`, 16 bytes for `lq` and `sq`.

16.4 Opcode map for RV128I

The diagrams in this section are the fields the kernels use. Two of them are fixed by the unfrozen chapter together with the ordinary I-type and S-type layouts: `slli` puts the amount in the low 7 bits of the I-immediate, and `addi` is the existing OP-IMM word. The funct3 values of `ldu`, `lq`, and `sq`, and the major opcodes of the `*D` instructions, are the placement in QEMU's `insn32.decode`. The chapter names the major opcodes and does not draw those funct3 fields. A decoder that follows the sentences literally, and a decoder that follows QEMU, will not agree on `addid`. The hex in the kernel tables is the QEMU placement, and the prose says where the two readings part.

16.4.1 Load double unsigned — `ldu`

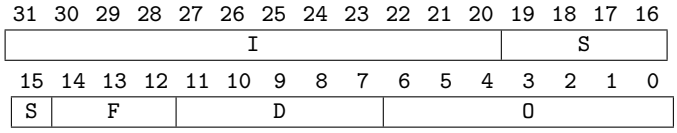
I-type. Major opcode LOAD, which is the chapter's assignment. funct3 is 111, the LOAD width QEMU decodes and the width the other LOAD funct3 values leave open. `ldu t0, 0(a0)` is the word `00057283`.



I = imm[11:0]
S = rs1
F = 111
D = rd
O = 0000011

16.4.2 Load quadword — `lq`

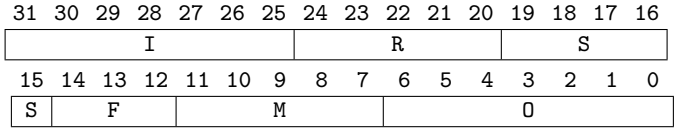
I-type. The chapter puts `LQ` on MISC-MEM, opcode `0001111`. QEMU uses funct3 010, and it rejects the word when rd is `x0`, because that pattern is the cache-block operation. The kernels write `t0`. `lq t0, 0(a0)` is the word `0005228F`.



I = imm[11:0]
S = rs1
F = 010
D = rd, not `x0`
O = 0001111

16.4.3 Store quadword — `sq`

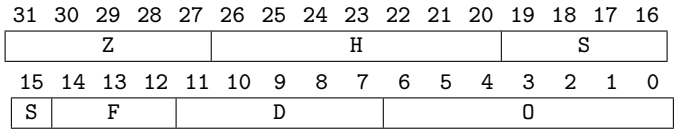
S-type. The chapter puts `SQ` on STORE. QEMU uses funct3 100. The immediate is split, as in every store. `sq t0, 0(a1)` is the word `0055C023`.



I = imm[11:5]
R = rs2 (the data)
S = rs1 (the base)
F = 100
M = imm[4:0]
O = 0100011

16.4.4 Shift immediate

OP-IMM, funct3 001 for a left shift, opcode `0010011`. Bits 26:20 hold the 7-bit amount. Bits 31:27 stay clear, which is how QEMU recognises the word. `slli a0, a0, 4` is `00451513`, the same parcel RV64 emits. `slli a0, a0, 64` is `04051513`. The kernel uses `t0` and the word `04029293`.



Z = 00000
H = shamt[6:0]
S = rs1
F = 001 for `slli`, 101 for a right shift
D = rd
O = 0010011

A right shift sets bit 30 when it is arithmetic, and keeps bits 31 and 29:27 clear. The amount still occupies bits 26:20.

16.4.5 The *D operations

The chapter says these instructions consume OP-IMM-64 and OP-64. Those major opcodes are 0011011 and 0111011. RV64 already uses them for addiw and addw, and the same chapter retains the *W set, so the two sentences do not name a free opcode. QEMU leaves *W on 0011011 and 0111011 and places *D on 1011011 and 1111011. The table is that decoder. addid t0, t0, 16 in the kernel is 010282DB. An implementation that reuses OP-IMM-64 will not match the image.

Mnemonic	Shape	Opcode in the text	QEMU placement
addid	I	0011011	1011011, funct3 000
sllid	I, shamt 6	0011011	1011011, funct3 001, bits 31:26 = 000000
srlid	I, shamt 6	0011011	1011011, funct3 101, bits 31:26 = 000000
sraid	I, shamt 6	0011011	1011011, funct3 101, bits 31:26 = 010000
addd	R	0111011	1111011, funct3 000, funct7 0000000
subd	R	0111011	1111011, funct3 000, funct7 0100000
slld	R	0111011	1111011, funct3 001, funct7 0000000
srld	R	0111011	1111011, funct3 101, funct7 0000000
srad	R	0111011	1111011, funct3 101, funct7 0100000

QEMU also decodes muld, divd, divud, remd, and remud on 1111011 with funct7 0000001. The fetched RV128I chapter does not list that multiply set. This book does not encode it. Only addid appears in an acceptance image.

addi a0, a0, 16 is 01050513. That word is the ordinary OP-IMM parcel, and on this base the add is 128 bits wide. addi t0, t0, 16 is 01028293.

16.4.6 Compressed parcels

The C extension can sit on RV32, RV64, or RV128. A few quadrants change meaning with the base. On RV128 the slot used for C.FLD becomes C.LQ, and the slot used for C.FSD becomes C.SQ. The stack forms are C.LQSP and C.SQSP. The offset is scaled by 16. C.LQ expands to lq rd', offset(rs1'). C.LQSP expands to lq rd, offset(x2), and the form with rd equal to x0 is reserved. C.LD and C.SD stay in their own slots.

C.SLLI64, C.SRLI64, and C.SRAI64 are shifts by 64 on RV128. On RV32 and RV64 those code points are hints. The acceptance images contain none of these 16-bit parcels. Every word in the tables is 32 bits.

16.5 Native fetch

Path	Escape	Address
RV128	native parcel	full 128-bit GPR

The parcel is little-endian. There is no escape and there is no harness directory. A host injection story does not apply: the address the base exists to provide is already in the GPR. COP3, the R800 ED prefix, the 68000 F-line, and the SuperH 0000 prefix stay on the machines that define them.

16.6 Assembler kernels that must run

Five images. Each one is a raw little-endian parcel stream. The filename records `xlen128` and `vlen0`: the image configures no `vtype`, and it does not pin VLEN. `sew64` in that name is the filename slot. It is the widest element a later vector unit may use. It is not an instruction field, and it is not SEW 128.

The first column of each table is RV128. Intel is x86-64, assembled by clang with AVX2 and without AVX-512. ARM is AArch64. Both of those machines have 64-bit general registers. `quad` moves 16 bytes through an `xmm` and through a NEON register. The other four strips write the high half of the quadword with a second store: a carry, a sign fill, a zero, or the shifted half. SVE is not in these bytes.

Table 16.1: Kernel analogues on this base.

Kernel	RV128	Intel	ARM
quad	`lq`, `sq`	`movdqu`	`ld1`
add128	`addi`	`add`, `adc`	`adds`, `adc`
addd16	`addid`	`add`, `sar`	`add`, `asr`
zext64	`ldu`	`mov`	`ldr`
sh64	`slli`	`mov`	`ldr`

`quad` copies one aligned quadword and then adds 16 to both pointers with `addi`. `add128` adds 16 across the whole register, so a carry out of bit 63 is kept. `addd16` adds 16 to the low 64 bits and sign-extends bit 63; the high half of the loaded value is discarded, and the `addid` word is the QEMU opcode. `zext64` zero-extends a doubleword into a stored quadword. `sh64` shifts a quadword left by 64. The bits that leave the top are gone, and the low half becomes the high half.

Table 16.2: Programs that run the RV128 kernels.

Kernel	Work	Where it ships
quad	16-byte move	One AES block of payload, the move. One IPv6 address. A 16-byte spill of a wide register.
add128	quadword plus 16	A 128-bit cursor stepped by one block. A limb counter that carries into bit 64.
addd16	low 64 bits plus 16	A 64-bit length kept in the low half of the register.
zext64	zero-extend	A 64-bit file size loaded into a 128-bit address register.
sh64	shift by one limb	The low half placed into the high half. One limb of a two-limb shift.

```
kernels/rv128/quad.sew64.lmul1.tu.mu.vlen0.xlen128.le.bin
kernels/rv128/add128.sew64.lmul1.tu.mu.vlen0.xlen128.le.bin
kernels/rv128/addd16.sew64.lmul1.tu.mu.vlen0.xlen128.le.bin
kernels/rv128/zext64.sew64.lmul1.tu.mu.vlen0.xlen128.le.bin
kernels/rv128/sh64.sew64.lmul1.tu.mu.vlen0.xlen128.le.bin
```

16.7 Harts

One RV128 hart is the default. A second hart, if the elaboration has one, is another RV128 integer hart. A vector coprocessor beside either hart is BX-128 or BX-512: ELEN 64, VLEN pinned by that chapter, SEW 128 illegal. The coprocessor does not widen the integer register. DMA may move quadwords. It does not fetch parcels, and it does not decode `lq`. Xsvm stays off. There is no host escape on this image.

Table 16.3: Kernel comparison: quad

a0 = src, a1 = dst. One quadword, then both pointers add 16.

ISA	Address	Hex	Assembly
RV128	0000	0005228F	quad: lq t0, 0(a0)
RV128	0004	0055C023	sq t0, 0(a1)
RV128	0008	01050513	addi a0, a0, 16
RV128	000C	01058593	addi a1, a1, 16
RV128	0010	00008067	ret
Intel	0000	F30F6F06	quad: movdqu xmm0, [rsi]
Intel	0004	F30F7F07	movdqu [rdi], xmm0
Intel	0008	4883C610	add rsi, 16
Intel	000C	4883C710	add rdi, 16
Intel	0010	C3	ret
ARM	0000	4C407000	quad: ld1 {v0.16b}, [x0]
ARM	0004	4C007020	st1 {v0.16b}, [x1]
ARM	0008	91004000	add x0, x0, #16
ARM	000C	91004021	add x1, x1, #16
ARM	0010	D65F03C0	ret

Table 16.4: Kernel comparison: add128

a0 = src, a1 = dst. Quadword plus 16. Carry enters bit 64.

ISA	Address	Hex	Assembly
RV128	0000	0005228F	add128: lq t0, 0(a0)
RV128	0004	01028293	addi t0, t0, 16
RV128	0008	0055C023	sq t0, 0(a1)
RV128	000C	00008067	ret
Intel	0000	488B06	add128: mov rax, qword ptr [rsi]
Intel	0003	488B5608	mov rdx, qword ptr [rsi+8]
Intel	0007	4883C010	add rax, 16
Intel	000B	4883D200	adc rdx, 0
Intel	000F	488907	mov qword ptr [rdi], rax
Intel	0012	48895708	mov qword ptr [rdi+8], rdx
Intel	0016	C3	ret
ARM	0000	F9400002	add128: ldr x2, [x0]
ARM	0004	F9400403	ldr x3, [x0, #8]
ARM	0008	B1004042	adds x2, x2, #16
ARM	000C	9A1F0063	adc x3, x3, xzr
ARM	0010	F9000022	str x2, [x1]
ARM	0014	F9000423	str x3, [x1, #8]
ARM	0018	D65F03C0	ret

Table 16.5: Kernel comparison: addd16

a0 = src, a1 = dst. Low 64 bits plus 16, then sign-extend.

ISA	Address	Hex	Assembly
RV128	0000	0005228F	addd16: lq t0, 0(a0)
RV128	0004	010282DB	addid t0, t0, 16
RV128	0008	0055C023	sq t0, 0(a1)
RV128	000C	00008067	ret
Intel	0000	488B06	addd16: mov rax, qword ptr [rsi]
Intel	0003	4883C010	add rax, 16
Intel	0007	488907	mov qword ptr [rdi], rax
Intel	000A	48C1F83F	sar rax, 63
Intel	000E	48894708	mov qword ptr [rdi+8], rax
Intel	0012	C3	ret
ARM	0000	F9400002	addd16: ldr x2, [x0]
ARM	0004	91004042	add x2, x2, #16
ARM	0008	F9000022	str x2, [x1]
ARM	000C	937FFC43	asr x3, x2, #63
ARM	0010	F9000423	str x3, [x1, #8]
ARM	0014	D65F03C0	ret

Table 16.6: Kernel comparison: zext64

`a0 = doubleword, a1 = quadword.  High 64 bits of the store are 0.`

ISA	Address	Hex	Assembly
RV128	0000	00057283	zext64: ldu t0, 0(a0)
RV128	0004	0055C023	sq t0, 0(a1)
RV128	0008	00008067	ret
Intel	0000	488B06	zext64: mov rax, qword ptr [rsi]
Intel	0003	488907	mov qword ptr [rdi], rax
Intel	0006	31D2	xor edx, edx
Intel	0008	48895708	mov qword ptr [rdi+8], rdx
Intel	000C	C3	ret
ARM	0000	F9400002	zext64: ldr x2, [x0]
ARM	0004	F9000022	str x2, [x1]
ARM	0008	F900043F	str xzr, [x1, #8]
ARM	000C	D65F03C0	ret

Table 16.7: Kernel comparison: sh64

`a0 = src, a1 = dst.  Quadword shifted left by 64.`

ISA	Address	Hex	Assembly
RV128	0000	0005228F	sh64: lq t0, 0(a0)
RV128	0004	04029293	slli t0, t0, 64
RV128	0008	0055C023	sq t0, 0(a1)
RV128	000C	00008067	ret
Intel	0000	488B06	sh64: mov rax, qword ptr [rsi]
Intel	0003	31D2	xor edx, edx
Intel	0005	488917	mov qword ptr [rdi], rdx
Intel	0008	48894708	mov qword ptr [rdi+8], rax
Intel	000C	C3	ret
ARM	0000	F9400002	sh64: ldr x2, [x0]
ARM	0004	F900003F	str xzr, [x1]
ARM	0008	F9000422	str x2, [x1, #8]
ARM	000C	D65F03C0	ret

Chapter 17

One Bluespec Description

17.1 Complete opcode contract

After the machines the architectural rectangle is fixed. The table is the single entry point for "which opcodes exist on which core".

Class / feature	SVM-8 (BX-8)	SVM-16 (BX-16)	Zve32f (BX-32)	Zve64d/V (BX-64)
`vsetvl/vsetvli`	yes	yes	yes	yes
`vsetivli`	opt.	yes	yes	yes
Unit-stride ld/st	EEW=8	8,16	8–32	8–64
Constant-stride	—	yes	yes	yes
Indexed ld/st	—	—	yes	yes
Segment ld/st	no	no	no	no
Integer arith (add/logic/shift)	yes	yes	yes	yes
min/max	—	yes	yes	yes
Compare + mask logicals	yes	yes	yes	yes
Saturating add/sub	vsadd only	yes	yes	yes
Widening multiply / narrow	—	yes	yes	yes
Reduction / slide1	—	yes	yes	yes
Gather / compress	—	—	yes	yes
Moves / splat (`vmv.*`)	yes	yes	yes	yes
FP32 vector + FMA	—	—	yes	yes
FP64 vector + FMA	—	—	—	yes
Division / sqrt / estimates	no	no	opt.	per claim
LMUL > 1	no	widen only	yes	yes
Xsvm (custom-0)	no	opt.	opt.	off on V
Tail policy	und.	und.	und.	und.

BX-128 and BX-512 use the Zve64d/V column. The pin is the register length.

	BX-128	BX-512
VLEN	128	512
VRF, bytes	512	2048
VLMAX at SEW 64	2	8

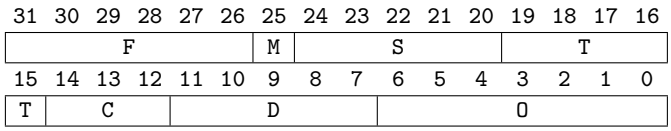
The bit-field diagrams that appear in Chapters 9–15 are the concrete encodings for each class. The tables below list every mnemonic the corresponding profile must implement. The Bluespec package that follows elaborates exactly the cells marked “yes” (or “opt.” when the corresponding parameter is true) and turns every other cell into an illegal-instruction path.

17.2 Aggregated opcode tables (ISA)

Canonical bit layouts are shown once, width-aligned. Profile columns then list only the mnemonics (and the few constants that differ). Detailed per-machine diagrams remain in Chapters 9–15; this section is the compact academic summary.

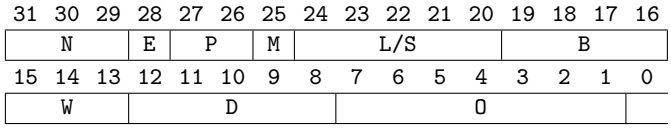
17.2.1 Canonical layouts (32-bit, two equal-width half-words)

OP-V arithmetic / compare / move / FP



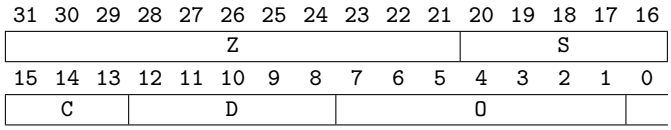
F = funct6 M = vm
S = vs2 T = vs1 / rs1 / fs1
C = funct3 D = vd
O = 1010111 (OP-V)

Unit-stride / stride / indexed load (store substitutes opcode 0100111)



N = nf E = mew
P = mop (00 unit, 10 stride, 01/11 indexed) M = vm
L/S = lumop or vs2 (index) B = rs1 (base)
W = width (000=8, 101=16, 110=32, 111=64) D = vd / vs3
O = 0000111 (load)

Configuration (`vsetvli`)



Z = zimm[10:0] S = rs1
C = 111 D = rd
O = 1010111

17.2.2 Mnemonic coverage by profile

Group	SVM-8	SVM-16	Zve32f	Zve64d/V
Configuration	`vsetvl/i`	+ `vsetivli`	same	same
Unit-stride	e8	e8,e16	e8–e32	e8–e64
Stride	—	e8,e16	e8–e32	e8–e64
Indexed	—	—	e8–e32	e8–e64
Integer arith	add/sub/logic/shift	+min/max	same	+SEW64
Saturating	vsadd	full set	full	full
Widening / narrow	—	vwmul, vnclip…	same	same
Compare + mask	full	full	full	full
Reduction / slide1	—	yes	yes	yes
Gather / compress	—	—	yes	yes
Moves / splat	`vmv.*`	same	+FP moves	same
FP32 + FMA	—	—	full	full
FP64 + FMA	—	—	—	full
Xsvm	no	opt.	opt.	off on V

17.2.3 Explicit summary — SVM-8 (BX-8)

Required	Notes
`vsetvl`, `vsetvli`	`vsetivli` optional
`vle8.v`, `vse8.v`	unit-stride only
`vadd/sub.vv/vx`, `vrsub.vx`	
`vand/vor/vxor.vv/vx`	
`vsll/vsrl/vsra.vv/vx`	
`vsadd.vv/vx`	`vssub` optional
`vmseq/ne/ltu/lt/leu/le.vv/vx`, `vmsgtu/gt.vx`	
mask logicals, `vcpop.m`, `vfirst.m`	
`vmv.v.v/x/i`, `vmv.x.s`, `vmv.s.x`	

17.2.4 Explicit summary — SVM-16 (BX-16)

Everything in SVM-8, plus:

Required	Notes
`vsetivli`	now required
`vle16.v`, `vse16.v`	
`vlse8/16.v`, `vsse8/16.v`	constant-stride
`vminu/min/maxu/max.vv/vx`	
`vsaddu/vsadd/vssubu/vssub`	full saturating set
`vwmul/u/su.vv/vx`	widening
`vnsrl/vnsra`, `vnclipu/vnclip`	narrowing
`vredsum.vs`	reduction
`vslide1up.vx`, `vslide1down.vx`	
Xsvm matrix / cross-product	optional (custom-0)

17.2.5 Explicit summary — Zve32f (BX-32)

Everything in SVM-16, plus:

Required	Notes
`vle32.v`, `vse32.v`	
`vluxei*`, `vloxei*`, indexed stores	
`vrgather.vv/vx`, `vcompress.vm`	
`vfadd/sub/mul.vv/vf`	FP32
`vfmacc/fnmacc/fmsac/fnmsac`	FMA family
`vfmadd/fnmadd/fmsub/fnmsub`	
`vfmin/max`, `vfsqrt`, `vfsgnj*`	
`vfcvt.*`, `vmfeq/ne/lt/le/gt/ge`	
`vfmv.v.f`, `vfmv.f.s`, `vfmv.s.f`	

17.2.6 Explicit summary — Zve64d / V (BX-64)

Everything in Zve32f, plus SEW = 64 and FP64. If the image claims full V, the entire V instruction set is required and the V bit is set in `misa`; otherwise the claim is Zve64d only. Xsvm must be off on any V-claiming image.

17.2.7 Key constants (funct6 / width / mop)

Operation	Constant	Value
`vadd`	funct6	000000
`vsub`	funct6	000010
`vand/vor/vxor`	funct6	001001/10/11
`vsll/vsrl/vsra`	funct6	100101/101000/101001
`vsadd`	funct6	100001
`vwmul`	funct6	111011
`vfmacc`	funct6	101100
width 8/16/32/64	W field	000/101/110/111
mop unit/stride/idx	P field	00/10/01 or 11

Any mnemonic required by a wider profile but absent from a narrower one is an illegal-instruction trap on the narrower core. The bit layouts above are identical on every machine; only the legal SEW, the legal mop values, and the legal funct6 set change.

17.3 Parameters

The vector unit is one Bluespec package. The profile is an elaboration parameter, not a source fork.

```
typedef struct {
  Integer elen;
  Integer vlen;
  Integer lanes;
  Bool    fp32;
  Bool    fp64;
  Bool    indexed;
  Bool    xsvm;
  Bool    claimV;
} VecParams deriving (Bits, Eq);
```

Legal combinations are exactly the four rows of the profile card. A function `paramsOk` returns false for every other combination, and the top-level module refuses to elaborate if it returns false. In particular `fp32` is false when `elen < 32`, and `claimV` is false when `elen < 64` or `vlen < 128` or `fp64` is false.

17.4 Interfaces

```
interface VectorIssue;
  method Action req(Parcel p, Regs rs);
  method Bool ready;
endinterface

typedef struct {
  Bit#(8)  escape0;     // first byte or low byte of the escape
  Bit#(8)  escape1;     // second byte; 0 when the escape is a halfword of zero
  Bit#(32) escapeWord;  // full word when the host instruction is 32 bits
  Bool     bigEndian;   // high half of the parcel travels first
  Integer  indexBits;   // 3 on R800, 4 on 68000 and SuperH, 5 on MIPS
  Bit#(4)  excKind;     // 0 illegal, 1 F-line, 2 coprocessor-unusable claimed
} HostAdaptor deriving (Bits, Eq);

interface HostFetch;
  method Action escape(Bit#(32) word);
  method Action parcel(Bit#(32) opv);
  method ActionValue#(Bit#(5)) mapRs(Bit#(5) rs);
  method Bool stall;
  method Action illegal;
endinterface

interface Lane;
  method Action enq(Elem a, Elem b, Opcode op, Bool mask);
  method ActionValue#(Elem) deq;
endinterface
```

The issue method is called from the hart's execute stage for OP-V parcels. `ready` is false while an in-order vector instruction occupies the unit. Loads use a third interface, `VecMem`, whose `beat` method is granted by the same arbiter as the video master on BX-8 and BX-16.

`HostFetch` is the same port with a foreign instruction stream in front of it. `escape` recognises the word, `parcel` delivers the following OP-V word after the endian assembly named by `HostAdaptor`, and `mapRs` turns the `rs1` field into a host register index. `stall` holds the softcore until `VectorIssue.ready` is true. The record in `HostAdaptor` is the table in Chapter 1: R800, 68000, SH-2, SH-4, R3000A, VR4300, Emotion Engine. An RV elaboration does not instantiate `HostFetch`.

Scalar writeback of `vsetvl`'s destination register is a method on the hart, not a side path. The vector unit returns the granted `vl` in the result structure of `req`.

17.5 The configuration rule

One rule handles `vsetvl`, `vsetvli`, and, where the profile includes it, `vsetivli`. It computes the requested SEW and LMUL, compares them with `paramsOk` for this instruction, and either updates `vtype` and `vl` or sets `vill`. The rule is atomic, so a configuration cannot be observed half-written by a following vector instruction in the same schedule. Give the rule a higher urgency than the arithmetic rule so that a `vsetvl` decoded in cycle n is visible to an instruction decoded in cycle $n + 1$.

17.6 Arithmetic rules

Integer rules are always elaborated. Their bodies case on SEW and produce an illegal-instruction side effect when the SEW is outside the profile. That side effect is a second line of defence behind `vill`. The first line is the configuration rule. The second line catches an instruction that ignores `vtype`.

Floating-point rules are wrapped in `if (p.fp32)` blocks so that a BX-8 elaboration does not instantiate DSP blocks for a single-precision multiplier. The same pattern gates FP64, indexed address generation, and the Xsvm decoder.

17.7 What SVM-8 still contains

The SVM-8 elaboration still contains the decode for opcodes it will trap: the illegal path is a rule, and the rule is source text. That is intentional. A later profile turns the same decode into a data path by changing parameters, not by pasting a second decoder. Keep the trap path in the source even when you are sure the profile will never enable the opcode. The tests of Chapter 19 include an illegal SEW on every profile, and they need something in the design that raises it.

Chapter 18

Bring-Up, from `vsetvl` to a Running Kernel

18.1 Order

The order is the same on every profile. Do not start with the widest machine on the theory that it contains the others. Start with BX-8, where a mistake is a byte lane, and carry the passing tests upward.

1. Scalar RV32I: loads, stores, branches, and a UART or a memory-mapped trace port.
2. `vsetvl` of a legal SEW, reading back `vl` and `vtype`.
3. `vsetvl` of an illegal SEW, observing `vill` and an illegal-instruction trap on the next vector opcode.
4. One vector add of `vl` elements, checked against a scalar loop.
5. Unit-stride store and load of those elements, including `vl` shorter than a register.
6. A masked add, undisturbed tail, checked byte by byte or halfword by halfword.
7. Only then the extra instructions of the profile: widening on BX-16, FP32 on BX-32, FP64 on BX-64.

18.2 The first vector add

Use SEW equal to ELEN and LMUL equal to 1. Fill two registers by stores to memory and a unit-stride load, or, before loads work, by a debug write to the file that is not part of the architecture and is compiled out of the board image. Prefer the load as soon as it exists, so that the debug write does not become a second programming model.

Compare the destination against the sum computed on the host in the testbench. Do this for `vl` from 0 through VLMAX. `vl` of 0 must retire no elements and must not raise a trap. It is the usual end condition of a strip-mined loop, and implementations get it wrong when the lane enable is a decrement that underflows.

18.3 Bus contention on the compatible machines

On BX-8, repeat the fill while the video processor is fetching a scanline. The architectural result must match the quiet-bus result. The cycle count may be longer. On BX-16, repeat a sixteen-bit fill while the audio master takes its reserved slots. A test that passes only with the other masters held in reset has not shown that the machine can replace an MSX or an Amiga. It has shown that the arithmetic works.

18.4 When to leave the simulator

Leave the simulator after the fill test and the illegal-SEW test pass on the profile you intend to synthesise. Synthesis of a vector unit that fails `vl`=0 is a long way to learn a one-line bug. The budget chapter assumes the tests are already green in simulation.

Chapter 19

Tests That Travel Up the Ladder

19.1 One suite

The architectural tests are a single directory. Each test names the narrowest profile that must run it. A wider profile runs it too. A narrower profile does not assemble it, or assembles it and expects an illegal-instruction trap, and the expected trap is itself a test.

Test	First profile	Also required on
`fill8`	SVM-8	all wider
`mask8`	SVM-8	all wider
`sew16`	SVM-16	every wider profile
`vwmul16`	SVM-16	every wider profile
`sad16`, `sw8`	SVM-16	every wider profile
`sew32`	Zve32f	BX-64 and above
`fma32`	Zve32f	BX-64 and above
`gemm32`	Zve32f	BX-64 and above
`scal32`–`iamax32`	Zve32f	BX-64 and above
`wmul32`, `bfly32`, `cmul32`	Zve32f	BX-64 and above
`sew64`	Zve64d or V	BX-64, BX-128, BX-512
`fma64`	Zve64d or V	BX-64, BX-128, BX-512
`gemm64`	Zve64d or V	BX-64, BX-128, BX-512
`scal64`–`iamax64`	Zve64d or V	BX-64, BX-128, BX-512
`wide32`, `wide64`	BX-512	BX-512
`badsew`	each profile	trap on that profile

19.2 The fill

```
// n bytes at dst become the byte fill
// assembled once, linked into every image
void fill8(uint8_t *dst, uint8_t fill, size_t n);
```

The reference is a scalar loop in the testbench, not in the hart. The hart's result is read back through the bus with the vector unit idle, so that a buggy load unit cannot hide a buggy

store unit by reading its own write buffer. Compare all `n` bytes, including when `n` is one more than a multiple of VLMAX. That last iteration is the one that sets `vl` to a remainder.

19.3 Illegal SEW

On each image, execute `vsetvli` with a SEW the profile forbids: 16 on SVM-8, 32 on SVM-16, 64 on Zve32f, 128 on BX-128, 512 on BX-512. Then execute `vadd.vv`. The add must trap. `vl` must be unchanged from the previous legal configuration, or zero if there was none, according to the rule you wrote down for `vill`. The test checks the CSR, not only the trap, because a trap with a silently updated `vl` will break the next legal loop.

19.4 RV128 images

`quad`, `add128`, `addd16`, `zext64`, and `sh64` run on an RV128 hart. They are integer parcels, and they are outside the travelling vector suite. On BX-128, XLEN is 64, so `lq` and a shift immediate of 64 are illegal. That trap is the check that the VLEN pin and the integer base stayed apart.

19.5 Xsvm tests are separate

Tests that emit the custom matrix or cross-product opcode live in another directory and run only on elaborations with `xsvm` true. They are not part of the travelling suite. A clean BX-16 image must fail those tests by trapping, and the travelling suite must still pass.

19.6 Random parcels

After the directed tests, a small generator that emits legal OP-V for the profile, runs the parcel on the hart and on a scalar reference model, and compares the destination register, will find the lane-enable bugs that VLMAX and `vl`=1 do not catch. Bound it. A few thousand parcels per profile are enough for a teaching design. Record the seed.

Chapter 20

What Fits on the First FPGA

20.1 The file, in bytes

Thirty-two registers times VLEN/8 is the architectural file.

Machine	VLEN	File	First RAM style
BX-8	32	128 B	distributed
BX-16	128	512 B	distributed or one block
BX-32	128	512 B	block, replicated if ports demand
BX-64	256	1 KiB	block

These numbers are the vector file alone. The video memory of an MSX-compatible image, and the chip memory of an Amiga-compatible image, are larger and are the memories you should budget first. The vector file is not the part that will surprise the map.

20.2 DSP blocks

An eight-bit add does not need a DSP block. A sixteen-bit multiply does, or it needs a multi-cycle shift-add, which is acceptable on the first BX-16 image if the cycle count is published. A single-precision fused multiply-add wants the DSP column the FPGA actually has; read the primitive and match the lane to it instead of inferring a behavioural `*` and hoping. A double-precision unit is several primitives or a sequenced mantissa multiplier. That is why the recommended first BX-64 image has two lanes, not eight.

20.3 The first images

The first BX-8 image contains the peripheral set required by the compatibility checklist, one integer lane of SVM-8, and a UART. It does not contain Xsvm, a second hart, or a floating-point unit.

The first BX-16 image contains the sixteen-bit bus, the blitter-class master, video and audio masters, and four lanes of SVM-16 without Xsvm. Geometry, if you want it in hardware, is a second image.

The first BX-32 image contains one hart, VLEN 128, four integer lanes, and one FP32 pipeline if the device is small. The first BX-64 image contains one hart, VLEN 128, Zve64d rather than a rushed V, and two lanes.

20.4 Clocks

Use one clock for the hart, the vector unit, and the peripheral bus on BX-8 and BX-16. A separate pixel clock is allowed for the display output, with a dual-clock FIFO you can explain. Do not clock the vector lanes faster than the bus on the compatible machines. The bus is the limit, and a faster lane only builds a queue in front of the arbiter.

On BX-32 and BX-64 a faster core clock than the memory clock is reasonable. Cross it with an asynchronous or mesochronous FIFO whose depth is in the profile card, and test a vector load that drains it.

20.5 A finished first board

A first board is finished when three things are true. The travelling tests for that profile pass on the hardware, read back through the bus. The picture and the sound meet the compatibility checklist for BX-8 or BX-16, or the memory test meets the map you published for BX-32 or BX-64. The illegal-SEW test traps on the hardware, not only in simulation. Cycle counts may differ from the simulator if the arbiter is busier. Architectural results may not.

Profile Card

	SVM-8	SVM-16	Zve32f	V / Zve64d
Machine	BX-8	BX-16	BX-32	BX-64
Replaces	MSX	Amiga	—	—
XLEN	32	32	32	64
Datapath	8	16	32	64
ELEN	8	16	32	64
VLEN	32	128	≥ 128	≥ 128
EEW	8	8, 16	8–32	8–64
FP	—	—	FP32	FP32, FP64
`misa`.V	no	no	per spec	only if V
Xsvm	no	optional	optional	off on V image
Tail policy	und.	und.	und.	und.

Discovery on SVM-8 and SVM-16 is the custom CSR you name here, not the V bit.

Custom CSR address: _____
`vsetivli` on SVM-8: yes / no
Indexed load from: BX-32
`vstart` precise from: BX-64 with a page table
Lanes in the first image: 1, 4, 4, 2

Host	Escape	Endian	Chapter
RV32 / RV64	native OP-V	little	every RV image
R800	`ED 00`	little	BX-8
68000	`F400`	big	BX-16
SH-2	`0000`	big	BX-16
SH-4	`0000`	little	BX-16
R3000A	`4C000000`	little	BX-32
VR4300	`4C000000`	big	BX-32
Emotion Engine	`4C000000`	little	BX-32

	BX-128	BX-512
XLEN	64	64
ELEN	64	64
VLEN	128	512
Registers	32	32
VRF, bytes	512	2048
VLMAX at SEW 64	2	8

BX-128 and BX-512 fetch as the BX-64 RV64 image, or through the BX-32 COP3 pattern with the low 64 bits of the GPR. RV128I is not a row on this card. Its integer register is 128 bits, and the chapter is Chapter 16.

Host on this image: RV32 / RV64 / R800 / 68000 / SH-2 / SH-4 / R3000A / VR4300 / EE

Checklist against RVV 1.0

For each instruction you implement, the encoding matches the specification, the register numbers are the specification's, and an illegal SEW traps. This list is the minimum the travelling tests exercise. It is not the whole of V.

- Configuration: `vsetvl`, `vsetvli`, and `vsetivli` where the card includes it.
- `vill` on an illegal SEW or LMUL, without a surprising update of `vl`.
- Unit-stride load and store at every legal EEW.
- Stride load and store from SVM-16 upward.
- Indexed load and store from BX-32 upward.
- Integer add, subtract, logic, shift, vector-vector and vector-scalar.
- Compare and mask logicals where the card includes them.
- Saturating add where the card includes it, with `vxsat`.
- Widening multiply and narrowing shift on SVM-16, BX-32, and BX-64.
- Integer reduction and slide-by-one from SVM-16 upward.
- FP32 fused multiply-add and conversions on BX-32 and BX-64.
- FP64 fused multiply-add on BX-64.
- Undisturbed tail and inactive mask elements.
- `vl` = 0 retires nothing and does not trap.
- No claim of Zve32x on BX-8 or BX-16.
- No V bit unless every instruction required by V is present.

MSX and Amiga Compatibility Checklist

BX-8, in place of an MSX

- Eight-bit data bus shared by the hart's data port, the video display processor, and the sound generator.
- Slot and cartridge signals documented against the MSX reference you follow.
- Video modes you claim: register map, timing, and a captured frame compared to that reference.
- Sound: tone, noise, and envelope registers, and a captured waveform.
- Keyboard and joystick ports in the documented map.
- Video refresh can take the bus during a vector load; the picture remains stable; the load's bytes match a quiet-bus run.
- The hart is RV32I with SVM-8. Guest binaries of the original processor run only through software you have written for that purpose, and that software is named on the card.

BX-16, in place of an Amiga

- Sixteen-bit shared bus with hart, blitter-class master, video, and audio.
- A reserved-slot or priority rule for audio, with a measured worst-case wait.
- Display fetch can be driven from a programmed list.
- Blitter-class master, if present, is a peripheral, not a second instruction set.
- A vector fill of halfwords matches a scalar reference while audio slots are taken.
- The hart is RV32I with SVM-16. Xsvm is off on the compatibility image.

BX-32 and BX-64

These images are compatible with the RISC-V strings on their profile cards, and with the travelling tests. They are not MSX or Amiga replacements. Their checklist is the RVV checklist plus the memory map you publish.

Exercises

1. Elaborate SVM-8 with one lane and with four lanes. Show that `fill8` matches and that the cycle counts differ by about four on a long quiet-bus run.

2. Add a video master that steals every other bus cycle. Show that `fill8` still matches.

3. Implement `vill` for SEW=16 on SVM-8. Write the test that reads both the trap and the CSR.

4. Add widening multiply on SVM-16. Produce a thirty-two-bit product of two sixteen-bit lanes and a narrowing shift back to sixteen bits with saturation. Compare with a scalar reference.

5. Implement Xsvm's cross product on a second BX-16 image. Show that the compatibility image traps on that opcode and still passes the travelling suite.

6. Bring FP32 up on BX-32 for `vfmacc.vv` at SEW=32. Publish lanes of integer work and lanes of floating-point work separately if they differ.

7. Take the SVM-16 `vwmul` test binary and run it, unchanged, on BX-32 and on BX-64.

8. Implement `vstart` for a fault in the middle of a unit-stride load on BX-64. Show a restart that retires each element once.

9. Draw the scoreboard for three vector register numbers and a pending load. List the pairs that may overlap and the pairs that must not.

10. Write the `paramsOk` function and a table of ten illegal parameter records that must not elaborate, including FP32 with ELEN 16 and `claimV` with VLEN 64.

11. The published R800 map leaves `ED 00` through `ED 3F` empty, and also `ED 80` through `ED 9F`. Say why this book freezes `ED 00`, and name one other empty slot you would refuse because a multiply or a block move already owns its row.

12. Name the Amiga software path that executes an A-line word, and say what a board does to existing libraries if the adaptor claims that word.

13. Rewrite `copy8` at SEW 16. List every line that changes in the RVV source. List what changes in the Intel column of the kernel table when the register grows from `mm` to `xmm`.

14. Draw the adaptor record for the NEC V810: escape, parcel endian, which registers `rs1` may name, and which exception fires. Do not invent a bit pattern you have not checked against the V810 opcode map. The record is the exercise. A frozen escape is not.

Glossary

BX-8
MSX-compatible FPGA machine. RV32I, eight-bit data path, SVM-8.

BX-16
Amiga-compatible FPGA machine. RV32I, sixteen-bit data path, SVM-16.

BX-32
RV32 machine with Zve32f.

BX-64
RV64 machine with Zve64d or V.

BX-128
RV64 machine. ELEN 64. VLEN pinned at 128. The integer register stays 64 bits.

BX-512
RV64 machine. ELEN 64. VLEN pinned at 512.

Datapath width
Width of the integer ALU and the data bus. Not XLEN.

ELEN
Widest vector element the implementation supports.

Profile
A named, unratified subset of RVV behaviour defined by this book: SVM-8 or SVM-16.

RV32
Frozen integer base used by BX-8, BX-16, and BX-32. XLEN 32. Thirty-two GPRs, 128 bytes.

RV64
Frozen integer base used by BX-64, BX-128, and BX-512. XLEN 64. Thirty-two GPRs, 256 bytes.

RV128I
Unfrozen integer base, version 1.7. XLEN 128. A flat 128-bit address. ELEN stays 64 if a vector unit is attached. BX-128 is the VLEN pin on an RV64 hart.

SEW
Selected element width, held in `vtype`. 8, 16, 32, or 64 in this book. The Intel opcode names the same choice: `paddb`, `paddw`, `addps`, `addpd`.

SVM
Scalar–vector–matrix. The scalar hart plus an RVV profile; the matrix operations are Xsvm and are optional.

VLEN
Number of bits in one of the thirty-two vector registers.

VLMAX
VLEN/SEW at LMUL 1. The most elements one vector register holds.

XLEN
Width of an integer register. 32 until BX-64, then 64. 128 only on the RV128I base.

Xsvm
Optional custom-0 geometry instructions. Not part of a profile. Off on a V-claiming image.

Matrix
Memory. Column-major. Element (i, j) is at base $+i + j \times$ lda. Not a vector register, and not an LMUL group.

Lane
The slice of every vector register, and the functional unit beside it, that retires one element group per cycle.

Chaining
Starting a consumer vector instruction on the first ready element group while the producer is still writing later groups.

Strip mining
The software loop that asks `vsetvl` again until an arbitrary element count has been consumed.

Host escape
The R800, 68000, SuperH, or MIPS bit pattern that introduces one 32-bit OP-V parcel.

References

1. RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture.* Vector Extension, version 1.0, and the embedded vector subsets Zve32x, Zve32f, Zve64x, Zve64f, and Zve64d.

2. RISC-V International. *Volume II: Privileged Architecture.* Machine ISA register, traps, and the illegal-instruction exception.

3. M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, L. Benini. Ara: A 1 GHz+ scalable RISC-V vector processor with multi-precision floating-point support in 22 nm FD-SOI. *IEEE Transactions on VLSI Systems*, 2020.

4. M. Perotti, M. Cavalcante, and others. Ara2: an RVV 1.0 vector processor, and the multi-core study built on it. *IEEE Transactions on Computers*, 2024.

5. M. Perotti, S. Riedel, M. Cavalcante, L. Benini. Spatz: clustering compact RISC-V vector units. arXiv:2309.10137.

6. Vicuna and Vicuna 2. Open Zve32x, and later Zve32f, coprocessors using the CORE-V extension interface. `vproc` project.

7. MSX technical handbooks and datapacks, for the peripheral contract of BX-8.

8. Amiga hardware references for the chipset bus, blitter, and display fetch, for the peripheral contract of BX-16.

9. Bluespec SystemVerilog language reference, for the rule semantics assumed in Chapter 17.

10. T. M. Tran, P. Miller, and J. McLeod, *RISC-V Vector Primer*, Simplex Micro, 2025. External reading. This book does not reproduce it. The primer is published under CC BY-NC-ND 4.0.

11. Z80 and R800 opcode map, including the empty `ED` slots and the R800 multiply encodings, as published by the MSX Assembly Page.

12. Motorola, *M68000 Family Programmer's Reference Manual*, for the F-line coprocessor word and the coprocessor identity field.

13. Hitachi / Renesas, *SH-4 Software Manual*, and the SH-2 instruction grouping it is upward compatible with, for the 16-bit instruction word and the `Fxxx` FPU space.

14. MIPS instruction encoding for COP3, as implemented by the R3000A, the VR4300, and the Emotion Engine, none of which assign COP3.

15. Intel, *Intel 64 and IA-32 Architectures Software Developer's Manual*, Volume 1, for the MMX, SSE, AVX, and AVX-512 programming environments, and for the mnemonics in the kernel tables. The register widths in those chapters are the VLEN analogues in Chapter 7: `mm` is 64 bits, `xmm` is 128, `ymm` is 256, and `zmm` is 512.

16. Arm, *Arm Architecture Reference Manual* for A-profile, for the NEON, SVE, and SME mnemonics in the kernel tables.

17. The GEMM note of 28 September 2023, `https://tonpa.guru/stream/2023/2023-09-28%20GEMM%20AVX512.htm`. `gemm32` and `gemm64` are that note's rank-1 update at VLEN 128. The note's 24 by 8 tile is an AVX-512 register file.

18. M. Farrar. Striped Smith–Waterman speeds database searches six times over other SIMD implementations. *Bioinformatics* 23(2), 2007. `sw8` is the saturating add and the unsigned max of one stripe.

19. RISC-V International. *The RISC-V Instruction Set Manual, Volume I*, RV128I Base Integer Instruction Set, Version 1.7, and the C extension's RV128 forms. The RV128I chapter states that it is not frozen. This book uses it for the register width, the `*W` and `*D` sign extension, the seven-bit shift amount, and the major opcodes of `LDU`, `LQ`, and `SQ`.

20. QEMU's RISC-V decoder, `target/riscv/insn32.decode`. The funct3 values of `lq`, `sq`, and `ldu`, and the major opcodes `1011011` and `1111011` for the `*D` instructions, are that decoder's placement of the unfrozen chapter. They are not a ratified encoding.

The BX notes `btron-bx8`, `btron-bx16`, `btron-bx32`, `btron-bx64`, and `btron-bx-series` record the same ladder. Where an older paragraph in those notes speaks of a private sixteen-bit vector encoding, this book is the later contract: standard OP-V, thirty-two vector registers, and the profile card.